

UNIP

UNIVERSIDADE PAULISTA

Programação Orientada a Objetos com C#

Autor: Prof. Tarcísio de Souza Peres

Colaboradores: Prof. Antonio Palmeira de Araújo Neto
Profa. Christiane Mazur Doi

Professor conteudista: Tarcísio de Souza Peres

Tarcísio de Souza Peres é formado em Engenharia da Computação e mestre pela Universidade Estadual de Campinas (Unicamp). Atualmente, é doutorando pelo Departamento de Letras Clássicas e Vernáculas da Faculdade de Filosofia, Letras e Ciências Humanas da Universidade de São Paulo (USP). Atuou como pesquisador no Human Cancer Genome Project. Tem MBA em Gestão de TI pela Faculdade de Informática e Administração Paulista (Fiap) e especialização em Gestão de Projetos pela Fundação Instituto de Administração (FIA/USP). Coursou Gestão Estratégica e Competitividade pela Fundação Dom Cabral. Com perfil multidisciplinar, tem experiência nas áreas de tecnologia, saúde, governança, novos negócios e inovação. Trabalhou em multinacionais e órgãos públicos. Possui certificado pelo PMI e pelo Cobit. É autor de livros sobre mercado financeiro, comunicação e liderança, programação orientada a objetos básica e programação avançada em C#, desenvolvimento de software para a internet com ASP.Net, gestão de projetos ágeis, engenharia de requisitos, pensamento lógico computacional com Python e algoritmos, e estrutura de dados com Python. Atua como empreendedor (startups de inovação em tecnologia). É professor na Universidade Paulista (UNIP), na Fundação Getulio Vargas (FGV) e no Centro Paula Souza (CPS).

Dados Internacionais de Catalogação na Publicação (CIP)

P437p Peres, Tarcísio de Souza.

Programação Orientada a Objetos com C# / Tarcísio de Souza Peres. – São Paulo: Editora Sol, 2026.

224 p., il.

Nota: este volume está publicado nos Cadernos de Estudos e Pesquisas da UNIP, Série Didática, ISSN 1517-9230.

1. C#. 2. LINQ. 3 .NET I. Título.

CDU 681.3.062

U524.33 – 26

Prof. João Carlos Di Genio
Fundador

Profa. Sandra Rejane Gomes Miessa
Reitora

Profa. Dra. Marília Ancona Lopez
Vice-Reitora de Graduação

Profa. Dra. Marina Ancona Lopez Soligo
Vice-Reitora de Pós-Graduação e Pesquisa

Profa. Dra. Claudia Meucci Andreatini
Vice-Reitora de Administração e Finanças

Profa. M. Marisa Regina Paixão
Vice-Reitora de Extensão

Prof. Fábio Romeu de Carvalho
Vice-Reitor de Planejamento

Prof. Marcus Vinícius Mathias
Vice-Reitor das Unidades Universitárias

Profa. Silvia Renata Gomes Miessa
Vice-Reitora de Recursos Humanos e de Pessoal

Profa. Laura Ancona Lee
Vice-Reitora de Relações Internacionais

Profa. Melânia Dalla Torre
Vice-Reitora de Assuntos da Comunidade Universitária

UNIP EaD

Profa. Elisabete Brihy
Profa. M. Isabel Cristina Satie Yoshida Tonetto

Material Didático

Comissão editorial:

Profa. Dra. Christiane Mazur Doi
Profa. Dra. Ronilda Ribeiro

Apoio:

Profa. Cláudia Regina Baptista
Profa. M. Deise Alcantara Carreiro
Profa. Ana Paula Tôrres de Novaes Menezes

Projeto gráfico:

Prof. Alexandre Ponzetto

Revisão:

Hélio Iraha
Fernanda Felix

Sumário

Programação Orientada a Objetos com C#

APRESENTAÇÃO	7
INTRODUÇÃO	9

Unidade I

1 FUNDAMENTOS DE C#	11
1.1 Sintaxe, tipos, nullability, var, organização de solução/projeto	14
1.2 Ferramentas: Visual Studio 2026 e dotnet CLI (estrutura .csproj, SDK-style)	29
2 POO: PILARES E ENCAPSULAMENTO NA PRÁTICA	64
2.1 Encapsulamento, abstração, herança e polimorfismo: visão geral	68
2.1.1 Encapsulamento	68
2.1.2 Abstração	73
2.1.3 Herança	80
2.1.4 Polimorfismo	85
2.2 Encapsulamento na prática: propriedades, invariantes, imutabilidade com record	94

Unidade II

3 ABSTRAÇÃO E INTERFACES	105
3.1 Classes e membros abstratos, ocultação de detalhes e contratos	106
3.2 Interfaces, implementação explícita, substituíbilidade (LSP) com exemplos	113
4 HERANÇA E POLIMORFISMO	119
4.1 Hierarquias, abstract/sealed, composição como alternativa	120
4.2 Despacho virtual: virtual/override, polimorfismo por interface	126

Unidade III

5 COLEÇÕES E GENÉRICOS	141
5.1 List<T>, Dictionary<TKey,TValue>, iteração e IEnumerable<T>	142
5.2 Genéricos: restrições, igualdade/ordenação e boas práticas	150
6 LINQ ESSENCIAL	157
6.1 Where/Select/GroupBy/Any, projeções e diferimento de execução	166
6.2 IEnumerable<T> x IQueryable (noções para acesso a dados)	169

Unidade IV

7 ERROS E TESTES.....	177
7.1 Exceções, hierarquia, tratamento e criação de exceções.....	181
7.2 Testes com xUnit e uso do Gerenciador de Testes.....	186
8 MEMÓRIA E CONCORRÊNCIA INTRODUTÓRIA.....	205
8.1 Valor x referência, GC, struct x class.....	206
8.2 Task, async/await, cancelamento com CancellationToken	208

APRESENTAÇÃO

Olá, aluno!

A disciplina *Programação Orientada a Objetos com C#* (lê-se C sharp) foi concebida para formar desenvolvedores capazes de construir softwares de modo claro, confiável e sustentável no ecossistema .NET (lê-se dot net). O ponto de partida é simples: quem domina a linguagem de programação, a organização de projetos e os princípios de programação orientada a objetos (POO) consegue transformar requisitos de negócio em código-fonte que poderá evoluir sem grandes dificuldades, que suportará revisões frequentes e que facilitará a colaboração em equipe. Por isso, articulamos conteúdos que vão da sintaxe do C# à modelagem de tipos, do uso criterioso de coleções e LINQ (Language-Integrated Query) ao tratamento de erros e testes, chegando aos fundamentos de memória e a uma introdução segura a tarefas assíncronas. A proposta da disciplina não é que o leitor decore comandos isolados, mas que compreenda os porquês das escolhas técnicas e como elas impactam a prática profissional.

Você aprenderá a criar e navegar por soluções e projetos no Visual Studio 2026 e na dotnet CLI, entendendo o papel de arquivos como .sln e .csproj em estilo SDK (Software Development Kit). Esse objetivo é crucial para a vida real de um programador: equipes modernas versionam repositórios inteiros, automatizam builds e rodam testes continuamente; quem lê e estrutura projetos com segurança reduz atrito na integração diária, abre caminho para os chamados pipelines de entrega e ganha autonomia para isolar problemas ou adicionar módulos sem quebrar o que já existe. No mesmo bloco, definiremos sintaxe, tipos e inferência com var, ativaremos recursos de nullability e alinharemos convenções que promovem legibilidade. O ganho profissional é imediato: ao assumir um código legado ou iniciar um produto, você enxerga o mapa do projeto, identifica dependências e evita erros previsíveis de referência nula antes que cheguem à produção.

Também abordaremos os pilares de POO – encapsulamento, abstração, herança e polimorfismo – sempre atrelados a boas práticas. O objetivo aqui é fazê-lo projetar tipos que reflitam o domínio do problema, protegendo invariantes e reduzindo acoplamentos desnecessários. Encapsular com propriedades e validações adequadas preserva a integridade do estado da aplicação e torna o comportamento dos objetos mais previsível; para a formação profissional, isso significa menos bugs difíceis de rastrear, código mais testável e menor custo de manutenção.

A imutabilidade entra como estratégia pragmática para representar valores de negócio com igualdade estrutural e sem efeitos colaterais ocultos. Essa decisão de design tem reflexos diretos em testes, em coleções que dependem de chaves estáveis e na clareza das intenções do código durante revisões.

Quando avançamos para abstrações e interfaces, o objetivo formativo é treinar o pensamento por contratos. Ao definir interfaces com precisão – inclusive com implementação explícita quando convém –, você separa o que é compromisso público do que é detalhe interno e conquista liberdade para trocar mecanismos sem alterar o restante do sistema. No ambiente profissional, contratos estáveis são a base para integrações entre times e para uma evolução de software que não interrompa entregas.

Herança e composição são avaliadas com rigor técnico. Projetar uma hierarquia abstrata pode reduzir duplicação, enquanto compor objetos frequentemente diminui acoplamento e facilita testes. O objetivo é que você diferencie cenários em que o reuso por herança compensa dos casos em que ele cria árvores rígidas e frágeis. No polimorfismo, o despacho virtual (virtual/override) e a distribuição por interface são praticados com exemplos em que a extensibilidade faz diferença, como selecionar estratégias de cálculo ou rotas de processamento. O vínculo com o mercado é claro: decisões acertadas aqui encurtam ciclos de refatoração, tornam a base de código convidativa para contribuições e evitam regressões em funcionalidades críticas.

Você aprenderá sobre o uso de coleções e genéricos. Escolher a estrutura correta não é detalhe, é ela que influencia custo de buscas, garantia de unicidade e desenho de APIs (Interfaces de Programação de Aplicativos).

O LINQ é abordado como instrumento expressivo para consultas em memória e, com noções cuidadosas, em cenários de dados. O objetivo é ensinar a formular comandos com clareza, entender projeções e, principalmente, dominar o diferimento de execução. Ao compreender quando materializar resultados e por que reiterações podem reprocessar fontes custosas, você adquire o hábito de raciocinar sobre desempenho e evitar problemas sutis que só aparecem em volumes reais.

O tratamento de erros e testes fecha o arco de qualidade. Ao mapear a hierarquia de exceções, criar tipos específicos quando necessário e capturar de forma direcionada, você aprende a sinalizar condições excepcionais com mensagens úteis, preservando contexto para análise posterior. Em paralelo, o uso do xUnit e do Gerenciador de Testes organiza a validação automática do comportamento, com casos de borda e cenários negativos que tendem a revelar falhas reais.

Os fundamentos de memória entram para esclarecer decisões que afetam desempenho e previsibilidade. Entender a diferença entre tipos por valor e por referência, escolher entre struct e class, evitar boxing desnecessário e reconhecer o papel do coletor de lixo prepara você para escrever código que utiliza recursos de maneira consciente. Na prática profissional, esse conhecimento se traduz em APIs mais claras, consumo de memória mais estável e métricas de execução que inspiram confiança.

A introdução à concorrência cumpre um objetivo específico: permitir que operações I/O-bound sejam modeladas sem bloqueios. Mesmo em caráter introdutório, esse conteúdo prepara para ambientes conectados, serviços web e integrações que demandam resiliência, algo cada vez mais presente em equipes que mantêm APIs e back-ends em produção.

Ao final dessa jornada, os objetivos gerais e específicos se convertem em competências que o mercado valoriza: ler e estruturar projetos .NET de forma consistente; aplicar POO para modelar regras de negócio com encapsulamento e invariantes; empregar coleções e genéricos de modo eficiente, com igualdade e ordenação corretas; escrever consultas LINQ idiomáticas e conscientes de custo; tratar erros com exceções adequadas e sustentar uma suíte de testes que dá segurança a refatorações; tomar decisões informadas sobre memória e nullability; e implementar tarefas assíncronas simples com cancelamento efetivo. Cada uma dessas metas foi escolhida por sua relevância direta para a prática profissional, pois afeta a velocidade de entrega, a estabilidade do sistema e a experiência do time em manutenção e evolução contínuas.

Assim, esta disciplina busca formar desenvolvedores que entendem o que escrevem, conseguem explicar suas escolhas e mantêm a porta aberta para mudanças futuras sem que todo o código tenha que ser reescrito.

INTRODUÇÃO

Programar com a linguagem C# no ecossistema .NET significa transformar regras de negócio em estruturas de código legíveis, testáveis e duráveis. Este livro-texto inicia essa jornada pela organização de soluções: esclarecendo a função dos arquivos .sln e .csproj, ensinando como configurar o projeto no Visual Studio 2026 e como habilitar recursos como nullability para que contratos fiquem explícitos desde o compilador. Essa base técnica sustenta práticas profissionais importantes, como o versionamento consistente, os builds reproduzíveis e a integração tranquila a repositórios já existentes.

A orientação a objetos fornece a gramática de design que guia as próximas decisões. O encapsulamento protege invariantes e torna previsível o comportamento do objeto; a abstração separa o "o quê" do "como", permitindo que detalhes evoluam sem a necessidade de reescrever o código-fonte; a herança e a composição são avaliadas com critério para evitar as hierarquias frágeis; e o polimorfismo, por despacho virtual ou por interfaces, organiza variações de comportamento sem espalhar condicionais no código-fonte. No cotidiano de trabalho, esse repertório torna o código mais fácil de revisar, favorece as refatorações graduais e reduz o custo de manutenção quando requisitos mudam.

Veremos como as coleções e os genéricos compõem o conjunto de ferramentas do dia a dia. Escolher entre `List<T>`, `Dictionary<TKey,TValue>` e `HashSet<T>` define os custos de busca, a ordenação e a unicidade; implementar `IEquatable<T>` e, quando necessário, `IComparable<T>` ou `IComparer<T>` garante coerência em chaves e ordenações; declarar restrições genéricas captura expectativas em tempo de compilação. Em equipes que lidam com integrações e processamento de dados, essas escolhas sustentam desempenho adequado e evitam surpresas sob carga real.

Na sequência, estudaremos como o LINQ se estabelece como uma linguagem de consulta que favorece clareza e intenção. Filtrar, projetar e agrupar com `Where`, `Select`, `GroupBy` e `Any` exige atenção ao diferimento de execução e ao momento certo de materializar resultados. Entender a diferença entre `IEnumerable<T>` e `IQueryable` evita confusões entre operações executadas em memória e consultas traduzidas por provedores de dados. Na prática, isso se manifesta em pipelines transparentes, em diagnósticos mais rápidos e na comunicação técnica objetiva com áreas que cuidam de persistência.

A confiabilidade depende de tratamento de erros consciente e testes automatizados. Mapear a hierarquia de exceções, criar tipos adequados quando fizer sentido e registrar mensagens informativas facilita o diagnóstico e encurta o tempo de correção. Com `xUnit` e o Gerenciador de Testes, casos representativos se tornam verificação executável do comportamento desejado, dando segurança para evoluir o código sem regressões. Esse hábito profissional reduz o custo de mudanças e dá base para cadência de entrega estável.

No tópico final, será possível apreender como os conhecimentos de memória fecham lacunas conceituais essenciais: as diferenças entre tipos por valor e por referência, o papel do coletor de lixo, as

implicações de struct e class, as anotações de nulabilidade e os riscos de boxing. Ao compreender como os objetos ocupam recursos e como as cópias acontecem, o desenvolvedor decide com mais consciência sobre a modelagem e sobre os efeitos de cada chamada em caminhos críticos do sistema.

O uso de Task e async/await, acompanhado de cancelamento com CancellationToken, prepara o código-fonte para fluxos I/O-bound que caracterizam serviços conectados. Escrever operações assíncronas que cooperam com o ambiente, respondem a cancelamentos e preservam responsividade é requisito frequente em APIs e integrações. Mesmo em caráter inicial, o domínio desses elementos amplia a capacidade de atuar com segurança em cenários reais.

Esses conteúdos se articulam para formar uma competência profissional integrada: organizar projetos .NET com clareza, modelar com POO respeitando contratos, invariantes, manipular dados com coleções e LINQ de modo responsável, tratar erros com rigor, validar comportamento por testes, tomar decisões informadas sobre memória e aplicar código assíncrono em contextos adequados. O resultado esperado é a capacidade de ingressar em bases de código existentes, evoluir funcionalidades com previsibilidade e dialogar tecnicamente com o time, sempre com foco no software que se mantém estável à medida que cresce.

Desejamos uma ótima leitura!

Unidade I

1 FUNDAMENTOS DE C#

Este tópico aborda os conceitos fundamentais da linguagem C#, incluindo sua sintaxe básica, tipos de dados e aspectos de nulabilidade, bem como noções de organização de projetos e uso de ferramentas de desenvolvimento modernas (Visual Studio 2026 e a interface de linha de comando do .NET, conhecida como dotnet CLI). Será considerada a versão 14 do C# e o ambiente .NET 10, contextualizando-se as explicações com exemplos de código comentados para ilustrar os conceitos.

C# é uma linguagem de programação criada pela empresa Microsoft. Ela serve para construir muitos tipos de software: sites, serviços em nuvem, programas de computador, aplicativos para celular e jogos. É moderna, com checagem de tipos (ajuda a evitar erros) e gerenciamento automático de memória, além de ter recursos de produtividade como `async/await` e LINQ. C# é a linguagem principal da plataforma .NET.

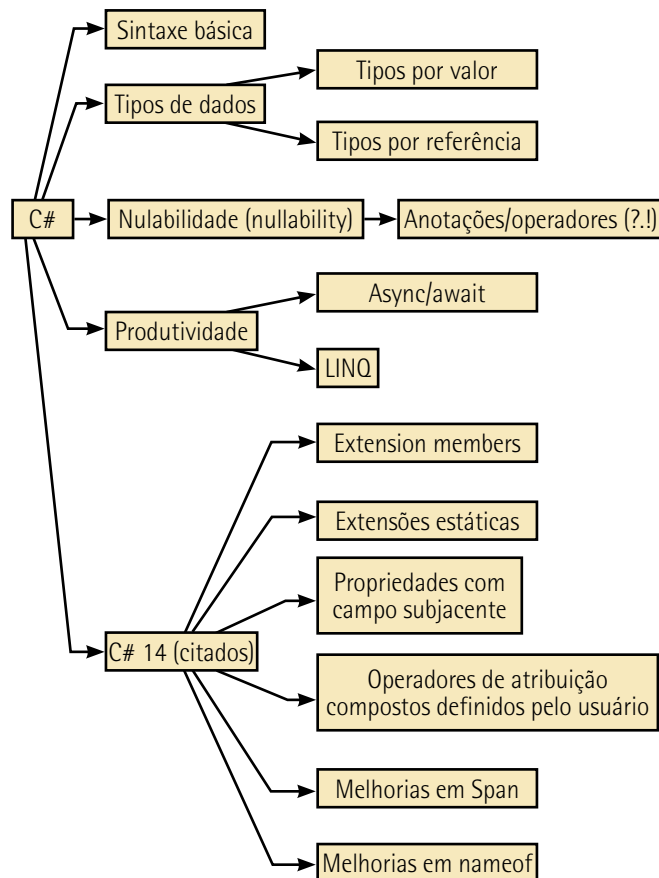


Figura 1 – Fundamentos da linguagem e novidades do C# 14

.NET é a plataforma que executa programas escritos em C#. A edição .NET 10 é o lançamento de 2025 (ciclo anual da Microsoft) e é um LTS (Long Term Support), ou seja, recebe correções e suporte por três anos. Nessa versão há ganhos de desempenho no runtime, novidades nas bibliotecas, atualizações no SDK e suporte ao C# 14 (por exemplo, propriedades com field, melhorias em nameof, extensões estáticas).

No *2025 Stack Overflow developer survey* (Stack Overflow, 2025), C# aparece entre as linguagens mais usadas (em 2025, 27,8% dos respondentes disseram ter trabalhado com C# no último ano). No universo web, segundo a mesma pesquisa, ASP.NET Core também figura entre as tecnologias populares. Além disso, o Unity, um dos motores de jogos mais difundidos no mundo, usa C# como linguagem de script, o que abre portas em games e 3D.

Começar a carreira com C# oferece uma base técnica versátil e coerente que se transfere de um domínio a outro sem ruptura de paradigma. A mesma linguagem sustenta o desenvolvimento de back-end web com ASP.NET Core, plataforma madura e multiplataforma para serviços e APIs, o que abre portas para projetos de alto desempenho e implantação em Linux ou Windows. Essa base se estende naturalmente ao ecossistema de nuvem da Microsoft, no qual o Azure fornece SDKs e guias específicos para .NET, viabilizando desde web apps até funções serverless com integração por pacotes NuGet. No desktop, tutoriais e modelos de projeto de Windows Forms e outras stacks permitem construir aplicações tradicionais com rapidez. Em dispositivos móveis e desktop modernos, o .NET MAUI centraliza a UI (User Interface) em um único framework para Android, iOS, macOS e Windows, preservando código compartilhado e produtividade. Já no universo de jogos, a engine Unity adota C# para scripts e arquitetura de gameplay, o que amplia as saídas profissionais sem abandonar a linguagem inicial. Esses vetores mostram que uma única proficiência abrange web, nuvem, desktop, mobile e games sem exigir requalificação completa.

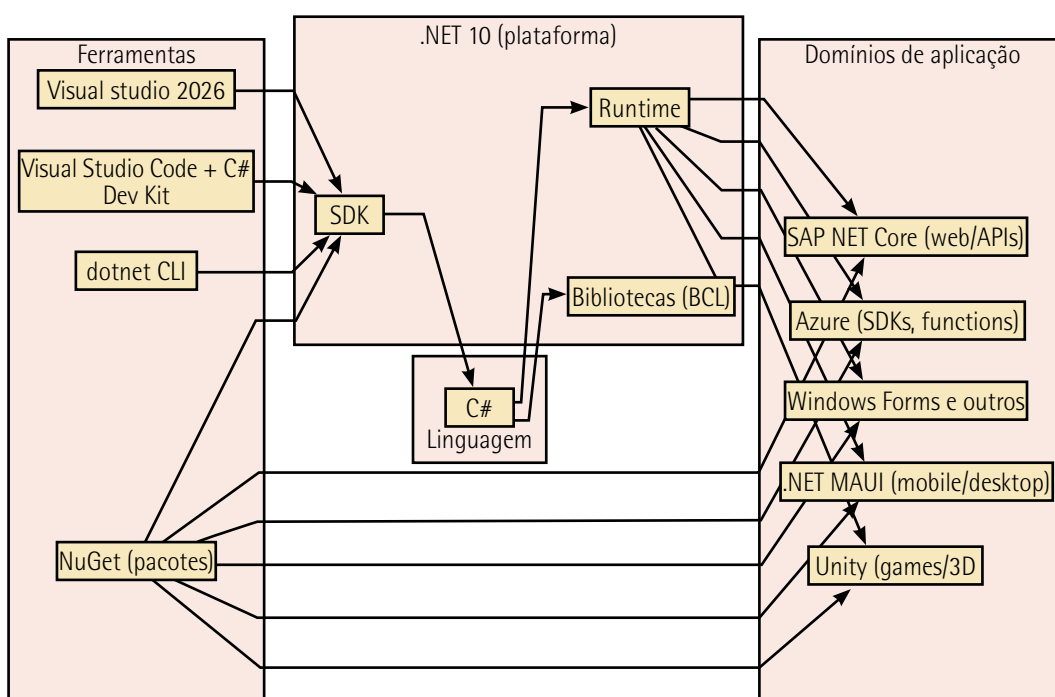


Figura 2 – Mapa do ecossistema

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

O cotidiano de trabalho de um desenvolvedor também é favorecido por ferramentas de alto nível. A família Visual Studio oferece uma IDE (Integrated Development Environment) completa com editor, depurador e designers, ao passo que o Visual Studio Code, com C# Dev Kit, fornece um ambiente leve e multiplataforma; ambos se alinham ao fluxo de projetos .NET, testes e publicação. A gestão de dependências é integrada pelo NuGet, repositório e conjunto de clientes que simplificam instalar, atualizar e versionar bibliotecas, reduzindo atrito operativo e padronizando builds. Esse conjunto encurta o caminho entre protótipo e produção e dá previsibilidade aos pipelines.

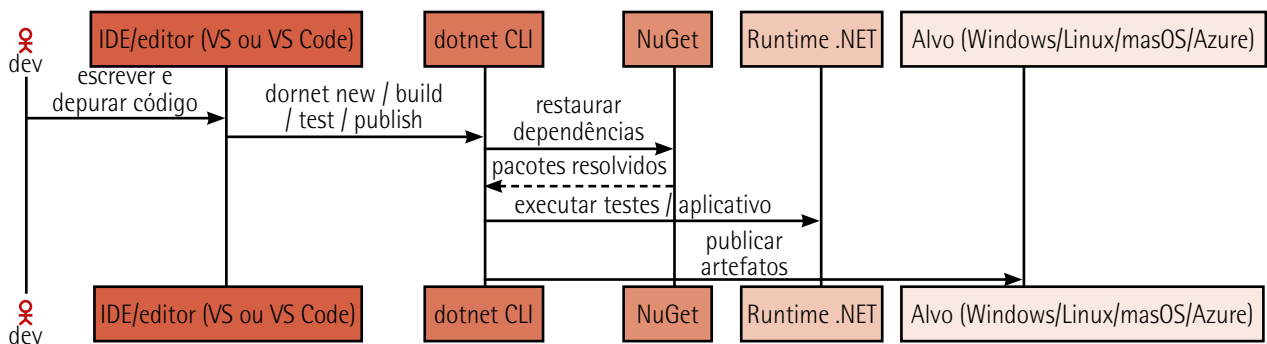


Figura 3 – Diagrama de sequência para um pipeline de desenvolvimento típico

No plano de desempenho e de recursos modernos, o runtime do .NET evolui em ciclos anuais, combinando melhorias de JIT (just-in-time) com opções de compilação AOT (ahead-of-time). As versões recentes documentam avanços substanciais em geração de código, coletor de lixo (garbage collector, GC) e rede, que aparecem sem custos adicionais quando o aplicativo opera sobre a runtime atual. Quando necessário, a estratégia Native AOT permite publicar binários nativos sem JIT, úteis em ambientes restritos e de cold start sensível. No nível da linguagem, o C# 14 – suportado no .NET 10 – reduz código cerimonial com recursos como extension members, propriedades com campo subjacente, operadores de atribuição compostos definidos pelo usuário e melhorias em Span<T>, o que ajuda a escrever um código mais conciso e seguro sem sacrificar performance.

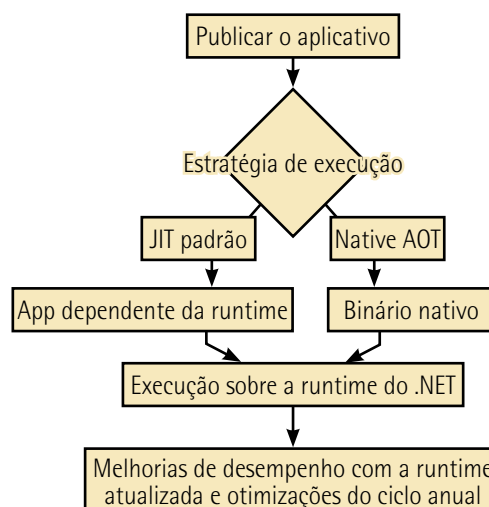


Figura 4 – Formas de execução/publicação de código

A estabilidade de longo prazo é outra razão prática para adotar C# cedo. A Microsoft mantém um calendário público de versões, com lançamentos anuais em novembro e ciclos de suporte bem definidos. O .NET 8 é LTS ativo, e o .NET 10 também é LTS, oferecendo janela estendida de atualizações, o que dá às empresas previsibilidade para planejar migrações e aos iniciantes segurança para aprender sobre uma base que permanecerá suportada.

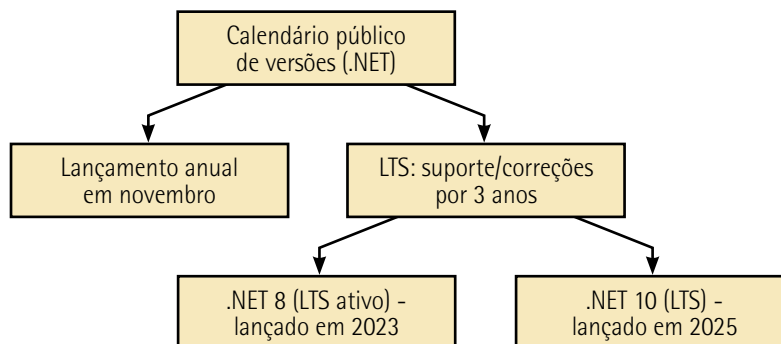


Figura 5 – Calendário e suporte LTS

Por fim, a curva de aprendizado é amparada por documentação oficial extensa e gratuita. O **Microsoft Learn** (Documentação..., s.d.) reúne guias, tutoriais, referências de linguagem e arquiteturas de aplicação para .NET e Azure, cobrindo do básico a tópicos avançados. Esse acervo reduz a dependência de conteúdos dispersos, orienta boas práticas desde o início e acelera a transição do estudo para projetos reais. Para iniciantes, C# se mostra uma escolha pragmática e eficiente, pois entrega uma plataforma de amplo alcance, ecossistema de ferramentas maduro, evolução contínua do runtime e da linguagem, suporte LTS e documentação de referência abrangente.

1.1 Sintaxe, tipos, nullability, var, organização de solução/projeto

A linguagem C# pertence à família das linguagens derivadas de C, possuindo uma sintaxe bastante familiar para quem já utilizou C, C++, Java ou JavaScript. Assim como nessas linguagens, em C# cada instrução termina com **ponto e vírgula (;)** e blocos de código são delimitados por **chaves { e }**. Os **identificadores** (nomes de variáveis, métodos, classes etc.) são **case-sensitive**, ou seja, diferenciam maiúsculas de minúsculas – por exemplo, Aluno e aluno seriam dois identificadores distintos. C# inclui as estruturas de controle usuais, como `if/else` para condicionais, `switch` para seleção de casos e laços de repetição `for`, `while` e `foreach`, com sintaxes muito semelhantes às do C/C++. Essa familiaridade sintática torna a linguagem acessível a desenvolvedores já experientes em outras linguagens imperativas, ao mesmo tempo que mantém consistência e rigor na definição de escopos e na terminação de comandos.

Como exemplo introdutório, podemos observar o clássico programa "Hello, World". Em versões modernas do C# (a partir do C# 9), é possível escrever um programa completo utilizando instruções de nível superior (top-level statements), sem a necessidade explícita de declarar uma classe `Program` ou um método `Main`. O código a seguir demonstra esse estilo minimalista:

```
// Programa "Olá, Mundo" utilizando instruções de nível superior em C#  
Console.WriteLine("Olá, Mundo!");
```

Esse código, embora simples, já é um programa C# válido. A linha iniciada com `//` é um comentário de única linha – comentários em C# começam com `//` e se estendem até o fim da linha, servindo para documentar ou desabilitar partes do código. Nesse exemplo, o comentário indica o propósito do programa. A linha seguinte usa a classe estática `Console` (definida no namespace `System`) e seu método `WriteLine` para imprimir a mensagem **"Olá, Mundo!"** no console. Graças às `using` directives implícitas habilitadas pelos modelos de projeto atuais, não foi necessário escrever `using System;` no início do arquivo – por padrão, o namespace básico `System` e alguns outros já estão disponíveis no escopo global em aplicações de console modernas.

Internamente, ao compilar esse código, o compilador C# irá gerar automaticamente uma classe de suporte e um ponto de entrada **Main** que encapsulam essas instruções de nível superior. Em outras palavras, esse exemplo é equivalente a um programa escrito da forma tradicional, como a seguir:

```
using System;  
  
namespace Exemplo {  
    class Program {  
        static void Main() {  
            // Imprime "Olá, Mundo!" no console  
            Console.WriteLine("Olá, Mundo!");  
        }  
    }  
}
```

No código tradicional, começamos declarando um namespace (**Exemplo**) para **organizar logicamente** a classe `Program`. Dentro da classe, definimos o método **Main** com o modificador `static`, indicando que é um método de classe (e não de instância). O método **Main** sem parâmetros é convencionalmente o ponto de entrada de programas C# executáveis – ou seja, é a primeira função a ser invocada quando o programa é iniciado. Nele, fazemos a chamada `Console.WriteLine` para produzir saída no console.



Observação

Saída no console é o texto que um programa envia para a tela do terminal, logo, a janela de letras na qual digitamos comandos. Em sistemas modernos, esse terminal pode ser o Prompt de Comando ou o PowerShell (no Windows), o aplicativo Terminal (no macOS e no Linux) e o terminal integrado (em IDE/editores), como o do Visual Studio Code.

Tecnicamente, essa saída é o fluxo de saída padrão (standard output, ou stdout). Na prática, tudo o que o programa "imprime" aparece como linhas de texto nesse terminal.

A utilização de **using System**; no topo do arquivo permite invocar `Console.WriteLine` diretamente, em vez de ser necessário escrever `System.Console.WriteLine` toda vez; o **using** importa o namespace de forma que seus membros possam ser referenciados de forma não qualificada. Em suma, seja adotando o novo recurso de instruções de nível superior ou a estrutura clássica com classe e método **Main**, ambos os formatos produzem o mesmo resultado e envolvem os mesmos conceitos fundamentais de sintaxe e organização de um programa em C#.

Outro ponto importante da sintaxe C# é o suporte a **comentários** de múltiplas linhas, iniciados por `/*` e terminados somente quando o compilador encontra a sequência `*/`. Esses comentários multilinha permitem documentar trechos maiores de código ou adicionar explicações que abrangem mais de uma linha, sem precisar prefixar `//` em cada linha. Comentários (tanto de linha única quanto multilinha) são ignorados pelo compilador e servem apenas para melhorar a compreensão do código por parte dos desenvolvedores.

Em resumo, a sintaxe básica de C# combina elementos familiares de linguagens C-like (uso de `;` e `{ }`) com algumas facilidades modernas (como omissão opcional de boilerplate em programas simples). Essa sintaxe clara, aliada à obrigatoriedade de declaração de tipos e ao uso de estruturas de controle explícitas, contribui para a legibilidade e a manutenção do código, características valorizadas em contextos acadêmicos e profissionais.



Observação

Boilerplate é o conjunto de partes do código que você repete por obrigação estrutural, não porque resolvam diretamente o problema em si. Pense nisso como o formato que precisa estar lá para que o programa exista e execute declarações, molduras e inicializações que preparam o ambiente, enquanto a lógica útil fica em poucas linhas.

C# é uma linguagem **fortemente tipada**, o que significa que cada variável e constante no código possui um tipo bem definido em tempo de compilação. Toda expressão em C# avalia para um tipo específico, conhecido pelo compilador, e operações envolvendo valores geralmente só são permitidas se os operandos forem de tipos compatíveis. Esse forte sistema de tipos promove a detecção precoce de erros: o compilador (ou mesmo as ferramentas do ambiente de desenvolvimento) emite erros ou avisos caso tentemos usar uma variável de modo inconsistente com seu tipo declarado. Por exemplo, se uma variável for declarada como inteira, o compilador não permitirá que tentemos atribuir a ela um valor de texto (string) sem conversão explícita, garantindo assim maior segurança e robustez ao código. Essa checagem em tempo de compilação ajuda a evitar uma classe de erros comuns e permite corrigir problemas antes mesmo de executar o programa.

A linguagem e a plataforma .NET fornecem diversos tipos de dados fundamentais (built-in), que incluem tipos numéricos, de caractere, booleano e outros. Esses tipos básicos residem em **namespaces** do .NET (principalmente em `System`), mas possuem palavras-chave em C# que os representam.

Algumas dessas palavras-chaves são **int** (em C#), que corresponde ao tipo inteiro de 32 bits assinado (`System.Int32` na plataforma .NET); da mesma forma, **double** é um tipo de ponto flutuante de 64 bits (`System.Double`); **char** representa um caractere Unicode de 16 bits (`System.Char`); **bool** é um valor booleano lógico (`System.Boolean`, que pode ser `true` ou `false`), e assim por diante. O quadro 1 ilustra alguns dos tipos primitivos e seus propósitos.

Quadro 1

Categoria	Detalhamento
Inteiros	<code>sbyte</code> , <code>byte</code> , <code>short</code> , <code>ushort</code> , <code>int</code> , <code>uint</code> , <code>long</code> , <code>ulong</code> – variando em tamanho (8 a 64 bits) e sinal
Ponto flutuante	<code>float</code> (32 bits, precisão simples) e <code>double</code> (64 bits, precisão dupla) para números em ponto flutuante; além de <code>decimal</code> (128 bits) para precisão decimal alta (usado em finanças, por exemplo)
Caractere	<code>char</code> – representa um único caractere Unicode
Booleano	<code>bool</code> – representa um valor lógico verdadeiro ou falso
Texto	<code>string</code> – é uma sequência de caracteres Unicode (embora <code>string</code> seja tecnicamente um objeto de referência, não um tipo por valor)
Objetos base	<code>object</code> – é o tipo que é base de todos os outros tipos em C#, equivalente a <code>System.Object</code> (todas as classes e tipos por valor derivam, direta ou indiretamente, de <code>object</code>)

Um aspecto crucial no sistema de tipos do C#/.NET é a distinção entre tipos por valor e tipos por referência. Essa distinção afeta como os dados são armazenados na memória e como são passados nas atribuições ou nas chamadas de métodos.

Tipos por valor são aqueles que armazenam diretamente seus dados. Isso significa que uma variável de um tipo por valor contém em si o valor atribuído, e não uma referência a um valor em outro lugar. Os tipos numéricos primitivos (`int`, `double` etc.), `bool`, `char` e também os tipos definidos com `struct` (estruturas) são tipos por valor. Por padrão, instâncias de tipos por valor são alocadas na **stack (pilha) de execução** ou embutidas dentro de estruturas de dados maiores. Uma consequência importante é que, ao atribuir uma variável de tipo por valor a outra, realiza-se uma cópia dos dados. Assim, duas variáveis inteiras distintas mantêm valores independentes – modificar uma não afeta a outra, já que cada qual possui sua própria cópia do número. Além disso, tipos por valor não admitem valor nulo, a menos que sejam explicitamente declarados como nullable (veremos adiante sobre tipos anuláveis). Em C#, todos os tipos por valor derivam implicitamente de `System.ValueType`, o que os distingue no tempo de execução da plataforma.

Tipos por referência, por outro lado, armazenam referências (endereços) para objetos alocados na **heap** (área de memória dinâmica). `Class` (classes), `arrays` (vetores) e `string` (cadeia de caracteres) são exemplos de tipos por referência. Quando declaramos uma variável de um tipo por referência, inicialmente ela não aponta para nenhum objeto concreto e seu valor padrão é `null` (nulo), indicando nenhuma referência válida. Para obter um objeto real, geralmente usamos o operador `new` para instanciar a classe, o que aloca memória na heap e retorna uma referência para essa região de memória. A variável então armazena esse ponteiro (de forma segura, sem aritmética de ponteiros explícita na linguagem gerenciada). Ao atribuir uma variável de referência a outra, não copiamos os dados do objeto em si, mas sim a referência (o endereço).



Observação

Instanciar a classe significa criar um objeto concreto a partir de uma classe. Imagine que a classe é a planta de uma casa: ela descreve como a casa deve ser, mas não é uma casa de verdade. Quando você instancia a classe, é como construir uma casa específica usando aquela planta: algo real, que "existe" na memória do computador e que você pode usar.

Dessa forma, após uma atribuição como `objB = objA;`, tanto `objA` quanto `objB` referenciarão o mesmo objeto na memória. Qualquer modificação feita através de `objB` refletirá em `objA` também, se ambos apontam para o mesmo objeto, já que o objeto subjacente é único. Para copiar efetivamente um objeto, seria necessário criar um novo e copiar manualmente os valores relevantes (ou usar mecanismos de clonagem quando disponíveis). Outra característica dos tipos por referência é que eles podem representar a ausência de um valor real através de `null`. Uma variável de referência não inicializada (ou explicitamente setada como `null`) indica que ela não aponta para nenhum objeto válido naquele momento; tentar usar (desreferenciar) uma referência nula em tempo de execução causa a exceção `System.NullReferenceException`.

Em memória de programas, `stack` e `heap` descrevem estratégias distintas de organizar dados ao longo da execução. A primeira se assemelha a uma pilha de pratos: tudo que entra por cima sai por cima, em ordem estritamente last-in, first-out (LIFO). A segunda lembra um almoxarifado: há espaço para itens de tamanhos variados, guardados e recuperados conforme a necessidade, sem seguir a mesma rigidez de ordem. Essas imagens ajudam a entender tanto o uso pretendido quanto o custo operacional de cada área.

A pilha serve de suporte imediato à execução de funções. Quando um procedimento começa, o sistema cria um quadro de ativação que costuma reunir parâmetros, variáveis locais e o endereço de retorno; ao término, esse quadro é descartado integralmente. Essa gestão é automática e baratíssima: basta mover o ponteiro da pilha para crescer ou encolher, o que tende a ser muito rápido e previsível, além de favorecer o uso de cache. Como contrapartida, o espaço é limitado e o padrão LIFO impõe vidas curtas e bem delimitadas aos dados; empilhar estruturas grandes ou encadear chamadas profundas pode resultar em estouro de pilha, e dados com tamanho muito variável ou que precisam persistir além da execução da função não são adequados para esse tipo de armazenamento.

O `heap`, por sua vez, atende a dados cujo tamanho ou duração não cabem nas amarras da pilha. Estruturas dinâmicas, coleções que crescem com a entrada do usuário e objetos que atravessam fronteiras de funções são alocados nessa área. Em ambientes com GC, como a plataforma .NET, o runtime monitora a alcançabilidade dos objetos e libera a memória quando deixam de ser referenciados; em linguagens sem coleta automática, cabe ao programador desalocar no momento correto. A flexibilidade tem custo: a alocação exige metadados e coordenação do alocador, e a liberação depende do coletor ou de disciplina manual. Além disso, referências mantidas desnecessariamente podem causar retenção de memória (ou vazamentos em modelos sem GC), e coletores mal configurados podem introduzir pausas perceptíveis em certos cenários.

Ainda, há sutilezas práticas importantes. Em C#, tipos por valor (structs) podem residir na pilha ou estar embutidos em objetos do heap, enquanto instâncias de tipos por referência vivem no heap gerenciado.

No cotidiano, essa divisão oferece um critério simples de engenharia: a pilha privilegia velocidade e previsibilidade ao preço de limites rígidos; o heap oferece maleabilidade com sobrecusto de gerenciamento. Entender esse equilíbrio ajuda a decidir em que lugar cada dado deve ser armazenado, a evitar estouros de pilha e a reduzir retenções desnecessárias, mantendo programas corretos e com desempenho estável.

Resumindo, a plataforma .NET define as duas categorias de tipos de dados, por valor e por referência, como estratégia de desempenho e organização de memória. Tipos por valor são eficientes e evitam alocação dinâmica quando contêm dados pequenos e imutáveis, mas não podem ser nulos por padrão. Tipos por referência acomodam estruturas de dados mais complexas e de tamanho variável, permitindo compartilhamento de objetos, ao custo de ser preciso gerenciar (indiretamente) o fato de que múltiplas referências podem apontar para o mesmo objeto. A compreensão dessa diferença é fundamental para escrever código C# correto e para entender certos comportamentos sutis, como em conversões de parâmetros para métodos (por valor vs. por referência), comparações de igualdade (== em tipos por valor compara conteúdo, enquanto em tipos por referência compara referência, exceto se o operador for sobrecarregado) ou questões de gerenciamento de memória (objetos referenciados ficam no heap até que o GC os libere quando não há mais referências ativas a eles).

Em C#, é útil distinguir o destino típico dos dados locais e o ciclo de vida dos objetos. Um exemplo como `int x = 5;` ilustra o caso comum de um valor simples associado ao escopo imediato da função: variáveis locais de tipos por valor tendem a residir na pilha durante a execução, o que favorece acesso rápido e descarte automático ao final do bloco. A documentação apresenta justamente essa intuição ao discutir como reduzir alocações ao preferir `struct` em cenários adequados.

Quando a instrução envolve `new`, como em `var pessoa = new Pessoa();`, a instância resultante é criada no heap gerenciado, enquanto a variável local retém apenas uma referência a esse objeto. Essa separação entre a variável que contém o valor (tipo por valor) e a variável que contém uma referência (tipo por referência) é um ponto central da especificação da linguagem.

No ecossistema .NET, a memória do heap é administrada pelo GC. O runtime acompanha a alcançabilidade dos objetos e, quando não existem mais referências ativas, sua memória se torna elegível para liberação; o processo é automático e poupa o desenvolvedor de desalocação manual, evitando erros clássicos de uso após liberação. Para casos estritamente temporários e com tamanho conhecido, a linguagem oferece ainda `stackalloc`, que reserva um bloco na pilha e o descarta integralmente ao retorno do método, fora do escopo do GC.

Há nuances importantes ligadas a otimizações do compilador e do runtime. A alocação efetiva pode ser influenciada por análises do JIT e por recursos recentes do .NET: além de otimizações tradicionais, o .NET 10 introduz melhorias em alocação na pilha para certos padrões (por exemplo, pequenos arrays) e incrementa a análise de escape, o que ajusta o local e a forma das alocações em cenários específicos. Esses detalhes não alteram a regra prática para fins didáticos – como valores locais simples na pilha;

objetos criados com `new` no heap gerenciado, com coleta automática –, mas explicam exceções e ganhos de desempenho observados em versões atuais da plataforma.

Cabe mencionar que, embora tipos por valor não possam ser nulos intrinsecamente, o C# introduziu, desde a versão 2.0, os tipos anuláveis por valor (nullable value types). Usando a sintaxe de interrogação após o tipo – por exemplo, `int?` em vez de `int` –, podemos declarar um tipo por valor que aceite representar também **a ausência de valor** através de `null`. Em termos técnicos, `int?` é um complemento sintático para `System.Nullable<int>`, uma estrutura genérica que encapsula um valor do tipo especificado e um flag indicando se há um valor presente ou não. Assim, `int? x = null;` é válido e inicializa `x` sem um valor inteiro concreto.

Esse recurso foi criado para lidar com situações como campos de banco de dados ou informações opcionais que possam ser desconhecidas ou indisponíveis, aplicando-se apenas a tipos por valor, já que tipos por referência sempre puderam ser nulos. Veremos adiante, porém, que o conceito de anulabilidade foi estendido também aos tipos por referência de forma mais recente, trazendo verificações em tempo de compilação para evitar usos indevidos de referências nulas.

Por muito tempo, nas versões iniciais do C#, todas as variáveis de tipos por referência podiam assumir o valor `null` livremente, o que também significava que o desenvolvedor precisava ter cuidado para checar se um objeto era nulo antes de utilizá-lo. A ausência dessa checagem resultava no famigerado erro de exceção de referência nula (**null reference exception**), uma fonte comum de bugs em aplicações. A partir do C# 8, lançado em 2019, e consolidando-se nos projetos modernos, a linguagem introduziu o conceito de **tipos por referência anuláveis** (nullable reference types) como um recurso opcional para minimizar a probabilidade de erro de referência nula em tempo de execução.



Observação

Em programação, o termo exceção designa uma situação de erro que interrompe o fluxo normal de execução de um programa. Imagine o programa como alguém seguindo uma receita, passo a passo, de maneira sequencial. Enquanto tudo ocorre conforme o previsto, as instruções são executadas na ordem planejada. No entanto, quando surge algo inesperado – por exemplo, quando o programa tenta usar um valor que não existe de fato, acessa um recurso indisponível ou encontra um dado em formato incorreto –, ele não consegue simplesmente continuar como se nada tivesse acontecido. Nesse ponto, acontece uma exceção: o programa sinaliza que houve um problema específico e repassa esse problema para um mecanismo especial de tratamento de erros, em vez de seguir o fluxo normal.

Uma exceção de referência nula é um tipo particular de exceção. Ela aparece quando o programa tenta usar um objeto que, na verdade, está ausente, isto é, possui o valor `null`. É como se alguém tentasse abrir uma caixa que não está ali: age como se a caixa existisse, mas ela não existe, então o erro é gerado. O recurso de tipos por referência anuláveis introduzido nas versões mais recentes da linguagem busca

justamente reduzir a ocorrência desse tipo de situação, ajudando o desenvolvedor a perceber, ainda durante a escrita do código, onde há risco de a variável estar nula, em vez de descobrir o problema somente quando o programa já estiver em execução.

Em essência, quando o recurso de nulabilidade está ativado no projeto, o compilador C# passa a diferenciar entre referências que podem ser nulas e referências que não devem ser nulas. Essa diferença é indicada sintaticamente pelo uso do operador `?` após o tipo por referência. Por exemplo, `string? nome` declara que a variável `nome` pode conter uma referência para um objeto `String` ou o valor `null`; já `string nome2` (sem o `?`) indica que `nome2` deve sempre referenciar uma instância válida de `String` (não anulável). O compilador então realiza uma análise estática do fluxo do programa para rastrear o estado nulo de cada referência ao longo do código.

Esse estado pode ser basicamente de dois tipos: ou a variável é considerada não nula naquele ponto (o compilador conseguiu deduzir que ela foi inicializada com um objeto e não foi invalidada desde então), ou é considerada possivelmente nula (há a possibilidade de ela não referenciar objeto algum). Com base nisso, novas regras são aplicadas:

- Se tentamos atribuir `null` a uma variável que foi declarada como de tipo não anulável (por exemplo, fazer `nome2 = null` onde `nome2` é `string` não anulável), o compilador emite um aviso ou um erro conforme a configuração, pois estamos violando a promessa de não nulidade. A linguagem nos força, portanto, a deixar explícito quando algo pode ser nulo, em vez de assumir isso silenciosamente.
- Se tentamos acessar um membro ou um método de uma variável que está em estado possivelmente nulo, o compilador também gera um aviso, apontando que poderíamos estar desreferenciando uma referência nula – o que, em tempo de execução, causaria exceção.
- Ao contrário, se uma variável é não anulável, o compilador espera que ela seja inicializada com um valor não nulo antes de ser usada. No caso de variáveis locais, isso é simples: precisamos atribuir algo não nulo antes de seu uso. Para membros não anuláveis de uma classe, existe a obrigação de eles serem inicializados no construtor ou diretamente na declaração, caso contrário o compilador emitirá avisos de que "a propriedade X não anulável não foi inicializada". Essa regra incentiva a definição de invariantes de não nulidade, garantindo que, por contrato, certos campos ou propriedades nunca serão nulos em uso normal.

Para ilustrar, considere o trecho de código a seguir, supondo que o contexto de tipos anuláveis esteja habilitado no projeto (o que, em projetos atuais do .NET, já vem habilitado por padrão):

```
string? mensagem = null;
Console.WriteLine(mensagem.Length); // Aviso do compilador: possível
desreferência nula (CS8602)

mensagem = "Olá, mundo!";
Console.WriteLine(mensagem.Length); // OK, mensagem não é nula aqui

string texto = mensagem; // Aviso: conversão de possível nulo para não anulável
(CS8600)
```

Nesse exemplo, declaramos mensagem como `string?`, ou seja, uma cadeia de caracteres anulável. Inicialmente, atribuímos `null` a ela, o que é permitido pela anotação `?`. Em seguida, tentamos acessar `mensagem.Length` (que calcula o comprimento da cadeia de caracteres) enquanto mensagem ainda poderia ser nula – o compilador nos avisa que isso é potencialmente problemático (indicamos o código do aviso CS8602 como referência). Depois atribuímos uma `string` literal real à mensagem (agora certamente não nula) e na linha seguinte o acesso `mensagem.Length` não gera aviso, pois o compilador acompanhou o fluxo e sabe que, após a atribuição, mensagem está não nula. Por fim, tentamos atribuir mensagem (um `string?` que pode ser nulo) a `texto`, que é declarado como `string` não anulável. O compilador alerta sobre uma possível violação – afinal, se mensagem fosse nula naquele momento, estaríamos colocando um `null` em uma variável que não admite isso.

Para suprimir intencionalmente um aviso desses (quando se tem certeza do estado não nulo), C# fornece o operador de null-forgiveness `!`. Por exemplo, poderíamos escrever `string texto = mensagem!`; para indicar ao compilador: "confie em mim, mensagem não é nula aqui". Isso remove o aviso, porém deve ser usado com cautela: é uma forma de sobrescrever o mecanismo de análise estática e, se usado incorretamente (quando a variável for de fato nula), o erro de runtime continuará existindo. Idealmente, deve-se preferir checagens explícitas de `null` (`if (mensagem != null) { ... }`) antes de utilizar a variável, ou empregar o operador de coalescência nula (`??`) para fornecer valores alternativos em caso de `null`.

Em síntese, os tipos por referência anuláveis adicionam ao C# uma camada de segurança em tempo de compilação contra erros de referência nula. Eles não mudam o comportamento em tempo de execução – nos bastidores, `string` e `string?` são o mesmo tipo `System.String` no runtime, e a distinção existe apenas para o compilador.

Portanto, uma variável marcada como não anulável não ganhará verificações automáticas de `null` em tempo de execução, mas confiamos que, seguindo os avisos do compilador, teremos tomado as precauções necessárias. Ferramentas e bibliotecas podem ler as anotações de nulabilidade (via atributos adicionados pelo compilador) para também adaptar seu comportamento. Por exemplo, frameworks ORM (Object-Relational Mapping), como o Entity Framework, podem interpretar uma propriedade `string?` Nome indicando que o campo Nome no banco de dados aceita `null`, enquanto `string` Nome indicaria não aceitar `null`, tudo isso com base nas anotações de tipos anuláveis.

Para utilizar tipos por referência anuláveis em um projeto, é preciso habilitar o contexto anulável. Em projetos recentes do .NET (por exemplo, criando um novo projeto com Visual Studio 2022/2026 ou `dotnet new`), o arquivo de projeto já inclui a propriedade `<Nullable>enable</Nullable>` por padrão, ligando o recurso. Em projetos mais antigos, pode ser necessário habilitá-lo manualmente, seja editando o `.csproj`, seja usando diretivas como `#nullable enable` no início dos arquivos de código (de forma local). O contexto anulável também pode ser desabilitado para partes do código com `#nullable disable` se necessário, o que pode ser útil durante a migração gradual de um código legado para o novo sistema.

Portanto, a nulabilidade no C# moderno envolve duas frentes: (1) os tipos por valor anuláveis (`T?`, em que `T` é um valor, via `Nullable<T>`), que existem desde C# 2.0 e permitem representar `null` para structs e primitivos; e (2) os tipos por referência anuláveis (habilitados em C# 8+), que introduzem

análise estática para evitar usos inadvertidos de referência nula. Juntos, esses recursos ajudam a produzir código mais seguro e intenções mais explícitas quanto à possibilidade (ou não) de ausência de valores.

Em C#, **declaramos variáveis** especificando primeiramente o tipo, seguido do nome da variável e, opcionalmente, de uma atribuição inicial. Por exemplo, podemos declarar e inicializar algumas variáveis de tipos primitivos assim:

```
int quantidade = 5;           // quantidade é um inteiro (int) inicializado
com 5
double preco = 3.99;         // preco é um double (ponto flutuante)
inicializado com 3.99
char inicial = 'A';          // inicial é um caractere com valor 'A'
bool ativo = true;           // ativo é um booleano com valor verdadeiro
string saudacao = "Bom dia"; // saudacao é uma string com o texto "Bom dia"
```

Em todos esses casos, o tipo está sendo especificado explicitamente (**int**, **double** etc.). No entanto, desde o C# 3.0, a linguagem suporta a chamada variável local de tipo implícito utilizando a palavra-chave **var**. Isso permite que o compilador infira o tipo da variável a partir da expressão de inicialização, simplificando a sintaxe em casos em que o tipo concreto já está evidente no lado direito da atribuição. Por exemplo:

```
var i = 5;                    // i é inferido como int (System.Int32):contentReference
[oaicite:25]{index=25}
var mensagem = "Olá";        // mensagem é inferido como string (System.String):content
Reference[oaicite:26]{index=26}
var valores = new int[] { 1, 2, 3 }; // valores é inferido como int[] (array de
int)
var cliente = new Cliente(); // cliente é inferido como Cliente (tipo definido
pelo usuário)
```

Nessas linhas, o compilador deduz o tipo de cada variável: em **var i = 5;**, como 5 é um literal inteiro, **i** será tratado como **int** em tempo de compilação. Da mesma forma, ao atribuir uma string literal "Olá" à mensagem, o tipo é inferido como string. Quando usamos **new Cliente()**, supondo que **Cliente** seja uma classe definida no projeto, o **var** infere o tipo dessa instância como **Cliente**. Vale destacar que a inferência ocorre em tempo de compilação; após essa etapa, a variável **i** tem um tipo fixo (**int**), não podendo posteriormente receber um valor de outro tipo incompatível. Ou seja, **var** não equivale a um tipo dinâmico ou genérico que muda – é apenas uma sintaxe conveniente para deixar o compilador determinar o tipo concreto.

É importante entender que a palavra-chave **var** não significa variável de tipo genérico **nem torna a variável fracamente tipada ou de tipagem dinâmica**. O uso de **var** não faz com que o C# se torne menos tipado; a variável continua fortemente tipada, apenas o tipo é omitido no código-fonte por conveniência. O compilador exige que a inicialização seja fornecida na mesma instrução em que o **var** é usado (pois ele precisa de uma expressão à direita para inferir o tipo). Se tentarmos escrever apenas **var x;** sem inicializar, o código não compila, pois o compilador não tem como deduzir o tipo de **x** nessa situação. Além disso, **var** só pode ser utilizado em variáveis locais (dentro de métodos ou blocos de código). Ele não é permitido em campos de classe ou estrutura, nem em parâmetros de métodos,

justamente para evitar ambiguidades de tipo em contexto de membros da classe. A inferência só funciona no escopo local no qual uma atribuição imediata está presente. A seguir, algumas considerações e boas práticas no emprego de **var**:

- Use **var** quando o tipo do lado direito é óbvio para um leitor humano ou quando é muito verboso. Por exemplo, ao escrever **var lista = new List<Cliente>();**, é bastante claro que **lista** será do tipo **List<Cliente>**, eliminando a repetição do tipo em ambas as partes da declaração. Por outro lado, em algo como **var numero = obterValor();**, no qual o tipo retornado por **obterValor()** pode não ser evidente, talvez seja melhor especificar o tipo explicitamente para melhorar a legibilidade, a menos que o contexto deixe claro do que se trata.
- Em situações de tipos anônimos, **var** é obrigatório. O C# permite criar objetos *ad hoc* com propriedades sem definir uma classe formal – os chamados tipos anônimos. Por exemplo: **var anon = new { Nome = "Maria", Idade = 30 };**. Aqui, **anon** terá um tipo anônimo gerado pelo compilador (sem nome acessível no código), então não haveria como declarar essa variável de outra forma senão com **var**. Em consultas LINQ, por exemplo, muitas vezes obtemos sequências de objetos anônimos, e nesses casos usar **var** é necessário para manipular os resultados.
- Evite abusar de **var** em detrimento da clareza. Se o tipo inferido não for aparente, empregar **var** pode dificultar a compreensão do código. Um código equilibrado tipicamente utiliza **var** quando a inferência torna o código mais conciso, porém ainda legível, e mantém declarações explícitas quando a ausência do tipo poderia confundir.

Exemplificando o cenário de tipos anônimos e **var** em conjunto, veja este trecho:

```
var resultado = from c in clientes
                where c.Cidade == "São Paulo"
                select new { NomeCliente = c.Nome, AnoNascimento =
c.DataNascimento.Year };

foreach (var item in resultado) {
    Console.WriteLine($"Cliente: {item.NomeCliente}, Nascido em: {item.
AnoNascimento}");
}
```

Suponha que **clientes** seja uma coleção de objetos de uma classe **Cliente** com propriedades **Nome** e **DataNascimento**. A expressão LINQ anterior filtra os clientes de **São Paulo** e projeta cada elemento em um objeto anônimo com duas propriedades: **NomeCliente** e **AnoNascimento**. O tipo de resultado é algo como **IEnumerable<>** (um enumerable de um tipo anônimo), impossível de escrever diretamente no código. Por isso usamos **var resultado** – não há alternativa prática. Dentro do loop **foreach**, novamente adotamos **var item** porque o tipo concreto dos elementos é esse tipo anônimo gerado no **select**. Essa é uma situação na qual **var** é útil, permitindo manipular tipos complexos ou desconhecidos para o programador de forma simples.

Em resumo, o uso de **var** no C# adiciona flexibilidade sintática e pode melhorar a legibilidade, reduzindo repetições desnecessárias, desde que o seu emprego seja criterioso. Implicitamente tipado não significa dinâmico ou não tipado, já que a verificação de tipos pelo compilador permanece tão rigorosa quanto seria com declarações explícitas. Assim, ao longo deste livro-texto, incentivamos a utilização de **var** nas ocasiões adequadas, ao mesmo tempo reforçando a compreensão do sistema de tipos para evitar mal-entendidos.

Em C# no ecossistema .NET, o código-fonte é escrito dentro de **projetos**, e esses projetos são agrupados em uma **solução**. A solução (arquivo `.sln`) é um contêiner lógico: ela guarda a lista de projetos, as dependências entre eles, as configurações de build e os perfis de execução. Cada projeto (arquivo `.csproj`) define como compilar um artefato (uma biblioteca, um aplicativo de console, uma Web API, um conjunto de testes). Essa separação permite evoluir módulos de forma independente e reutilizar bibliotecas em diferentes aplicativos.



Observação

Build é o processo que transforma o código-fonte e seus recursos em artefatos executáveis ou distribuíveis. No .NET, ele costuma englobar a restauração de pacotes, a geração de código quando houver, a compilação do C# para Intermediate Language (IL), a execução de analisadores, a criação de assemblies (DLLs/EXEs), os símbolos de depuração e, quando solicitado, o empacotamento e a publicação. A ordem de compilação respeita as dependências entre projetos da solução, as opções do `.csproj` controlam o comportamento (por exemplo, Debug com diagnósticos ou Release com otimizações) e a execução pode ocorrer pelo Visual Studio 2026 ou pela CLI com `dotnet build`, `dotnet test` e `dotnet publish`, mantendo reprodutibilidade quando o SDK e as propriedades compartilhadas estão fixados.

Dito de outro modo, o código é organizado em projetos (`.csproj`) que geram artefatos específicos – bibliotecas reutilizáveis, aplicativos de console, serviços web – e em uma solução (`.sln`) que agrega esses projetos, registra as dependências entre eles e guarda configurações de compilação e execução. A solução funciona como índice da aplicação e dita a ordem de build com base nas referências declaradas; cada projeto, por sua vez, define o que compilar, com quais opções e quais outras unidades de código consome. Vejamos um exemplo de estrutura sugerida de pastas para soluções de médio/alto porte:

```
MinhaSolucao/  
  src/  
    MinhaEmpresa.Produto.Core/ # biblioteca de domínio (class library)  
    MinhaEmpresa.Produto.Infrastructure/ # acesso a dados/integrações  
    MinhaEmpresa.Produto.Api/ # Web API (ASP.NET Core)  
    MinhaEmpresa.Produto.Cli/ # app de console (opcional)  
  tests/  
    MinhaEmpresa.Produto.Core.Tests/ # testes da camada Core  
    MinhaEmpresa.Produto.Infrastructure.Tests/ # testes da Infrastructure
```

```

MinhaEmpresa.Produto.Api.Tests/      # testes de integração/endpoint
samples/
MinhaEmpresa.Produto.Samples/        # exemplos de uso de APIs públicas
build/                                # scripts de build/pipeline
docs/                                 # documentação
Directory.Build.props                 # propriedades compartilhadas
Directory.Build.targets               # alvos de build compartilhados
Directory.Packages.props              # gerenciamento central de pacotes (NuGet)
MinhaSolucao.sln

```

Nessas soluções de médio e grande porte, é comum separar o repositório em diretórios lógicos como `src` para o código de produção, `tests` para testes automatizados, `samples` para exemplos de uso de APIs públicas, `build` para scripts de integração contínua e `docs` para documentação; no Visual Studio 2026, pastas de solução podem espelhar essa divisão lógica para facilitar a navegação diária sem impor vínculo rígido ao layout físico do disco. Convenções de nomenclatura ajudam a leitura: nomes no formato `Empresa.Produto.Módulo` tornam explícito o papel de cada projeto, enquanto o sufixo `.Tests` identifica com clareza os testes automatizados que exercitam módulos correspondentes.

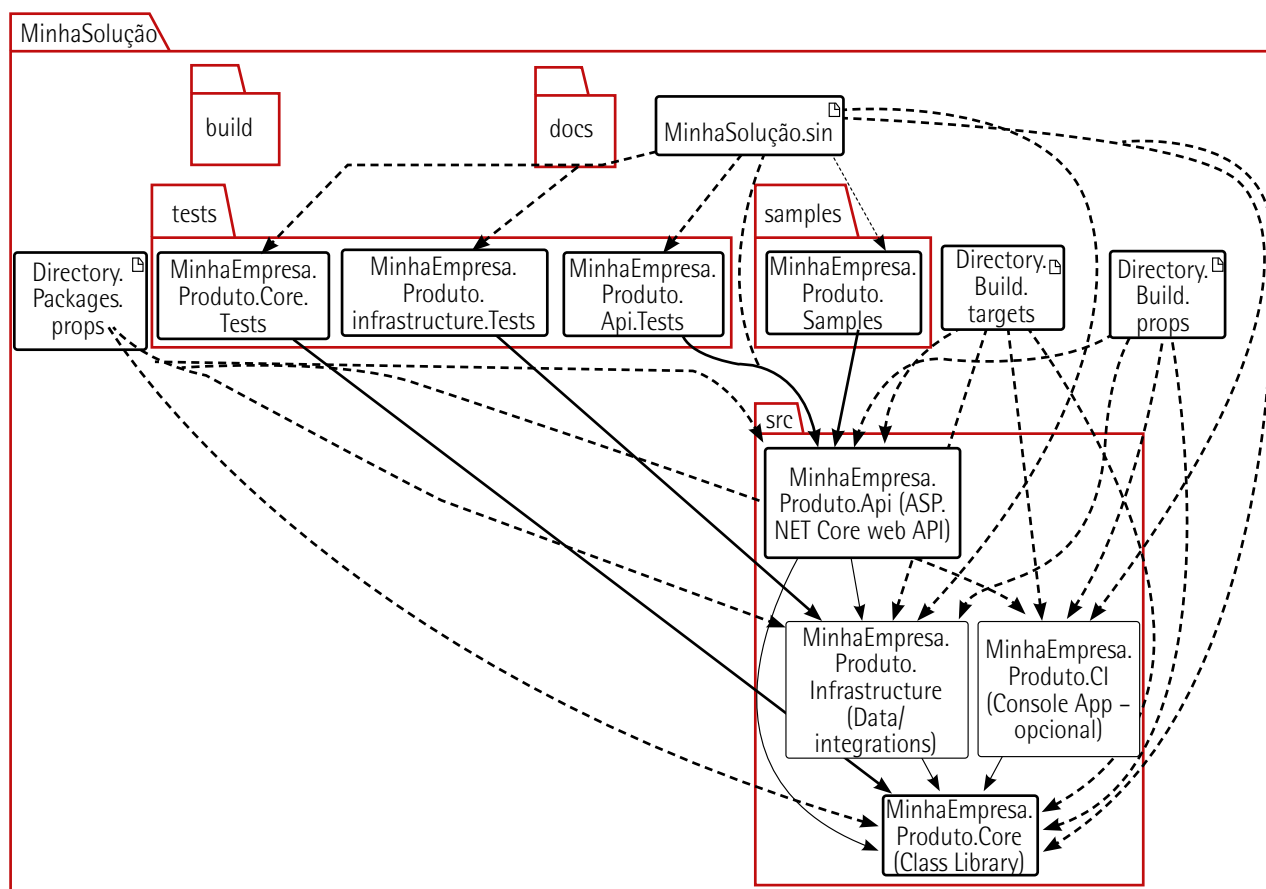


Figura 6 – Estrutura sugerida de pastas para soluções de médio/alto porte

Nos projetos criados a partir dos modelos atuais, os arquivos de projeto adotam o formato baseado no SDK (`<Project Sdk="Microsoft.NET.Sdk">`). Um `.csproj` mínimo para biblioteca concentra propriedades essenciais – versão da linguagem, nulabilidade e usings implícitos –, tornando as regras de compilação explícitas e reprodutíveis.

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <Deterministic>true</Deterministic>
  </PropertyGroup>
</Project>
```

Esse exemplo ilustra o conteúdo de um arquivo `.csproj` que pode estar, por exemplo, em:

```
src/MinhaEmpresa.Produto.Core/MinhaEmpresa.Produto.Core.csproj
src/MinhaEmpresa.Produto.Api/MinhaEmpresa.Produto.Api.csproj
tests/MinhaEmpresa.Produto.Core.Tests/MinhaEmpresa.Produto.Core.Tests.csproj
```

Ou seja, cada pasta de projeto (Core, Infrastructure, Api, Cli, *.Tests, Samples) tem um arquivo `.csproj`, e esse arquivo em formato XML (Extensible Markup Language) fica dentro dele.

A colaboração entre módulos da mesma solução ocorre por referências de projeto, que mantêm o grafo de dependências explícito e evitam acoplamento via assemblies pré-compilados. O vínculo é feito com **ProjectReference**, apontando para o `.csproj` do módulo requerido:

```
<ItemGroup>
  <ProjectReference Include="..\MinhaEmpresa.Produto.Core\MinhaEmpresa.Produto.Core.csproj" />
</ItemGroup>
```

Nesse caso, esse conteúdo também vai dentro do `.csproj` do projeto consumidor. São exemplos de arquivos com esse tipo de conteúdo:

```
src/MinhaEmpresa.Produto.Api/MinhaEmpresa.Produto.Api.csproj referencia ..\
MinhaEmpresa.Produto.Core\MinhaEmpresa.Produto.Core.csproj

e tests/MinhaEmpresa.Produto.Core.Tests/MinhaEmpresa.Produto.Core.Tests.csproj
referencia ..\..\src\MinhaEmpresa.Produto.Core\MinhaEmpresa.Produto.Core.csproj
```

A obtenção de bibliotecas externas usa pacotes NuGet (falaremos dele em outro tópico). Em soluções com múltiplos projetos, o gerenciamento centralizado de versões no arquivo **Directory.Packages.props** garante coerência e simplifica atualizações; cada `.csproj` passa a declarar apenas os pacotes, enquanto as versões ficam unificadas na raiz do repositório.

```
<!-- Directory.Packages.props -->
<Project>
  <ItemGroup>
    <PackageVersion Include="Newtonsoft.Json" Version="13.0.3" />
    <PackageVersion Include="xunit" Version="2.7.0" />
    <PackageVersion Include="xunit.runner.visualstudio" Version="2.5.7" />
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.11.1" />
  </ItemGroup>
</Project>
```

Esse bloco é o conteúdo do arquivo `Directory.Packages.props`, que fica na raiz da solução:

```
MinhaSolucao/
  Directory.Packages.props  <-- esse XML ficará aqui
  MinhaSolucao.sln
  Directory.Build.props
  Directory.Build.targets
  ...
<!-- Em cada .csproj -->
<ItemGroup>
  <PackageReference Include="Newtonsoft.Json" />
</ItemGroup>
```

Quando uma mesma biblioteca precisa atender a mais de uma versão do .NET ou plataformas distintas, o multi-targeting concentra variações num único projeto. A propriedade **TargetFrameworks** lista os Target Framework Monikers (TFMs) pretendidos, e o pipeline de build passa a validar cada alvo de forma sistemática.

```
<PropertyGroup>
  <TargetFrameworks>net8.0;net6.0</TargetFrameworks>
</PropertyGroup>
```

Políticas transversais à solução ficam mais bem documentadas e aplicadas por arquivos compartilhados. O `Directory.Build.props` estabelece padrões como nulabilidade habilitada, usings implícitos e tratamento de avisos; o `Directory.Build.targets` permite acoplar etapas comuns do processo de build, como geração de código ou cópia de artefatos; o `.editorconfig` define estilo de código e regras de analisadores para que o Visual Studio 2026 e o `dotnet format` mantenham a base consistente entre equipes e máquinas.

A presença de testes em projetos dedicados é parte da organização. Um projeto de testes com xUnit, por exemplo, referencia o módulo em avaliação e instala o runner adequado, mantendo o artefato de testes como não distribuível (propriedade **IsPackable** falsa) e com dependências isoladas:

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <IsPackable>>false</IsPackable>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="xunit" />
    <PackageReference Include="xunit.runner.visualstudio" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" />
  </ItemGroup>
</Project>
```

```
</ItemGroup>
<ItemGroup>
  <ProjectReference Include="..\..\src\MinhaEmpresa.Produto.Core\MinhaEmpresa.
  Produto.Core.csproj" />
</ItemGroup>
</Project>
```

Na execução local, APIs e aplicativos de console usam perfis definidos em `Properties\launchSettings.json`, onde ficam as portas, as variáveis de ambiente e as opções de depuração; quando a solução possui mais de um executável, o Visual Studio permite marcar múltiplos projetos de inicialização para simular os cenários completos, como a API e o worker executando lado a lado. Esse arquivo fica em:

```
src/MinhaEmpresa.Produto.Api/
  Properties/
    launchSettings.json
```

A seguir, apresenta-se um resumo que mapeia o diretório com o tipo de arquivo:

```
MinhaSolucao/
MinhaSolucao.sln      # solução (não é XML de MSBuild)
Directory.Packages.props # XML com <Project><ItemGroup><PackageVersion...>
Directory.Build.props  # XML com <Project> para propriedades globais
Directory.Build.targets # XML com <Project> para targets globais

src/
MinhaEmpresa.Produto.Core/
MinhaEmpresa.Produto.Core.csproj # tem <Project Sdk=...>, PropertyGroup, Package
MinhaEmpresa.Produto.Infrastructure/
MinhaEmpresa.Produto.Infrastructure.csproj
MinhaEmpresa.Produto.Api/
MinhaEmpresa.Produto.Api.csproj
Properties/launchSettings.json      # JSON, não XML
MinhaEmpresa.Produto.Cli/
MinhaEmpresa.Produto.Cli.csproj

tests/
MinhaEmpresa.Produto.Core.Tests/
MinhaEmpresa.Produto.Core.Tests.csproj # XML de projeto de teste xUnit
...

samples/
MinhaEmpresa.Produto.Samples/
MinhaEmpresa.Produto.Samples.csproj
```

1.2 Ferramentas: Visual Studio 2026 e dotnet CLI (estrutura .csproj, SDK-style)

Para que você consiga executar os códigos deste livro-texto, comece instalando o Visual Studio 2026 (edição Community, que você pode obter gratuitamente pelo site oficial da Microsoft) e, no instalador (Visual Studio Installer), selecione o workload .NET Desktop Development (figuras 7 e 8). Na aba Language Packs, você pode selecionar também o idioma português brasileiro (figura 9).

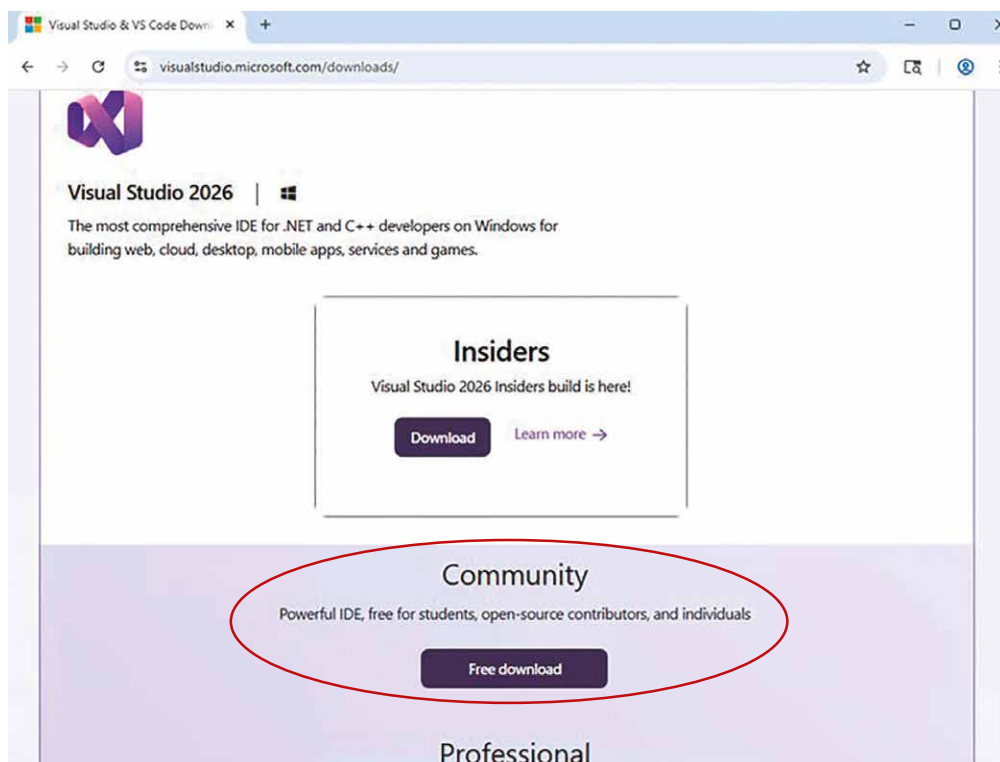


Figura 7 – Fazendo o download do Visual Studio 2026

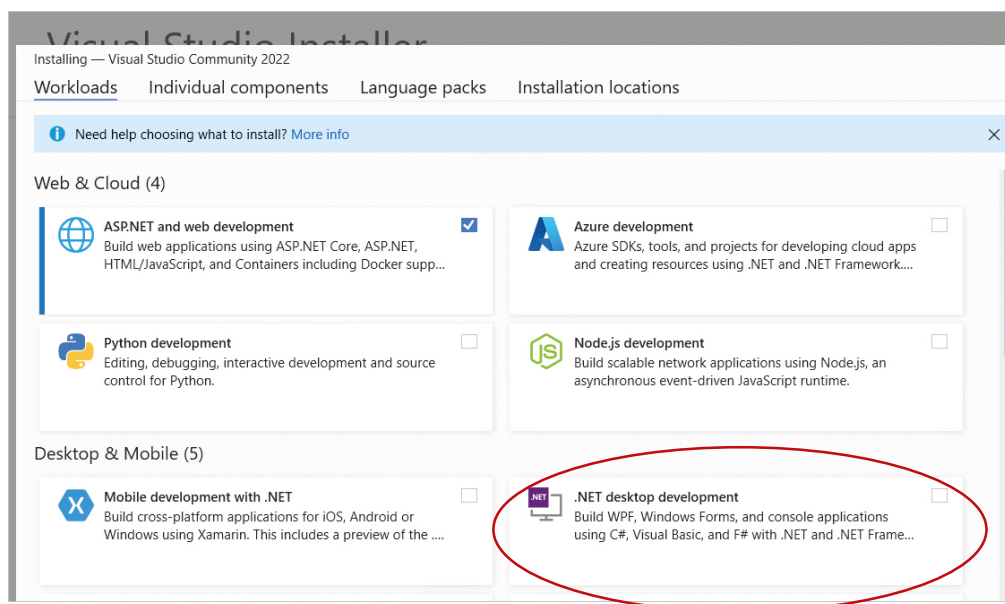


Figura 8 – Escolhendo, na instalação, o workload para desenvolvimento .NET

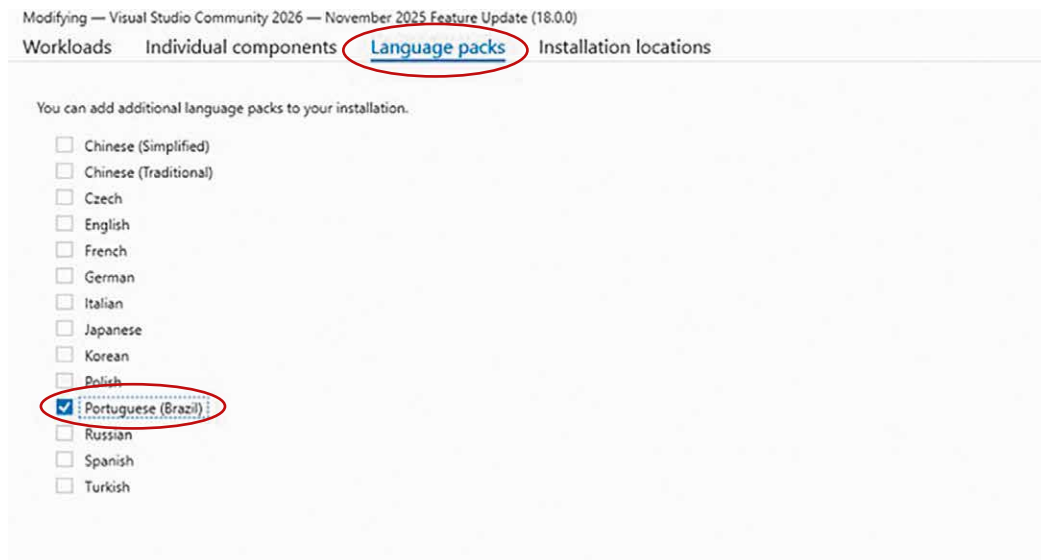


Figura 9 – Selecionando o idioma Português (Brasil)

Para finalizar a instalação, basta clicar no botão Install.

O Visual Studio 2026 é a versão mais recente da IDE da Microsoft, lançada em novembro de 2025 e identificada internamente como versão 18, com foco em integração profunda de inteligência artificial e no suporte nativo a .NET 10 e C# 14, além de apresentar melhorias de desempenho em relação ao Visual Studio 2022.

Em vez de um único produto, ele aparece como uma família de versões, que combinam basicamente o mesmo núcleo técnico, mas com diferenças de público-alvo, direitos de uso e conjunto de ferramentas adicionais.

A edição **Visual Studio Community 2026** é a porta de entrada gratuita. A própria Microsoft a descreve como uma IDE "free, fully-featured" para estudantes, contribuidores de projetos open source e desenvolvedores individuais.

Em termos práticos, isso significa que o conjunto de recursos centrais de edição de código, depuração, testes, integração com Git e suporte a .NET 10, C# 14, C++ e demais workloads modernos é o mesmo da linha paga, com restrições concentradas no tipo de organização e no número de usuários que podem utilizar tal versão em cenários comerciais: organizações não empresariais, com até cinco usuários, podem adotar o Community de forma legítima, enquanto grandes empresas precisam migrar para o Professional ou o Enterprise, conforme as licenças oficiais.

Git é um sistema de controle de versão distribuído usado para acompanhar o histórico de alterações em arquivos, especialmente código-fonte. Ele permite trabalhar com branches, mesclar contribuições de vários desenvolvedores e manter um repositório completo em cada máquina, com sincronização posterior com servidores remotos.



Saiba mais

O livro a seguir é uma introdução completa ao Git, do básico ao avançado, sendo adequado para estudo sistemático ou consulta pontual no dia a dia. Cobre branching, workflows distribuídos, Git em servidores e GitHub, com muitos exemplos práticos.

CHACON, S.; STRAUB, B. *Pro Git*. 2. ed. Berkeley: Apress, 2014.

Já a obra seguinte é recomendada para quem já programa e deseja entender Git em profundidade, sendo útil para papéis de liderança técnica e administração de repositórios. Concentra-se em linha de comando, internals do repositório, estratégias de branching e integração com outros sistemas.

PONUTHORAI, P. K; LOELIGER, J. *Version control with Git: powerful tools and techniques for collaborative software development*. 3. ed. Sebastopol: O'Reilly Media, 2022.

Acima da edição Community está o Visual Studio Professional 2026, pensado para equipes pequenas e ambientes corporativos em que já existe a necessidade de licenciamento comercial formal, suporte e benefícios de assinatura.

Em termos de experiência, ele compartilha praticamente todos os recursos de desenvolvimento da edição Community, mas vem integrado com vantagens ligadas a assinaturas (acesso a serviços, suporte e benefícios de nuvem em alguns planos) e é o caminho típico para empresas menores que precisam cumprir requisitos de compliance e gestão de licenças sem necessariamente demandar todos os recursos avançados de engenharia de software presentes no Enterprise.

No topo da linha está o Visual Studio Enterprise 2026, voltado a organizações de qualquer porte que precisam de recursos avançados de arquitetura, depuração, testes e governança. Em outras palavras, é a edição que agrega, por exemplo, ferramentas mais sofisticadas de diagnóstico e análise de desempenho, recursos de teste automatizado em escala e integrações pensadas para cenários de grandes equipes, múltiplos serviços e ambientes complexos, além de apresentar benefícios adicionais de nuvem atrelados às assinaturas Enterprise.



Saiba mais

A Microsoft disponibiliza uma página detalhada da comparação entre as edições em:

COMPARAR as edições do Visual Studio. *Microsoft*, [s.d.]. Disponível em: <https://tinyurl.com/ym7j8c8d>. Acesso em: 26 nov. 2025.

Além dessas três edições principais da IDE, o ecossistema Visual Studio 2026 oferece versões complementares que muitas vezes aparecem na mesma página de download e acabam sendo confundidas com edições da IDE. Uma delas é o Visual Studio 2026 Insiders, que funciona como canal de pré-lançamento: builds mais novos, com recursos em teste, instalados lado a lado com versões estáveis e projetados para quem deseja experimentar novidades de desempenho, integração com GitHub Copilot e a interface atualizada antes de elas chegarem ao canal principal.

Outra versão é o Visual Studio Team Explorer 2026, um cliente gratuito destinado a quem não programa, mas precisa interagir com Azure DevOps (itens de trabalho, repositórios, pipelines etc.) sem instalar a IDE completa.

Também aparecem no grupo de downloads do Visual Studio 2026 os Remote Tools e os Build Tools. Os Remote Tools para Visual Studio 2026 servem para depuração remota, testes e análise de desempenho em máquinas que não têm a IDE instalada, permitindo, por exemplo, anexar o depurador a um servidor ou dispositivo remoto usando apenas esse pacote auxiliar, desde que exista uma licença válida de Visual Studio associada.

Já os Build Tools para Visual Studio 2026 formam um conjunto específico para compilação via linha de comando ou sistemas de build (MSBuild, CMake etc.) sem o ambiente gráfico. Eles permitem compilar projetos ASP.NET, .NET, C++, Node.js, Python e outros workloads suportados pela IDE, o que é também condicionado a uma licença de Visual Studio quando utilizados em cenários proprietários, com exceção do uso na construção de dependências open source, de acordo com os termos de licença publicados.

Do ponto de vista conceitual, portanto, falar em versões do Visual Studio 2026 significa distinguir, de um lado, as edições comerciais da IDE – Community, Professional e Enterprise, que compartilham o mesmo núcleo técnico e variam sobretudo por licença e recursos avançados – e, de outro, os pacotes auxiliares e os canais paralelos, como Insiders, Team Explorer, Remote Tools e Build Tools, que não são IDEs completas, mas ampliam o ecossistema ao cobrir cenários específicos de desenvolvimento, colaboração, depuração e automação de builds. Tudo isso está ancorado na mesma base tecnológica do Visual Studio 2026, de modo que a escolha entre as versões é menos uma questão de capacidade de programar em X ou Y linguagem e mais uma decisão sobre requisitos organizacionais, governança e fluxo de trabalho em equipes de software.

Agora, vamos exercitar um pouco a programação C# no ambiente que você instalou. Abra o Visual Studio 2026. Na tela inicial, escolha Criar um projeto, digite "console" na busca, selecione Aplicativo do Console em C# e confirme as opções a seguir. Finalmente, clique em Criar. O assistente cria a solução com um `Program.cs` inicial.

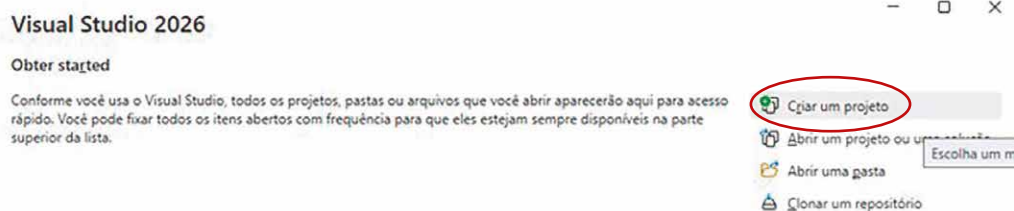


Figura 10 – Criando um novo projeto no Visual Studio 2026

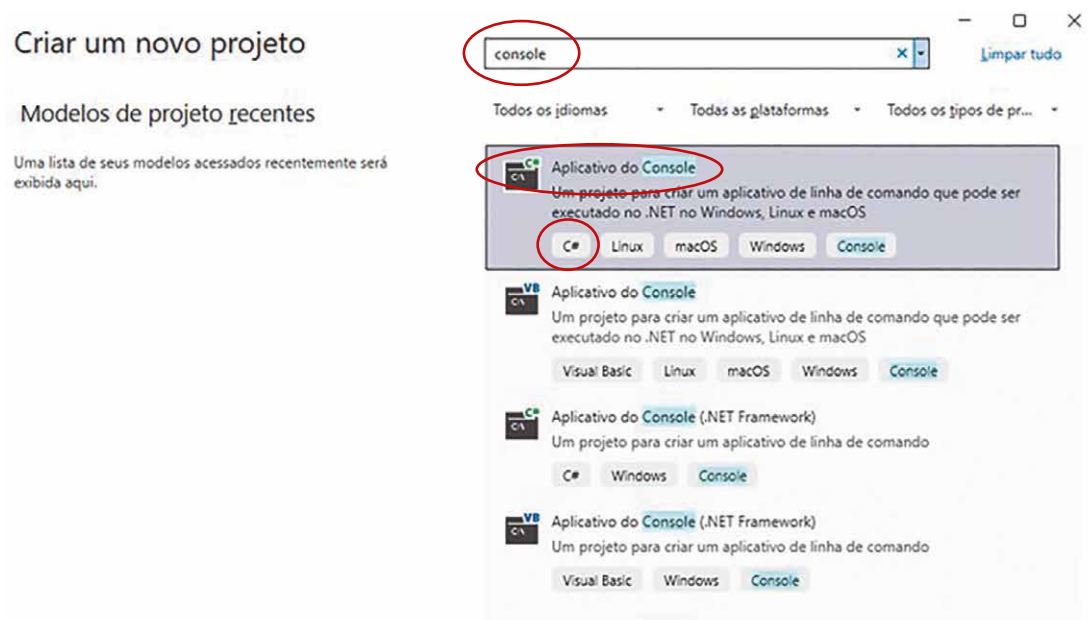


Figura 11 – Escolhendo a solução Aplicativo do Console em C#

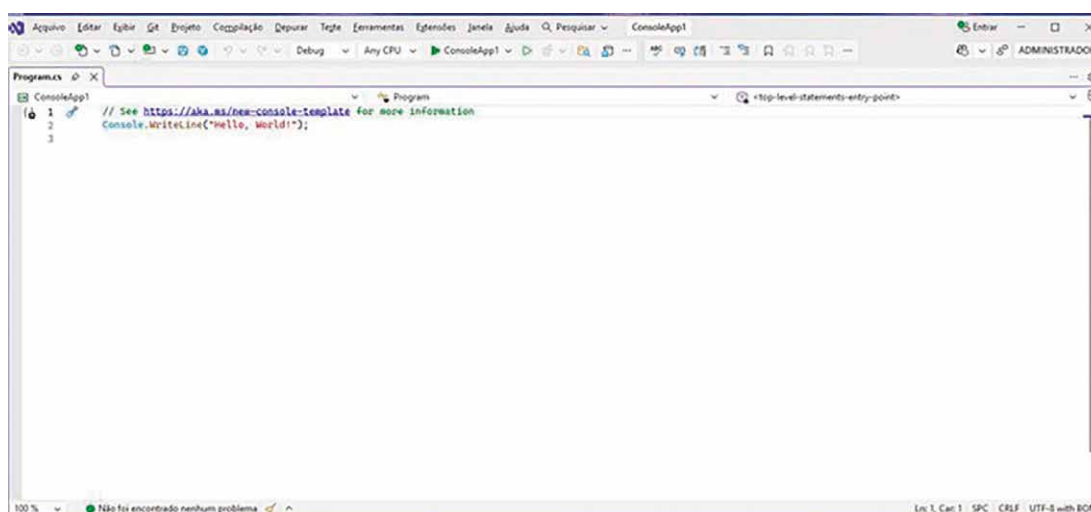


Figura 12 – Solução Program.cs criada e aberta no modo de edição

Quando o projeto abre, você vê o editor com `Program.cs`. Como falamos anteriormente, nas versões modernas do C#, você pode escrever instruções de nível superior, ou seja, código direto no arquivo sem declarar um `Main` manualmente. Apague o conteúdo gerado, cole o código-fonte fornecido a seguir e execute-o com o botão ► Executar ou com a tecla F5 (depurando) ou com as teclas Ctrl+F5 (sem depurar) pressionadas simultaneamente. O código a seguir possui exemplos práticos de conceitos de sintaxe introduzidos no subtópico anterior, bem como novos conceitos que você aprenderá em breve.

PROGRAMAÇÃO ORIENTADA A OBJETOS COM C#

```
using System;

Console.WriteLine("Olá! Vamos testar a sintaxe básica do C# 14.");

// Tipos e variáveis
int ano = 2026;
var nome = "Ana";
Console.WriteLine($"Boas-vindas, {nome}. Estamos em {ano}.");

// Nulabilidade e operadores relacionados
string? apelido = null;
var exibir = apelido ?? "visitante";
Console.WriteLine(exibir); // "visitante"
apelido ??= "convidado"; // atribui só se for nulo
Console.WriteLine(apelido);

// Controle de fluxo e pattern matching com expressão switch
object? x = 150;
string descricao = x switch
{
    null => "nulo",
    int n when n > 100 => "inteiro grande",
    string s => $"texto com {s.Length} chars",
    _ => "desconhecido"
};
Console.WriteLine(descricao);

// Métodos locais e expressão de corpo
static decimal Calcular(decimal preco, decimal taxa = 0.12m)
    => preco * (1 + taxa);
Console.WriteLine(Calcular(100m)); // 112,00

// Records (dados com igualdade estrutural) e with-expressions
var p1 = new Produto(1, "Café", 15m);
var p2 = p1 with { Preco = 18m };
Console.WriteLine(p2);

// Propriedade com o token contextual 'field' (C# 14)
var cfg = new Config { Tema = "claro" };
Console.WriteLine(cfg.Tema);

// Atribuição condicional com ?. à esquerda (C# 14)
Cliente? cliente = new Cliente();
cliente?.Total += 100m; // só avalia se cliente != null
Console.WriteLine(cliente?.Total);

public record Produto(int Id, string Nome, decimal Preco);
public class Cliente { public decimal Total { get; set; } }
public class Config
{
    public string Tema
    {
        get;
        set => field = value ?? throw new ArgumentNullException(nameof(value));
    }
}
```

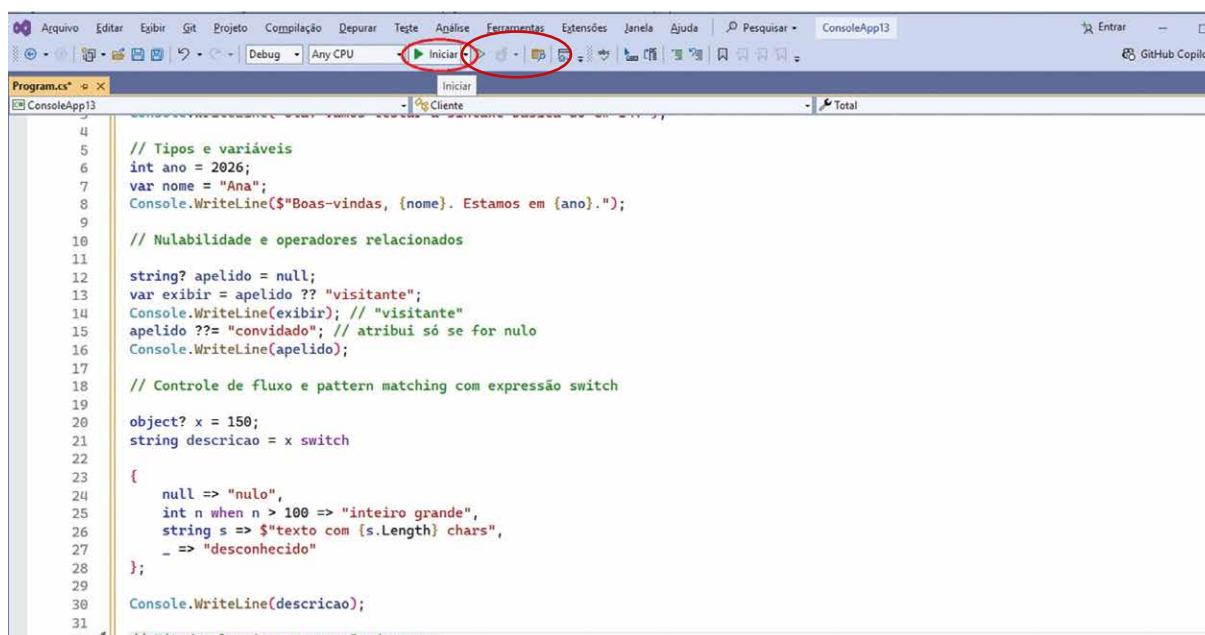


Figura 13 – Código-fonte C# copiado no editor de código da IDE. No destaque, o botão de compilação e execução

A saída aparece numa janela de console:

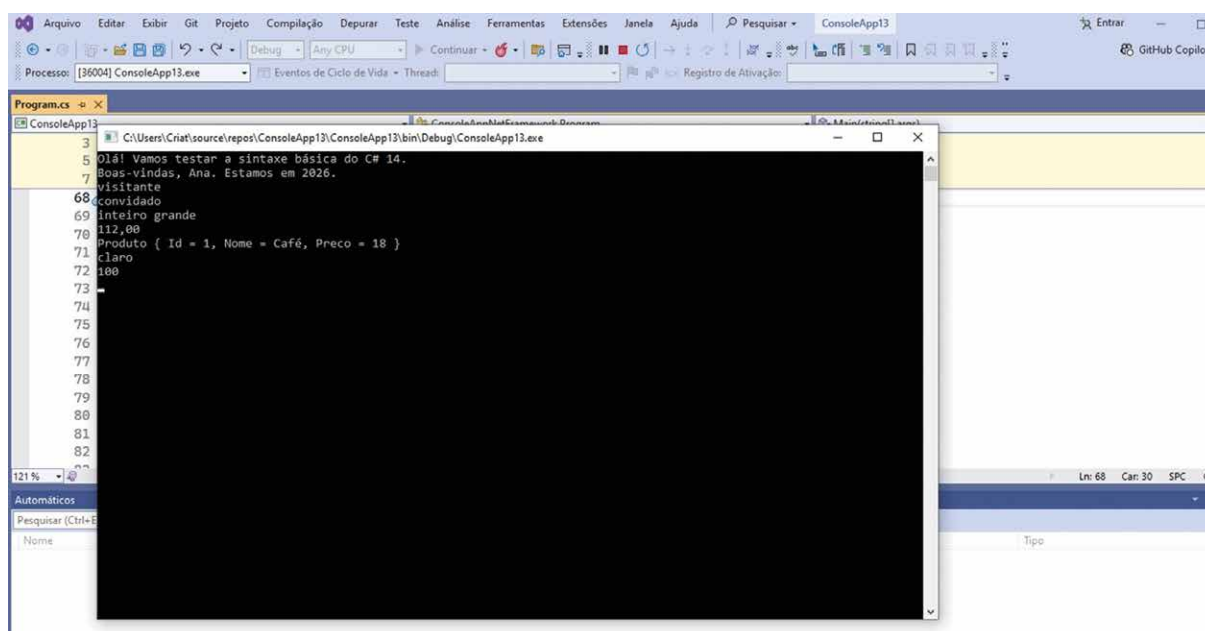


Figura 14 – Janela de console

Esse código-fonte ilustra os conceitos de variáveis e de literais, de interpolação de string, de nulabilidade com `?`, de coalescência (`??` e `??=`), de `switch` com padrões, de métodos com expressão de corpo, de record para dados imutáveis, do token contextual field em propriedades e da atribuição com navegação condicional na esquerda – novidades e aprimoramentos válidos no C# 14. Vamos entender alguns detalhes desse código-fonte.

Ele começa com `using System;`, uma diretiva para trazer o namespace `System` para o escopo do arquivo. Namespaces agrupam tipos relacionados; ao colocar `System` no escopo, fica possível referir-se a `Console` diretamente, sem qualificar com `System.Console`. O arquivo utiliza instruções de nível superior: não há um `Main` explícito, porque o compilador gera nos bastidores um ponto de entrada e executa as linhas na ordem em que aparecem, desde que declarações de tipos (como classes) venham depois desses comandos.

As chamadas a `Console.WriteLine` escrevem no fluxo de saída. Os literais entre aspas representam strings, e o prefixo `$` ativa a interpolação: os trechos entre chaves avaliam expressões, como o `{nome}` e o `{ano}`. A declaração `int ano = 2026;` define uma variável com seu tipo explícito, enquanto `var nome = "Ana";` usa uma inferência local: o compilador determina que `nome` é do tipo string a partir do valor atribuído. A inferência não torna a variável sem tipo; apenas evita a repetição quando o tipo é evidente.

O código demonstra anotações de nulabilidade para referências. A declaração `string? apelido = null;` indica que a variável pode ser nula e faz o analisador estático alertar sobre acessos potencialmente inseguros. Para tratar valores nulos, aparecem dois operadores. O coalescente nulo `??` devolve o operando da esquerda quando não é nulo e, caso contrário, o da direita; além disso, a análise de fluxo "afunila" o tipo do resultado para não anulável quando o alternativo é não anulável, como em `var exibir = apelido ?? "visitante";`. Já `??=` atribui apenas se o destino estiver nulo: `apelido ??= "convidado";` preenche uma lacuna sem sobrescrever um valor já existente.

Em `object? x = 150;`, o literal inteiro é convertido para `object` (boxing) e armazenado numa referência que também pode ser nula. A variável é então analisada por uma expressão `switch`, que é uma construção que avalia padrões e produz um valor. Cada braço contém um padrão seguido de `=>`. O primeiro testa o padrão constante `null`; o segundo verifica simultaneamente o tipo e captura o valor com um padrão de tipo (`int n`) e ainda impõe uma guarda `when n > 100`; o terceiro reconhece string e usa a interpolação para incluir `s.Length`; o curinga `_` é o descarte para qualquer outro caso. Diferentemente do `switch` de instruções, essa forma é expressão: o resultado agregado é a string atribuída a `descricao`.

Mais adiante há uma função definida no próprio arquivo, antes de qualquer tipo, o que é permitido no contexto de nível superior. Trata-se de uma função local marcada como `static`, o que sinaliza que ela não captura variáveis do escopo externo. A sintaxe do corpo da expressão usa a seta para indicar que o valor de retorno é a expressão à direita. O parâmetro `taxa` tem valor padrão (`0.12m`), e o sufixo `m` especifica o tipo decimal, adequado a cálculos monetários por evitar erros de representação binária comuns ao tipo `double`. A saída exata de `Console.WriteLine(Calcular(100m));` depende de formatação e da cultura; com a formatação padrão, zeros à direita podem não ser exibidos.

A linha `public record Produto(int Id, string Nome, decimal Preco);` define um `record` de classe com forma posicional. O compilador gera propriedades públicas imutáveis por inicialização (`init`), construtor, métodos de deconstrução e igualdade baseada no conteúdo (dois registros `Produto` com os mesmos componentes são iguais). A expressão `p1 with { Preco = 18m };`

cria uma cópia de `p1` alterando apenas o membro indicado, sem modificar o original; além disso, `records` geram um método `ToString` que imprime os componentes, o que explica a saída legível ao escrever o objeto no console.



Observação

Em C#, deconstrução é o nome dado ao mecanismo que permite desmontar um objeto em múltiplas variáveis, em geral por atribuição em tupla. Por exemplo:

```
var p = new Produto(1, "Café", 12.5m);  
  
var (id, nome, preco) = p;    // deconstrução
```

Aqui, o compilador chama (direta ou indiretamente) um método denominado `Deconstruct`, que fornece os componentes do objeto. Em `records` posicionais, o compilador gera automaticamente um `Deconstruct(out int Id, out string Nome, out decimal Preco)` correspondente aos parâmetros posicionais.

A classe `Config` mostra uma autopropriedade com validação no acessor `set`. O acessor de leitura `get`; é automático e o acessor de escrita usa corpo de expressão. Dentro do `set`, `value` é o valor recebido na atribuição e `field` é um token contextual que referencia o campo de apoio gerado para a propriedade automática. A atribuição condicionada `field = value ?? throw new ArgumentNullException(nameof(value));` impede que `Tema` receba `null`, lançando uma exceção que usa `nameof(value)` para registrar o identificador do argumento. A criação `new Config { Tema = "claro" };` usa inicializador de objeto, que chama o construtor padrão e, em seguida, atribui às propriedades, acionando a validação do `set`.

A sequência com `Cliente? cliente = new Cliente(); cliente?.Total += 100m;` ilustra acesso condicional. O operador `?.` só avalia o membro seguinte se o receptor não for nulo; quando aplicado a uma atribuição composta, a operação só acontece quando existe um destino válido e, se o receptor for nulo, nada é executado. A leitura `cliente?.Total` segue a mesma regra e resulta em `null` quando o receptor é nulo, o que `WriteLine` trata como ausência de valor. A classe `Cliente` define uma autopropriedade `Total` do tipo `decimal`; como `decimal` é um tipo por valor, o padrão inicial é zero.

Por fim, um detalhe estrutural: em arquivos com instruções de nível superior, quaisquer declarações de tipos (`records` e `classes`) precisam vir depois dessas instruções, como no exemplo anterior. Reunindo tudo, o código apresenta o uso de namespace por `using`, programa de nível superior, escrita no console com interpolação de strings, variáveis com tipo explícito e inferido, anotações e operadores para nulabilidade, correspondência por padrões com `switch` em forma de expressão, função local estática com corpo de expressão e parâmetro opcional, `record` com igualdade por conteúdo e `with` para cópia

não destrutiva, propriedade automática validada acessando o campo de apoio via `field`, inicializador de objeto e acesso condicional em leitura e em atribuição composta. Tudo isso mostra recursos modernos da linguagem voltados a clareza, segurança e concisão, preservando semânticas precisas de tipo e de fluxo.

Considerando que os conceitos ilustrados nesse simples trecho de código-fonte podem ser complexos para o desenvolvedor iniciante em C#, vamos percorrer novamente o código-fonte fornecido.

Tudo começa pela ideia de que um programa é uma sequência de instruções que o computador executa na ordem em que aparecem. Em C#, essas instruções podem escrever textos na tela, fazer contas, criar objetos, testar condições e assim por diante. No nosso código de exemplo, a interação com o usuário acontece por meio de uma janela de console, que é uma janela de texto na qual o programa imprime mensagens e, eventualmente, pode ler entradas.

Logo no início surge a linha `using System;`. Aqui aparece o primeiro conceito de organização do código: o namespace. Em C#, **namespaces** são agrupamentos lógicos de tipos relacionados, algo análogo a pastas que guardam arquivos. O namespace `System` concentra muitos tipos básicos da biblioteca padrão, entre eles o tipo `Console`, que representa a tal janela de texto.

A diretiva `using System;` serve para dizer ao compilador que queremos usar esses tipos sem precisar escrever o nome completo. Assim, em vez de escrever `System.Console.WriteLine(...)` a cada vez, basta digitar `Console.WriteLine(...)`. Em seguida, o código não define explicitamente um método `Main`; ele usa as chamadas instruções de nível superior. Isso significa que podemos escrever comandos diretamente no arquivo e o compilador gera, nos bastidores, um ponto de entrada equivalente a um `Main` tradicional, executando as linhas na ordem em que aparecem. Há apenas uma regra estrutural importante: declarações de tipos, como `class` e `record`, precisam aparecer depois dessas instruções iniciais.

Com isso esclarecido, podemos observar a primeira instrução concreta:

```
Console.WriteLine("Olá! Vamos testar a sintaxe básica do C# 14.");
```

Aqui, `Console` é o tipo fornecido pela **biblioteca padrão**, `WriteLine` é um método desse tipo que escreve uma linha de texto no console, e o texto entre aspas é uma string, isto é, uma cadeia de caracteres. **Métodos** são blocos de código que podem receber entradas e realizar alguma ação; nesse caso, o método apenas envia a string para a tela e adiciona uma quebra de linha ao final.

Logo depois aparecem variáveis e tipos:

```
// Tipos e variáveis
int ano = 2026;
var nome = "Ana";
Console.WriteLine($"Boas-vindas, {nome}. Estamos em {ano}.");
```

Uma **variável** é um espaço na memória que recebe um nome e pode armazenar um valor. Aqui, `ano` e `nome` são variáveis. O tipo `int` indica que `ano` guarda números inteiros, enquanto `nome` guarda

um texto (string). O valor `2026` é um literal inteiro; `"Ana"` é um literal string. A palavra-chave `var` não torna a variável sem tipo; ela apenas diz ao compilador que o tipo será deduzido a partir do valor inicial. Nesse caso, como o literal `"Ana"` é uma string, o compilador decide que a variável `nome` é do tipo string. Depois de inferido, o tipo é fixo: não seria válido atribuir um número a `nome` mais tarde.

A terceira linha desse trecho introduz outro recurso importante: a interpolação de strings. A expressão `$"Boas-vindas, {nome}. Estamos em {ano}."` usa o símbolo `$` antes das aspas para indicar que o texto contém trechos que serão calculados, não apenas exibidos literalmente. As partes entre chaves, como `{nome}` e `{ano}`, são expressões C# avaliadas em tempo de execução. Se `nome` for `"Ana"` e `ano` for `2026`, o resultado final será a string `"Boas-vindas, Ana. Estamos em 2026."`, que então é enviada ao console por `WriteLine`.

Em seguida, o código passa a lidar com a possibilidade de ausência de valor, um conceito fundamental em linguagens modernas: o `null`. Na linha `string? apelido = null;`, a variável `apelido` é declarada como `string?`. O ponto de interrogação indica que essa variável pode assumir o valor especial `null`, que significa "não há valor". Em termos concretos, em vez de apontar para algum texto, ela pode estar vazia. A anotação com `?` faz parte do sistema de **nulabilidade** da linguagem, que ajuda a detectar, em tempo de compilação, usos perigosos de valores possivelmente nulos.

Para evitar erros ao lidar com `null`, o código usa dois operadores específicos. O primeiro é o operador de coalescência nula `??`:

```
var exibir = apelido ?? "visitante";  
Console.WriteLine(exibir);
```

A leitura intuitiva é: se `apelido` não for `null`, use o valor de `apelido`; se for `null`, use `"visitante"`. Em código mais longo, isso corresponderia a um condicional `if` que testa se `apelido` é nulo e escolhe qual string atribuir à variável `exibir`. O segundo operador é o `??=`, que realiza uma atribuição condicionada à nulidade:

```
apelido ??= "convidado";           // atribui só se for nulo  
Console.WriteLine(apelido);
```

Aqui, a lógica é: se `apelido` estiver `null`, atribua `"convidado"`; caso contrário, deixe o valor atual como está. É um modo compacto de preencher um valor padrão sem sobrescrever algo que já foi definido.

Mais adiante surge um exemplo que mistura tipos por valor, tipos por referência e o tipo base `object`: `object? x = 150;`

Em C#, tipos como `int` são tipos por valor: o número é armazenado diretamente. Já variáveis de tipos por referência armazenam um ponteiro para um objeto na memória. O `object` é o tipo base de quase todos os outros tipos; ele é capaz de representar muitos valores diferentes. Quando um `int` é atribuído a uma variável do tipo `object`, ocorre um processo chamado **boxing**: o valor inteiro

é embrulhado em um objeto para poder ser tratado como **object**. O ponto de interrogação em **object?** indica que aquela referência também poderia ser nula.

A seguir, esse valor é analisado por uma construção que ilustra pattern matching – uma expressão **switch** que tenta fazer o casamento de padrões:

```
string descricao = x switch
{
    null => "nulo",
    int n when n > 100 => "inteiro grande",
    string s => $"texto com {s.Length} chars",
    _ => "desconhecido"
};
Console.WriteLine(descricao);
```

Essa expressão avalia *x* e, para cada ramo do **switch**, verifica se o valor se encaixa ("casa") no padrão descrito. O primeiro braço compara com o literal **null**: se *x* for nulo, o resultado é a string **"nulo"**. O segundo ramo usa um padrão de tipo com guarda: **int n** verifica se *x* é do tipo inteiro; se for, armazena o valor em *n*, e a condição **when n > 100** impõe que esse inteiro seja maior que 100. Nessa situação, o resultado será **"inteiro grande"**. O terceiro ramo do **switch** testa se *x* é do tipo string (**string s**); se for, captura em *s* e usa outra string interpolada para informar o número de caracteres, por meio de *s.Length*. Por fim, o último ramo usa **_**, que funciona como um curinga: corresponde a qualquer valor que não tenha sido tratado pelos casos anteriores, e produz a string **"desconhecido"**. Diferentemente do **switch** em forma de comando, em que não há um valor único resultante, o **switch** em forma de expressão sempre devolve um valor, que aqui é atribuído à variável *descricao*. Note também que a sintaxe **=>** funciona em cada ramo do **switch** como um "então", cuja leitura seria: caso o teste da esquerda seja satisfeito, então a variável *descricao* recebe o valor da direita.

O código apresenta também uma função definida no próprio arquivo, logo depois das instruções de nível superior:

```
static decimal Calcular(decimal preco, decimal taxa = 0.12m)
    => preco * (1 + taxa);
Console.WriteLine(Calcular(100m));           // 112,00
```

Funções (ou **métodos**) são blocos de código que recebem parâmetros, realizam operações e devolvem um resultado. Esse método **Calcular** recebe um **preco** e uma **taxa**, ambos do tipo **decimal**, e devolve um **decimal**. O tipo **decimal** é bastante usado para cálculos monetários, porque representa números com casas decimais de maneira adequada a finanças, reduzindo erros de arredondamento típicos de representações binárias como **double**. O sufixo **m** nos literais (**0.12m** e **100m**) sinaliza que o literal é um **decimal**. O parâmetro **taxa** tem um valor padrão **0.12m**; isso significa que, ao chamar **Calcular(100m)**, a linguagem entende que a taxa é **0.12m** e faz a conta com essa taxa. A sintaxe **=>** é um corpo de expressão: indica que o valor de retorno é exatamente o resultado da expressão à direita. Seria equivalente a escrever um método com bloco entre chaves e uma instrução **return**.

Depois dessa parte, surgem os tipos definidos pelo programador (um **record** e duas **class**):

```
public record Produto(int Id, string Nome, decimal Preco);
public class Cliente { public decimal Total { get; set; } }
public class Config
{
    public string Tema
    {
        get;
        set => field = value ?? throw new ArgumentNullException(nameof(value));
    }
}
```

Em C#, tanto **record** quanto **class** definem tipos por referência, isto é, tipos cujas variáveis armazenam referências para objetos que vivem na memória gerenciada. A diferença principal não está na natureza física do objeto, mas no uso típico e no comportamento que o compilador gera automaticamente.

Uma **class** comum é pensada como um tipo orientado a comportamento e identidade: você costuma se preocupar com "este objeto" específico, que pode ir mudando seu estado interno ao longo do tempo. Se você criar dois objetos **Cliente** com os mesmos valores em suas propriedades, por padrão o C# considera que são objetos diferentes; a igualdade padrão de uma **class** é por referência, isto é, duas variáveis só são iguais se apontarem para a mesma instância na memória, a menos que alguém sobrescreva manualmente os métodos de comparação.

Um **record**, por outro lado, é voltado para dados. A ideia é que o foco está nos valores que ele carrega, não na identidade da instância. A declaração **public record Produto(int Id, string Nome, decimal Preco);** usa a forma posicional, em que os parâmetros do construtor também se tornam componentes de dados do **record**.

O compilador gera **propriedades públicas** correspondentes (**Id**, **Nome**, **Preco**), um **construtor**, métodos de comparação que consideram o conteúdo dessas propriedades e um método **ToString** que monta uma representação textual do objeto com seus campos. A característica que mais chama atenção é a igualdade estrutural: dois **Produto** com os mesmos **Id**, **Nome** e **Preco** são considerados iguais, mesmo se forem instâncias distintas na memória.

A expressão **var p1 = new Produto(1, "Café", 15m);** introduz outro conceito importante, que é o uso do operador **new**. Em C#, **new** é o operador que cria uma instância de um tipo por referência ou por valor. Quando se escreve **new Produto(1, "Café", 15m)**, o que acontece, de forma simplificada, é: o runtime reserva espaço na memória gerenciada para um objeto do tipo **Produto**, chama o construtor correspondente passando os argumentos **1**, **"Café"** e **15m**, e no fim devolve uma referência para esse objeto. A variável **p1** passa a guardar essa referência.

Em seguida, o código usa a sintaxe **with**:

```
var p2 = p1 with { Preco = 18m };
Console.WriteLine(p2);
```

A expressão `p1 with { Preco = 18m }`; cria um novo objeto `Produto` copiando todos os componentes de `p1` e alterando apenas o valor de `Preco` para `18m`. O `p1` permanece inalterado com `Preco = 15m`, enquanto o `p2` é outra instância. Como records geram um `ToString` amigável, a chamada `Console.WriteLine(p2)`; exibe uma descrição legível com os dados do produto.

A classe `Cliente` é bem simples:

```
public class Cliente { public decimal Total { get; set; } }
```

Antes de olhar para os detalhes dessa declaração, vale lembrar a ideia geral de classe e objeto em C#. Uma classe é um tipo definido pelo programador que descreve um molde ou modelo para certos dados e comportamentos. Os atributos (em C#, normalmente representados por campos e propriedades) guardam o estado do objeto, enquanto os métodos representam ações ou operações que podem ser executadas sobre esse estado. Em uma classe mais completa, além de propriedades como `Total`, seria comum haver métodos como `AplicarDesconto`, `FecharCompra` ou `CalcularImpostos`, que encapsulam regras de negócio relacionadas ao cliente.

Essa classe define o tipo `Cliente` com uma **propriedade** `Total` do tipo `decimal`. O par `get; set;` indica uma propriedade automática: o compilador cria um campo interno para guardar o valor, além dos métodos de leitura e escrita. Como `Total` é `decimal`, um tipo por valor, ele nasce com o valor padrão do tipo, que é zero, caso não seja explicitamente inicializado.

A declaração de `Total` introduz um conceito que aparece também na classe `Config`: o de **acessor**. Em C#, uma propriedade é formada por um tipo, um nome e, normalmente, dois blocos menores chamados acessores: o `get` e o `set`. O acessor `get` é o trecho de código que define como o valor da propriedade é lido; o acessor `set` define como o valor é escrito.

Quando se escreve `public decimal Total { get; set; }`, isso significa que `Total` é uma propriedade com acessor de leitura (`get`) e de escrita (`set`). O formato `{ get; set; }`, nesse caso, é uma propriedade automática: o compilador entende que você quer guardar um valor simples, cria um campo escondido para armazenar esse valor e gera os dois acessores de forma padrão, usando esse campo interno. Ao ler `cliente.Total`, na prática, o código chama o acessor `get`; ao escrever `cliente.Total += 100m`, o código chama o acessor `set`, passando `100m` como valor. Como `Total` é um `decimal`, um tipo por valor, ele nasce automaticamente com o valor padrão do tipo, que é zero, se você não atribuir nada explicitamente.

A classe `Config` mostra uma propriedade com validação explícita no acessor `set`:

```
public class Config
{
    public string Tema
    {
        get;
        set => field = value ?? throw new ArgumentNullException(nameof(value));
    }
}
```

Aqui, `Tema` é uma propriedade do tipo `string`. O acessor `get` continua automático, ou seja, o compilador gera o código padrão para retornar o valor armazenado. Já o acessor `set` é escrito manualmente com um corpo de expressão. Isso significa que você está substituindo o comportamento padrão ao escrever na propriedade por uma lógica personalizada.

Dentro desse acessor de escrita, a palavra `value` representa o valor que está sendo atribuído à propriedade: se alguém escreve `cfg.Tema = "claro"`, o `set` é chamado com `value` igual a `"claro"`. O token contextual `field` faz referência ao campo de apoio gerado pelo compilador para guardar o valor de `Tema`. Em uma propriedade automática simples, você não teria acesso direto a esse campo; aqui, o `field` permite que você intercepte a atribuição e, ao final, escreva de fato no local onde o valor é armazenado.

A expressão `value ?? throw new ArgumentNullException(nameof(value));` é avaliada antes da atribuição a `field`. Se `value` não for nulo, o resultado da expressão é o próprio `value` e ele é atribuído normalmente. Se `value` for null, em vez de permitir que o campo interno receba null, o código lança a exceção `ArgumentNullException`, interrompendo o fluxo do programa.

O uso de `nameof(value)` produz a string `"value"`, que é registrada na exceção como o nome do argumento problemático. Isso faz com que a propriedade `Tema` seja mantida, na prática, sempre com um texto não nulo: qualquer tentativa de atribuir null gera um erro explícito. Repare que, tanto aqui quanto em `Cliente`, o conceito de acessor é o mesmo; o que muda é que, em `Cliente`, os dois acessores são automáticos, enquanto em `Config` o `get` é automático e o `set` contém uma regra de validação escrita manualmente.

Além da estrutura interna da classe (propriedades, campos e métodos), é importante notar o que acontece quando o código usa essas definições. A classe, por si só, é somente a descrição de um tipo. Quando o programa executa `new Config()` ou `new Cliente()`, ele cria objetos, ou seja, instâncias concretas dessas classes em memória. Cada instância possui seu próprio conjunto de valores para os atributos definidos pela classe. Assim, dois objetos do tipo `Cliente` podem ter valores diferentes em `Total`, embora compartilhem a mesma definição de classe.

A criação de uma instância dessa classe usa outra convenção da linguagem:

```
var cfg = new Config { Tema = "claro" };  
Console.WriteLine(cfg.Tema);
```

A expressão `new Config { Tema = "claro" };` é um inicializador de objeto. Ela chama o construtor padrão de `Config` e, em seguida, realiza a atribuição `Tema = "claro"`, disparando o acessor `set` e, portanto, a validação. Essa forma compacta substitui duas etapas que poderiam ser escritas separadamente: criar o objeto e depois atribuir à propriedade.

Por fim, o código retorna ao tema da nulabilidade, dessa vez com um tipo por referência criado pelo programador:

```
Cliente? cliente = new Cliente();  
cliente?.Total += 100m;    // só avalia se cliente != null  
Console.WriteLine(cliente?.Total);
```

Aqui aparece novamente a distinção entre classe e objeto: `Cliente` (com C maiúsculo) é a definição de tipo; `cliente` (com c minúsculo) é uma variável que faz referência a uma instância concreta desse tipo. A mesma classe `Cliente` poderia ser usada para criar várias instâncias, como `cliente1` e `cliente2`, cada uma com seu próprio valor em `Total` e demais membros que eventualmente existam na classe.

A declaração `Cliente? cliente = new Cliente();` utiliza a anotação `?` para indicar que a variável `cliente` pode, em algum momento, ser `null`, ainda que ela seja inicializada agora com um objeto real. O operador `?.`, chamado de **operador de navegação condicional**, aparece em seguida. Na linha `cliente?.Total += 100m;`, a leitura é: se `cliente` não for nulo, acesse `Total` e some `100m` ao seu valor; se `cliente` for nulo, não faça nada. Isso evita acessos ilegais a membros de objetos inexistentes.

A mesma lógica se aplica a `cliente?.Total` na chamada de `WriteLine`: se `cliente` for nulo, o resultado da expressão é nulo; se não for, o valor de `Total` é usado. Como `Total` é um decimal, e `Total += 100m` só é executado se `cliente` não for nulo, o efeito é acumular o valor de compra apenas quando há um cliente válido.

Quando olhamos o código inteiro sob essa lente, fica mais fácil enxergar a ligação entre os conceitos:

- As instruções de nível superior simplificam a estrutura do programa.
- O `using` traz o namespace necessário para usar `Console` com menos verbosidade.
- Variáveis com tipo explícito e inferido permitem controlar e deduzir tipos a partir dos literais.
- A interpolação de strings integra valores numéricos e textuais em mensagens legíveis.
- O sistema de nulabilidade com `?`, `??`, `??=` e `?.` ajuda a tratar a ausência de valor de forma segura.
- O `switch` com padrões e guarda permite analisar valores de maneira expressiva.
- Métodos locais com corpo de expressão e parâmetros padrão tornam cálculos reutilizáveis e concisos.
- `records` representam dados com igualdade baseada no conteúdo e possibilitam cópia não destrutiva por meio de `with`.
- Propriedades automáticas, tanto simples quanto com validação via `field`, encapsulam o acesso a campos internos; inicializadores de objeto encurtam a criação de instâncias configuradas.
- A navegação condicional em leitura e em atribuição composta evita erros em presença de referências potencialmente nulas.

Tudo isso compõe um panorama de recursos contemporâneos da linguagem, voltados a tornar o código mais claro, mais conciso e com regras de tipos e de fluxo de execução bem definidas. Vamos agora a um segundo exemplo. Crie um novo projeto do tipo Aplicativo do Console no Visual Studio 2026 (ver figuras 10 a 12) e cole nele o código a seguir:

```
using System; // Importa o namespace básico da biblioteca padrão

//=====
// EXEMPLO SIMPLES DE CONDICIONAIS E LAÇOS EM C#
//=====
// - Cada instrução termina com ponto e vírgula (;)
// - Blocos de código são delimitados por chaves, { e }
// - Identificadores são case-sensitive: "aluno" e "Aluno" são nomes diferentes
//=====

Console.WriteLine("=== Demonstração de condicionais e laços em C# ===");
Console.WriteLine();

// -----
// 1) IDENTIFICADORES CASE-SENSITIVE
// -----

// Aqui criamos duas variáveis com nomes quase iguais:
// "aluno" (com 'a' minúsculo) e "Aluno" (com 'A' maiúsculo).
// Em C#, isso representa dois identificadores diferentes.
int aluno = 10;
int Aluno = 20;

Console.WriteLine("Exemplo de identificadores case-sensitive:");
Console.WriteLine($"aluno = {aluno}"); // Usa a variável com 'a' minúsculo
Console.WriteLine($"Aluno = {Aluno}"); // Usa a variável com 'A' maiúsculo
Console.WriteLine();

// -----
// 2) IF / ELSE IF / ELSE
// -----
// Vamos usar uma variável "nota" e classificar o desempenho do aluno.
// Somente UM dos blocos será executado, de acordo com o valor de "nota".
int nota = 75; // Altere este valor (de 0 a 100) e veja o resultado mudar

Console.WriteLine("Exemplo de if / else if / else com a variável 'nota'.");
Console.WriteLine($"Nota = {nota}");

if (nota < 50)
{
    // Este bloco executa se a condição (nota < 50) for verdadeira
    Console.WriteLine("Resultado: abaixo da média.");
}
else if (nota < 70)
{
    // Este bloco é testado se o anterior NÃO é verdadeiro
    Console.WriteLine("Resultado: na média.");
}
```

```
else if (nota < 90)
{
    Console.WriteLine("Resultado: bom desempenho.");
}
else
{
    // Este bloco é o "caso padrão", quando nenhuma condição anterior é verdadeira
    Console.WriteLine("Resultado: excelente desempenho!");
}

Console.WriteLine();

// -----
// 3) SWITCH
// -----
// Agora usamos um "switch" para reagir a um comando simples.
// Valor possível em "comando": 'A' (Atacar), 'D' (Defender), 'F' (Fugir).
char comando = 'A'; // Experimente mudar para 'D', 'F' ou outro caractere

Console.WriteLine("Exemplo de switch com um comando simples (A, D ou F).");
Console.WriteLine($"Comando = {comando}");

switch (comando)
{
    case 'A':
        // Se o comando for 'A', este bloco é executado
        Console.WriteLine("Ação escolhida: Atacar.");
        break; // Encerra o switch aqui

    case 'D':
        Console.WriteLine("Ação escolhida: Defender.");
        break;

    case 'F':
        Console.WriteLine("Ação escolhida: Fugir.");
        break;

    default:
        // Este bloco executa quando o valor de "comando" não é A, D nem F
        Console.WriteLine("Ação desconhecida.");
        break;
}

Console.WriteLine();

// -----
// 4) LAÇO FOR
// -----
// O laço for é útil para repetir algo um número conhecido de vezes.
// Aqui vamos contar de 1 a 5.
Console.WriteLine("Exemplo de laço for: contar de 1 até 5.");

for (int i = 1; i <= 5; i++)
{
```

```
// Dentro das chaves está o corpo do laço.
// A cada repetição, "i" é incrementado em 1 (i++)
Console.WriteLine($"i vale {i}");
}
Console.WriteLine();

// -----
// 5) LAÇO WHILE
// -----
// O while repete ENQUANTO a condição for verdadeira.
// Aqui fazemos uma contagem regressiva simples.
Console.WriteLine("Exemplo de laço while: contagem regressiva a partir de 3.");

int contador = 3; // Valor inicial, você pode alterar

while (contador > 0)
{
    // Este bloco roda enquanto contador for maior que 0
    Console.WriteLine($"Contador = {contador}");

    // Diminui o valor de contador em 1 a cada repetição
    contador--;
}
// Quando contador chega a 0, a condição (contador > 0) fica falsa
// e o laço while termina
Console.WriteLine("Fim da contagem regressiva.");
Console.WriteLine();
Console.WriteLine("Programa encerrado. Pressione ENTER para sair.");
Console.ReadLine(); // Mantém a janela do console aberta até o usuário apertar
ENTER
```

Compile e execute esse código para analisar a sintaxe e seu funcionamento. Todas as explicações detalhadas sobre a sintaxe estão no próprio código-fonte, na forma de comentários. Você pode alterar os valores nos pontos indicados, compilar novamente e observar o que mudou na saída do console. Esse ciclo de modificar o código, recompilar e verificar as diferenças é uma etapa natural do processo de desenvolvimento de software. Faça o mesmo com o código a seguir, cujo foco são classes (definição, métodos e propriedades) e objetos (instanciação, uso das propriedades e métodos).

```
// Exemplo completo em C# focado em CLASSES e OBJETOS,
// com um minijogo de batalha simples em console.
//
// O Visual Studio usa este arquivo como código-fonte,
// mas a ideia aqui é didática: mostrar claramente a diferença
// entre definir classes (molde) e criar objetos (instâncias) e
// usar suas propriedades e métodos.

using System;

namespace MiniJogoDeBatalha
{
    // -----
    // CLASSE Jogador
```



```
// -----  
//  
// Esta é a DEFINIÇÃO da classe Jogador.  
// Ela descreve quais dados um jogador tem (propriedades)  
// e quais ações ele pode executar (métodos).  
//  
// Importante: ao declarar a classe, nenhum jogador é criado  
// ainda. A classe é apenas o molde. Os objetos serão  
// criados com "new".  
class Jogador  
{  
    // Propriedade que representa o nome do jogador.  
    // É um ATRIBUTO do jogador.  
    public string Nome { get; set; }  
  
    // Propriedade que indica quanta vida o jogador tem.  
    // A leitura é pública, mas só a própria classe pode alterar (set  
privado).  
    public int Vida { get; private set; }  
  
    // Propriedade que indica a força do ataque.  
    public int Forca { get; set; }  
  
    // Propriedade que acumula pontos ganhos durante o jogo.  
    public int Pontuacao { get; private set; }  
  
    // Propriedade somente de leitura que indica se o jogador ainda está  
vivo.  
    // Ela é calculada a partir da Vida.  
    public bool EstaVivo => Vida > 0;  
  
    // Construtor da classe Jogador.  
    //  
    // Quando usamos "new Jogador(...)" em outro lugar do código,  
    // este método é executado para INICIALIZAR o objeto recém-criado.  
    public Jogador(string nome, int vidaInicial, int forcaInicial)  
    {  
        Nome = nome;  
        Vida = vidaInicial;  
        Forca = forcaInicial;  
        Pontuacao = 0;  
    }  
  
    // Método Atacar:  
    //  
    // Recebe um Inimigo como parâmetro e diminui a vida dele.  
    // Aqui vemos a interação entre objetos: um objeto Jogador chama  
    // um método que afeta um objeto Inimigo.  
    public void Atacar(Inimigo alvo)  
    {  
        if (!EstaVivo)  
        {  
            Console.WriteLine($"{Nome} não pode atacar porque está sem  
vida.»);  
            return;  
        }  
    }  
}
```

```

        if (!alvo.EstaVivo)
        {
            Console.WriteLine($"{Nome} não pode atacar {alvo.Nome} porque
ele já foi derrotado.");
            return;
        }

        Console.WriteLine($"{Nome} ataca {alvo.Nome} causando {Forca} de
dano.");
        alvo.ReceberDano(Forca);
    }

    // Método ReceberDano:
    //
    // Altera a PROPRIEDADE Vida deste objeto Jogador.
    // Note que a alteração do estado do objeto acontece dentro da classe,
    // usando a propriedade como se fosse um campo comum.
    public void ReceberDano(int dano)
    {
        Vida -= dano;

        if (Vida < 0)
        {
            Vida = 0;
        }

        Console.WriteLine($"{Nome} agora tem {Vida} de vida.");
    }

    // Método GanharPontos:
    //
    // Aumenta a pontuação do jogador. Aqui a classe define COMO os pontos
    // são atualizados; o código que usa o Jogador só precisa chamar o
método.
    public void GanharPontos(int pontos)
    {
        Pontuacao += pontos;
        Console.WriteLine($"{Nome} ganhou {pontos} pontos. Pontuação atual:
{Pontuacao}.");
    }
}

// -----
// CLASSE Inimigo
// -----
//
// Outra classe, com sua própria combinação de propriedades
// e métodos. Ela também é apenas um molde até que alguém
// crie objetos Inimigo com o operador "new".
class Inimigo
{
    // Nome do inimigo.
    public string Nome { get; set; }

    // Vida atual do inimigo.
    public int Vida { get; private set; }
}

```

```
// Força do ataque do inimigo.
public int Forca { get; set; }

// Pontos de recompensa que o jogador ganha ao derrotar este inimigo.
public int PontosDeRecompensa { get; set; }

// Propriedade calculada que indica se o inimigo ainda está vivo.
public bool EstaVivo => Vida > 0;

// Construtor da classe Inimigo.

public Inimigo(string nome, int vidaInicial, int forcaInicial, int
pontosDeRecompensa)
{
    Nome = nome;
    Vida = vidaInicial;
    Forca = forcaInicial;
    PontosDeRecompensa = pontosDeRecompensa;
}

// Método Atacar:
//
// Recebe um Jogador como parâmetro e diminui a vida dele.
public void Atacar(Jogador alvo)
{
    if (!EstaVivo)
    {
        Console.WriteLine($"{Nome} não pode atacar porque já foi
derrotado.»);
        return;
    }

    if (!alvo.EstaVivo)
    {
        Console.WriteLine($"{Nome} não pode atacar {alvo.Nome} porque
ele já está sem vida.");
        return;
    }

    Console.WriteLine($"{Nome} ataca {alvo.Nome} causando {Forca} de
dano.");
    alvo.ReceberDano(Forca);
}

// Método ReceberDano:
//
// Diminui a vida do inimigo e escreve o novo valor no console.
public void ReceberDano(int dano)
{
    Vida -= dano;

    if (Vida < 0)
    {
        Vida = 0;
    }
}
```

```

        Console.WriteLine($"{Nome} agora tem {Vida} de vida.");
    }
}

// -----
// CLASSE Jogo
// -----
//
// Esta classe organiza o fluxo do minijogo.
// Ela cria OBJETOS a partir das classes Jogador e Inimigo,
// chama métodos e acessa propriedades desses objetos.
//
// Aqui fica bem visível a diferença entre:
// - Definir a classe (Jogador, Inimigo).
// - Criar objetos (new Jogador(...), new Inimigo(...)).
// - Usar métodos e propriedades (heroi.Atacar(...), slime.Vida etc.).
class Jogo
{
    // Método principal desta classe: ele coordena a execução do minijogo.
    public void Executar()
    {
        Console.WriteLine("=== Minijogo: batalha entre classes e objetos
===");

        Console.WriteLine();
        Console.WriteLine("Nesta tela vamos ver classes (moldes) sendo
usadas para criar objetos (instâncias).");
        Console.WriteLine();

        // -----
        // 1) Criação de um OBJETO Jogador
        // -----
        //
        // Aqui usamos o operador "new" para criar UMA instância da classe
Jogador.

        // A variável 'heroi' é uma referência a esse objeto na memória.
        //
        // Jogador = nome da CLASSE (tipo definido pelo programador).
        // heroi   = nome do OBJETO (variável que aponta para a instância).
        var heroi = new Jogador(nome: "Herói", vidaInicial: 100,
forcaInicial: 20);

        // A seguir, acessamos PROPRIEDADES do objeto 'heroi'.
        // Os códigos heroi.Nome e heroi.Vida estão lendo o estado desse
objeto específico.
        Console.WriteLine("Estado inicial do jogador:");
        Console.WriteLine($"{Nome: {heroi.Nome}}");
        Console.WriteLine($"{Vida: {heroi.Vida}}");
        Console.WriteLine($"{Força: {heroi.Força}}");
        Console.WriteLine();

        // -----
        // 2) Criação de DOIS objetos Inimigo a partir da MESMA classe
        // -----
        //
        // Aqui fica muito claro que a classe é o molde:
        // usamos o mesmo tipo Inimigo para criar objetos diferentes

```

```
// (slime e goblin), com valores próprios em suas propriedades.
var slime = new Inimigo(nome: "Slime", vidaInicial: 30,
forçaInicial: 5, pontosDeRecompensa: 10);
var goblin = new Inimigo(nome: "Goblin", vidaInicial: 40,
forçaInicial: 8, pontosDeRecompensa: 20);

Console.WriteLine("Estado inicial dos inimigos:");
Console.WriteLine($"Inimigo 1: {slime.Nome} | Vida: {slime.Vida} |
Força: {slime.Força} | Recompensa: {slime.PontosDeRecompensa}");
Console.WriteLine($"Inimigo 2: {goblin.Nome} | Vida: {goblin.Vida} |
Força: {goblin.Força} | Recompensa: {goblin.PontosDeRecompensa}");
Console.WriteLine();

Console.WriteLine("Pressione ENTER para iniciar a primeira batalha
(Herói vs Slime)...");
Console.ReadLine();

// -----
// 3) Primeiro combate: o jogador luta contra o Slime
// -----
//
// Perceba que o método Batalhar recebe DOIS OBJETOS:
// um objeto Jogador e um objeto Inimigo.
// Dentro de Batalhar, esses objetos terão seus MÉTODOS chamados
// (Atacar, ReceberDano, GanharPontos) e suas PROPRIEDADES lidas.
Batalhar(heroi, slime);

// Se o jogador morrer na primeira batalha, o jogo termina.
if (!heroi.EstaVivo)
{
    Console.WriteLine();
    Console.WriteLine("O jogador foi derrotado pelo Slime. Fim de
jogo.");
    return;
}

Console.WriteLine();
Console.WriteLine("Pressione ENTER para iniciar a segunda batalha
(Herói vs Goblin)...");
Console.ReadLine();

// -----
// 4) Segundo combate: o jogador luta contra o Goblin
// -----
Batalhar(heroi, goblin);

Console.WriteLine();
Console.WriteLine("Estado final do jogador após as batalhas:");
Console.WriteLine($"Nome: {heroi.Nome}");
Console.WriteLine($"Vida: {heroi.Vida}");
Console.WriteLine($"Pontuação: {heroi.Pontuacao}");
Console.WriteLine();

Console.WriteLine("Demonstração concluída.");
Console.WriteLine("Observe como as CLASSES definem o que os
personagens podem fazer");
```

```
        Console.WriteLine("e como os OBJETOS representam personagens  
específicos no jogo.");  
    }  
  
    // Método auxiliar Batalhar:  
    //  
    // Recebe um Jogador e um Inimigo (objetos já criados) e faz um loop de  
turnos.  
    // Em cada turno, o jogador ataca e, se o inimigo ainda estiver vivo,  
    // o inimigo contra-ataca.  
    //  
    // Esse método não cria classes novas; apenas usa os objetos existentes.  
    private void Batalhar(Jogador jogador, Inimigo inimigo)  
    {  
        Console.WriteLine();  
        Console.WriteLine($"--- Início da batalha: {jogador.Nome} vs  
{inimigo.Nome} ---");  
        Console.WriteLine();  
  
        // Enquanto ambos estiverem vivos, a batalha continua.  
        while (jogador.EstaVivo && inimigo.EstaVivo)  
        {  
            // OBJETO jogador chamando seu MÉTODO Atacar,  
            // passando o OBJETO inimigo como argumento.  
            jogador.Atacar(inimigo);  
  
            Console.WriteLine();  
  
            // Se o inimigo ainda estiver vivo, ele contra-ataca.  
            if (inimigo.EstaVivo)  
            {  
                inimigo.Atacar(jogador);  
                Console.WriteLine();  
            }  
  
            Console.WriteLine("Pressione ENTER para avançar para o próximo  
turno...");  
            Console.ReadLine();  
        }  
  
        // Quando o loop termina, um dos dois (ou ambos) não está mais vivo.  
        if (jogador.EstaVivo)  
        {  
            Console.WriteLine($"{inimigo.Nome} foi derrotado!");  
  
            // Aqui, um objeto (jogador) usa um método para atualizar sua  
pontuação  
            // a partir de uma propriedade de outro objeto (inimigo.  
PontosDeRecompensa).  
            jogador.GanharPontos(inimigo.PontosDeRecompensa);  
        }  
        else  
        {  
            Console.WriteLine($"{jogador.Nome} foi derrotado.");  
        }  
  
        Console.WriteLine($"--- Fim da batalha: {jogador.Nome} vs {inimigo.
```

```
Nome} ---");
    }
}

// -----
// CLASSE Program
// -----
//
// Esta classe contém o ponto de entrada do programa:
// o método Main. É aqui que a execução realmente começa.
//
// Dentro de Main criamos um OBJETO do tipo Jogo e chamamos
// o MÉTODO Executar(), que controla todo o fluxo.
class Program
{
    static void Main()
    {
        // Criação de um objeto da classe Jogo.
        // 'jogo' é uma instância concreta de Jogo.
        var jogo = new Jogo();

        // Aqui chamamos um MÉTODO do objeto Jogo.
        // Esse método, por sua vez, cria outros objetos (Jogador, Inimigo)
        // e aciona seus métodos e propriedades.
        jogo.Executar();

        // Após Executar terminar, o programa chega ao fim.
    }
}
}
```

A linguagem C# vive dentro de uma plataforma maior, chamada .NET, composta de um runtime e de bibliotecas. O **.NET Framework** é uma plataforma dividida em componentes: em primeiro lugar, o Common Language Runtime (CLR), que é o ambiente de execução virtual responsável por executar o código gerenciado; em segundo lugar, a biblioteca de classes – Base Class Library (BCL) e demais bibliotecas de framework.

É possível enxergar o .NET como um conceito geral de plataforma: um runtime que implementa a especificação CLI, acompanhado de uma biblioteca padrão, sobre os quais linguagens como C#, Visual Basic e F# trabalham.

Dentro desse quadro, o .NET Framework é a implementação original, voltada ao Windows, que consolidou a pilha CLR + BCL + bibliotecas de aplicação (Windows Forms, ASP.NET, WPF etc.). A BCL é apresentada como o núcleo de tipos e APIs do Framework, sobre o qual se constroem bibliotecas de nível mais alto.

Com o tempo, a Microsoft introduziu uma linha mais recente, inicialmente chamada **.NET Core** e depois unificada sob o nome .NET nas versões numeradas a partir de .NET 5. Em termos conceituais, tanto o .NET Framework quanto o .NET Core/.NET 5+ podem ser entendidos como implementações diferentes do mesmo modelo: CLR + BCL + bibliotecas adicionais, com conjuntos de APIs em grande parte compatíveis, mas embalados e evoluídos em ritmos distintos.

A linguagem C# é definida formalmente pela especificação ECMA-334 (ECMA International, 2023). Ela não é o .NET, mas sim uma linguagem que compila para a infraestrutura de linguagem comum (CLI, do inglês Common Language Infrastructure): o código-fonte em C# é compilado para um código intermediário (CIL, do inglês Common Intermediate Language) e metadados compatíveis com a CLI, que serão executados pelo CLR.

O CLR é um runtime utilizável por diferentes linguagens, e C# é apenas uma delas. Em outras palavras, C# fica acima do CLR: ela determina a sintaxe e as regras da linguagem; o resultado da compilação é alimentado ao runtime de .NET (seja o do .NET Framework, seja o do .NET Core/.NET 5+), no qual de fato será executado.

A BCL é o elo que faz C# e as demais linguagens .NET compartilharem o mesmo alicerce de tipos e APIs. Ela é um conjunto de bibliotecas fundamentais da plataforma, intimamente ligado ao `Common Type System` e ao runtime, além de ser usado por todas as linguagens que visam a CLI. O C# define a forma do código; a BCL define o "vocabulário" de tipos e operações básicos que esse código usa, independentemente de estar rodando sobre o .NET Framework ou sobre o .NET mais recente.

O CLR é responsável por carregar assemblies, verificar tipos, gerenciar memória com coleta de lixo e fornecer serviços de segurança, exceções, interoperabilidade e diagnóstico. Dentro do CLR, o JIT (Just-In-Time compiler) é o componente que transforma o código intermediário (CIL) em código nativo específico da CPU no momento da execução. O compilador C# produz módulos gerenciados contendo IL; o CLR carrega esses módulos, verifica o código e chama o JIT para gerar código nativo, que então é executado diretamente pelo processador. Nessa cadeia, o JIT não é algo separado de .NET ou de C#, mas um estágio interno do runtime usado por qualquer linguagem que gere IL compatível com a CLI.

Por fim, o Visual Studio entra nesse cenário não como parte do runtime, mas como o ambiente de desenvolvimento que orquestra esses componentes. Ele é uma IDE desenhada com o desenvolvimento .NET em mente, que simplifica a escrita, a compilação e a depuração de código C#. É um ambiente integrado para criar, executar e depurar aplicações escritas em diversas linguagens .NET, reforçando que ele hospeda os compiladores e as ferramentas que produzem assemblies gerenciados a partir do código C#.

O Visual Studio é a peça central do fluxo de trabalho do desenvolvedor, ainda que separado da infraestrutura de execução em si. Ele se relaciona com C#, CLR e BCL como uma espécie de "casca" produtiva: oferece editor, designers, build system e depurador sobre os compiladores C# e o SDK de .NET, facilitando o uso coordenado de todos esses elementos.

Consolidando todas essas informações, o C# é uma linguagem padronizada cuja implementação principal compila para a infraestrutura de linguagem comum, e o CLR é o runtime que executa esse código intermediário (CIL), conforme definido pelo padrão ECMA-335 (ECMA International, 2012). Já a BCL é a coleção de tipos básicos que concretiza, do lado das bibliotecas, o modelo de tipos da CLI; enquanto o .NET Framework e o .NET Core/.NET 5+ são pacotes que combinam versões específicas de CLR, BCL e bibliotecas adicionais. Por fim, o Visual Studio é o ambiente de desenvolvimento que integra compiladores, depuradores e designers para produzir assemblies destinados a essa infraestrutura.

Tudo isso constitui o ecossistema .NET exemplificado na figura 15 e que mostra a relação entre os diversos componentes.

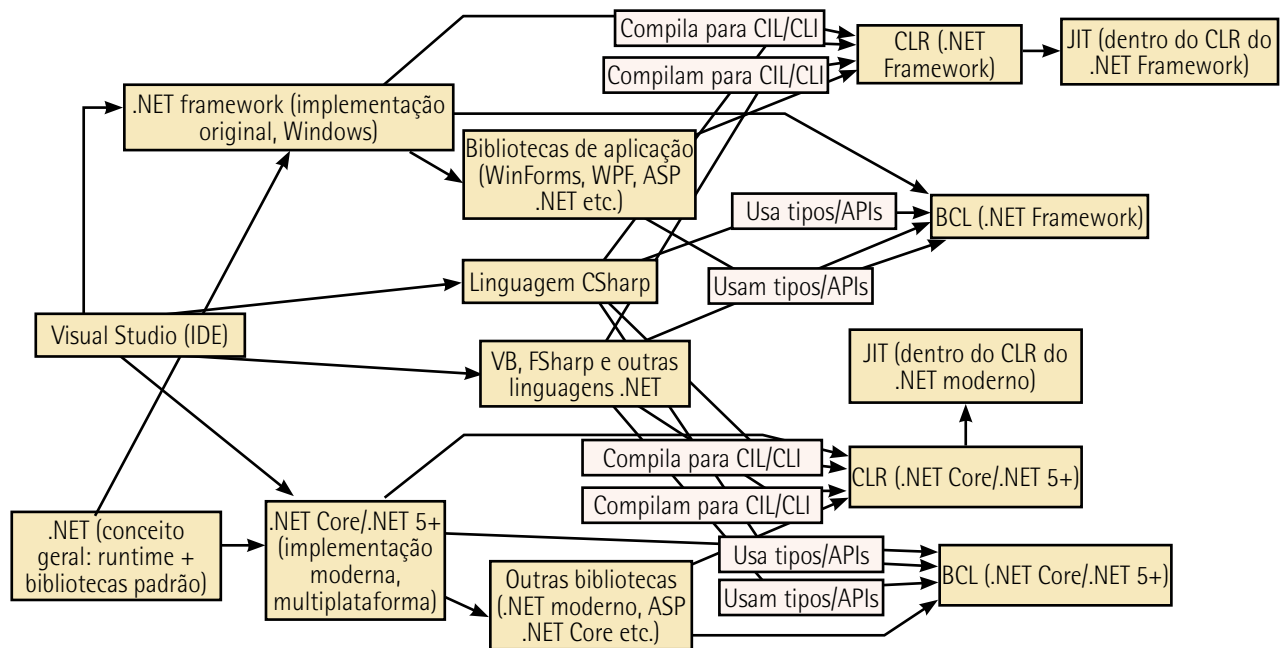


Figura 15- Relacionamento entre os componentes do ecossistema .NET

Runtime é o conjunto de componentes que fazem o programa ganhar vida na hora de executar. O arquivo do programa, sozinho, é só texto ou código gravado em disco. Quando você pede para executar uma aplicação .NET, o runtime entra em ação: carrega o programa na memória, traduz o código para instruções que o processador entende, administra o uso de memória, cuida da execução passo a passo e encerra tudo de forma organizada quando o programa termina. Em termos simples, o runtime é o ambiente que acompanha o programa enquanto ele está executando, garantindo que ele tenha tudo o que precisa para funcionar.

Biblioteca, nesse contexto, é um conjunto de funcionalidades prontas, empacotadas em arquivos de código que o desenvolvedor pode reaproveitar. Em vez de cada programador ter que escrever do zero tudo o que um programa faz (por exemplo, mostrar texto na tela, acessar arquivos, conversar com a internet, trabalhar com datas e números grandes e assim por diante), essas tarefas comuns ficam reunidas em bibliotecas. O código da aplicação evoca essas bibliotecas quando precisa de algo. Assim, o desenvolvedor escreve menos código, com menos chance de erro, apoiando-se em componentes já testados.

SDK é a sigla para Software Development Kit, ou kit de desenvolvimento de software. A ideia é parecida com um kit de ferramentas que você compra para realizar um tipo de trabalho específico. Se você quer montar móveis, compra um kit com chaves de fenda, parafusos, manual e, às vezes, até gabaritos. No mundo da programação acontece algo semelhante: um SDK é um pacote que reúne tudo de que um desenvolvedor precisa para criar programas para uma determinada plataforma ou tecnologia.

O **dotnet CLI** ocupa o lugar de ferramenta fundamental de linha de comando. A documentação oficial o descreve como uma interface multiplataforma para desenvolver, compilar, executar e publicar aplicações .NET, distribuída junto com o SDK do .NET; ao instalar o SDK, o desenvolvedor recebe a CLI, o runtime e bibliotecas de suporte em um único conjunto de ferramentas.

Do ponto de vista conceitual, o `dotnet CLI` organiza-se em torno do executável `dotnet`, que funciona como uma espécie de driver ou despachante de comandos. Ao invocar `dotnet` no terminal, seguido de um verbo como `new`, `build` ou `run`, o driver carrega o SDK instalado, interpreta os argumentos e delega o trabalho aos compiladores e ferramentas apropriados, seja em Windows, macOS ou Linux. Essa estrutura é exatamente a mesma independentemente de o projeto usar C#, F# ou Visual Basic, e vale para as versões recentes do SDK utilizadas com .NET 10 e C# 14.

A relação entre o `dotnet CLI` e o cenário C# 14/Visual Studio 2026 pode ser descrita como uma convivência de camadas. O Visual Studio 2026 é uma IDE que integra de forma nativa .NET 10 e C# 14, fornecendo editor, depuração, designers e integração com serviços de nuvem, mas ele se apoia justamente no SDK do .NET – que, por sua vez, inclui a CLI – para compilar e executar os projetos. Assim, o mesmo projeto que é aberto na IDE pode ser manipulado inteiramente pelo terminal, usando os mesmos arquivos de projeto (`.csproj`) e os mesmos targets de compilação; a CLI fornece uma superfície estável que permite alternar entre ambiente gráfico e script de automação sem alterar o código-fonte.

O modelo de comandos da CLI é relativamente simples, mas muito expressivo. A sintaxe básica segue o padrão `dotnet <comando> [opções]`, em que cada comando corresponde a uma etapa do ciclo de vida da aplicação. Com `dotnet new`, cria-se um novo projeto ou solução a partir de um modelo, definindo o tipo de aplicação (console, biblioteca, API, aplicação web, testes, entre outros) e o framework-alvo, como `net10.0` para projetos que exploram o C# 14.



Observação

Framework é basicamente o conjunto de regras e peças prontas que a sua aplicação usa para existir e funcionar. Imagine que você vai construir uma casa, você pode tentar fazer tudo do zero, desde moldar tijolo até desenhar fiação, ou pode seguir um padrão de construção já estabelecido, com tipos de paredes, medidas de portas e janelas e normas de elétrica e hidráulica. Esse padrão predefinido, com material e normas, seria o framework da sua casa. No desenvolvimento em .NET, quando mencionamos um "framework-alvo, como `net10.0`", estamos dizendo: "este programa foi feito para executar seguindo as regras, as bibliotecas e as capacidades específicas dessa versão da plataforma .NET". Isso define quais recursos da linguagem C# você pode usar, quais bibliotecas da própria Microsoft estão disponíveis, em que sistemas esse programa poderá executar e como o código será compilado. Em termos simples, o framework é o ambiente padrão para o qual o programa é construído.

Comandos como `dotnet build`, `dotnet run`, `dotnet test` e `dotnet publish` conduzem, respectivamente, à compilação, à execução, à execução de testes automatizados e à geração de artefatos prontos para distribuição. A documentação da Microsoft descreve `dotnet build`, por exemplo, como a operação responsável por compilar o projeto e todas as suas dependências em binários, usando MSBuild nos bastidores e permitindo parâmetros avançados de configuração, `framework-alvo` e `runtime de destino`.



Observação

O MSBuild pode ser entendido como o "pedreiro automatizado" que pega todos os arquivos do seu projeto e monta o resultado final, que é o programa pronto para executar. Quando você escreve código, o computador não entende diretamente esse texto; é preciso transformar esse código em arquivos binários (executáveis ou bibliotecas) que a máquina consiga executar. Esse processo se chama compilação. O comando `dotnet build` é a forma simpática de pedir: "por favor, pegue este projeto, todas as bibliotecas de que ele depende, aplique as regras do `framework-alvo` e gere os arquivos prontos para uso". Quem faz esse trabalho pesado nos bastidores do comando é o MSBuild.

O MSBuild lê os arquivos de configuração do projeto (os `.csproj`, por exemplo), nos quais estão descritos o nome do projeto, as dependências externas, o `framework-alvo`, configurações de compilação (como modo de depuração ou de produção) e uma série de detalhes técnicos. A partir dessas informações, ele organiza as etapas: compila cada parte do código na ordem correta, junta tudo, resolve dependências, coloca os arquivos resultantes nas pastas apropriadas e sinaliza se houve erros.

A gestão de dependências também é incorporada a esse fluxo. Com o modelo atual do SDK, boa parte dos comandos realiza restauração implícita de pacotes; sempre que uma compilação ou uma criação de novo projeto exige que pacotes NuGet sejam resolvidos, a CLI executa essa restauração de forma automática. Ainda assim, o comando `dotnet restore` permanece disponível e recomendado em cenários em que é importante separar claramente as etapas de restauração e compilação, como em pipelines de integração contínua em servidores de build. O resultado é um conjunto de comandos que pode ser usado de maneira interativa por um desenvolvedor em sua máquina ou acionado por scripts em servidores CI/CD, mantendo a mesma semântica em ambos os contextos.

Vejamos alguns detalhes importantes não mencionados anteriormente:

- Seu projeto eventualmente depende de bibliotecas externas, que ficam disponíveis em um repositório chamado NuGet. Cada biblioteca é um pacote NuGet, com nome e versão. Resolver pacotes NuGet significa descobrir exatamente quais versões desses pacotes devem ser usadas, baixar o que estiver faltando da internet e deixar tudo registrado para que a compilação saiba

onde estão essas bibliotecas. Depois que essa etapa termina, dizemos que os pacotes foram resolvidos: o projeto tem todas as dependências definidas e disponíveis, prontas para serem utilizadas na compilação.

- Pipelines de integração contínua são sequências automáticas de passos que executam em um servidor sempre que alguém altera o código de um sistema. Em vez de um desenvolvedor se lembrar de compilar, executar testes, verificar se os pacotes foram restaurados e, às vezes, preparar o que será publicado, tudo isso é descrito em um roteiro (pipeline). Esse roteiro é executado de forma automática a cada alteração enviada ao repositório de código. A ideia é detectar problemas cedo: se uma mudança quebrou a compilação, falhou em testes ou trouxe uma dependência incompatível, o pipeline acusa o problema logo após o envio do código.
- Servidores de build são máquinas (físicas ou na nuvem) dedicadas a executar esses pipelines. Em vez de compilar apenas no computador pessoal de cada desenvolvedor, a organização usa um ou mais servidores configurados especificamente para isso. Nesses servidores, comandos como `dotnet restore` e `dotnet build` são executados de maneira previsível, em um ambiente controlado, com versões fixas de ferramentas e SDKs. Isso reduz o risco de "funciona na minha máquina, mas quebra no servidor", já que o padrão oficial de compilação passa a ser o que roda nesse servidor.
- CI/CD é a sigla para continuous integration/continuous delivery (ou continuous deployment, dependendo do contexto). Continuous integration é justamente essa prática de integrar alterações ao repositório principal com frequência, usando pipelines automatizados de compilação, restauração de dependências e testes em servidores de build. Continuous delivery/deployment amplia essa automação até as etapas de entrega: depois que o código passa por todas as verificações, o mesmo pipeline pode preparar versões instaláveis, publicar pacotes, atualizar um site ou um serviço em produção, tudo com o mínimo possível de intervenção manual. Dizer que a CLI pode ser acionada por scripts em servidores CI/CD significa que os mesmos comandos `dotnet` que o desenvolvedor digita na própria máquina são utilizados dentro desses pipelines automatizados, garantindo o mesmo comportamento tanto no uso interativo quanto no uso em automação.

Ao instalar o SDK, o desenvolvedor recebe dezenas de modelos pré-instalados para a criação de projetos e arquivos: aplicações de console, bibliotecas de classes, testes de unidade, aplicações ASP.NET Core (incluindo variações com Angular ou React), entre outros. O comando `dotnet new` instancia esses modelos e gerencia a pesquisa, a instalação e a remoção de modelos adicionais, inclusive personalizados. É possível criar modelos próprios, empacotá-los como pacotes NuGet e disponibilizá-los em feeds internos ou públicos; a CLI inclui um mecanismo de template engine responsável por copiar arquivos de origem, substituir parâmetros e executar operações de pós-processamento quando um novo projeto é gerado. Essa facilidade permite que equipes definam padrões internos de arquitetura, organização de camadas, configuração de testes e ferramentas de observabilidade, garantindo que cada novo projeto C# 14 criado via CLI já nasça aderente às diretrizes da organização.

Essa integração entre CLI, SDK e modelos se estende ao uso em automação. A própria documentação da Microsoft apresenta cenários nos quais a CLI é utilizada em servidores de integração contínua, em scripts que instalam o SDK, restauram dependências, compilam, executam testes e publicam artefatos com comandos `dotnet` encadeados. O fato de a CLI ser multiplataforma facilita a construção de pipelines que rodam em Windows, Linux ou contêineres Docker, algo muito relevante em projetos modernos que empregam .NET 10 para serviços distribuídos, APIs e aplicações em nuvem. Em todos esses ambientes, os mesmos comandos funcionam da mesma forma, o que reduz discrepâncias entre "funciona na minha máquina" e "funciona no servidor".

No contexto específico de C# 14, o `dotnet CLI` funciona como a camada operacional que permite explorar as novidades da linguagem sem depender exclusivamente da IDE. Recursos introduzidos nas versões recentes, como melhorias no tratamento de `Span<T>`, extensão de `partial` para construtores e eventos, operadores de atribuição condicionais a nulos e outros refinamentos, são compilados pelo mesmo toolchain exposto pela CLI, desde que o projeto seja configurado para uma versão de linguagem e um framework compatíveis, como .NET 10. O desenvolvedor pode, por exemplo, criar um projeto voltado a essas funcionalidades com `dotnet new`, ajustar o framework-alvo no arquivo de projeto e, a partir daí, usar `dotnet build` e `dotnet test` para validar o código em qualquer ambiente com o SDK adequado instalado, empregando, ou não, o Visual Studio 2026.

Em síntese, o `dotnet CLI` pode ser visto como o núcleo textual do desenvolvimento com .NET 10 e C# 14: é ele que concretiza, em comandos simples, as operações essenciais de criação, restauração, compilação, teste e publicação de projetos, servindo tanto de base técnica para ferramentas gráficas como o Visual Studio 2026 quanto de interface direta para scripts e terminais em ambientes diversos. A clareza do modelo de comandos, a integração com o SDK e o suporte a modelos extensíveis ajudam a explicar por que a CLI se tornou o caminho recomendado para padronizar o ciclo de vida de aplicações .NET em equipes que trabalham com IDEs diferentes e infraestruturas variadas.

Para testar o código-fonte anterior com o `dotnet CLI`, podemos criar um projeto de console, colar o código no arquivo de entrada e usar os comandos `dotnet` para compilar e executar.

O primeiro passo é garantir que o SDK do .NET esteja instalado. No terminal (Prompt de Comando, PowerShell ou bash), basta digitar `dotnet --info` ou `dotnet --version`. Se o comando for reconhecido e mostrar informações da instalação, o SDK já está disponível e, com ele, toda a CLI.

Em seguida, podemos escolher uma pasta de trabalho, criando um diretório para o projeto e entrando nele. Algo como `mkdir TesteCSharp14` e depois `cd TesteCSharp14`. Dentro dessa pasta, podemos inicializar um projeto de console com o comando `dotnet new console`. Esse comando usa o mecanismo de templates da CLI para gerar um projeto básico, com um arquivo `.csproj` e um `Program.cs` padrão, pronto para ser editado.

Depois de criado o projeto, é necessário abrir o arquivo `Program.cs` com o editor de sua preferência (pode ser o Visual Studio 2026, o Visual Studio Code ou outro editor de texto). Você verá um código de exemplo gerado pelo template. Substitua completamente o conteúdo desse arquivo pelo código apresentado anteriormente neste tópico. O modelo de console gerado pela CLI já é configurado

para compilar um executável, graças às configurações do arquivo de projeto que define o tipo de saída (`<OutputType>Exe</OutputType>`).

Com o código salvo em `Program.cs`, devemos retornar ao terminal, certificando-se de que o diretório atual seja o da pasta do projeto, isto é, aquele que contém o arquivo `.csproj`. A partir daí, o jeito mais direto de testar o programa é usar o comando `dotnet run`. Esse comando dispara, em uma única etapa, a restauração de pacotes (se necessário), a compilação e a execução da aplicação.

Se o código estiver correto e o compilador aceitar todos os recursos usados, o terminal exibirá, em sequência, as mensagens de saída do seu programa: saudações, manipulação de apelido, resultado da expressão `switch`, o valor calculado pelo método `Calcular`, a representação do `record Produto` e o valor do `Total do Cliente`.

Um arquivo `.csproj` moderno em **estilo SDK** (SDK-style) é um documento XML de MSBuild que descreve como o projeto será compilado, empacotado e publicado.

Fala-se em SDK-style quando o nó raiz do arquivo é algo como `<Project Sdk="Microsoft.NET.Sdk">`, como vimos anteriormente, ou variações como `Microsoft.NET.Sdk.Web`, `Microsoft.NET.Sdk.Worker` ou `Microsoft.NET.Sdk.WindowsDesktop`. O valor do atributo `Sdk` aponta para um conjunto de **targets** e **tasks** de MSBuild que fornecem as regras de compilação, publicação, empacotamento NuGet, padrões de arquivos incluídos, entre outras funcionalidades.

Um `.csproj` SDK-style típico para um aplicativo de console em .NET 10, com C# 14 e uso das convenções mais recentes, pode ser bem enxuto. Bastam algumas propriedades: `OutputType` para definir se o projeto gera um executável (`Exe` ou `WinExe`) ou uma biblioteca (`Library`), `TargetFramework` com o TFM correspondente, como `net10.0`, `ImplicitUsings` para habilitar ou não `implicit global usings`, `Nullable` para configurar referências anuláveis e, quando desejado, `LangVersion` para fixar explicitamente a versão do compilador, como `14.0`. Na maioria dos cenários, o SDK escolhe automaticamente a versão adequada de C# a partir do TFM e, portanto, `LangVersion` só é necessário quando se quer limitar ou padronizar a versão usada.

As propriedades aparecem em um ou mais blocos `<PropertyGroup>`, que são conjuntos de propriedades MSBuild. Nesse grupo, podem entrar, além das citadas, metadados como `AssemblyName`, `RootNamespace`, informações de pacote (`PackageId`, `Version`, `Authors`) e configurações específicas de build ou de ambiente. Essas propriedades são consumidas tanto pelo compilador quanto pelos targets do SDK, servindo de base para o que o `dotnet build`, o `dotnet pack` e o `dotnet publish` farão.

Outro bloco fundamental é o `<ItemGroup>`, que reúne itens sobre os quais o MSBuild executa ações. No estilo SDK, quase nunca é preciso listar os arquivos `.cs` individualmente, porque o SDK já define padrões (globs) que consideram todos os arquivos `**/*.cs` como itens `Compile`.



Lembrete

O uso mais comum de `<ItemGroup>` é para declarar dependências de pacote, com elementos `<PackageReference>` que indicam o nome do pacote e a versão, e dependências de outros projetos da solução, com `<ProjectReference>` apontando para outro `.csproj`. Quando se quer alterar o conjunto padrão de arquivos incluídos, usam-se itens com `Remove` para excluir arquivos ou diretórios específicos desses padrões, em vez de fazer a inclusão manual um por um.

O SDK-style também traz comportamentos implícitos em relação a metadados do assembly. Em vez de manter manualmente um `AssemblyInfo.cs` com atributos como `AssemblyTitle`, `AssemblyCompany`, `AssemblyVersion` e outros, o projeto pode ser configurado para que o SDK gere esses atributos automaticamente a partir das propriedades definidas no `.csproj`. Isso é controlado por uma propriedade como `GenerateAssemblyInfo`, que, quando verdadeira, indica que os atributos serão emitidos durante o processo de build. Se for necessário voltar ao controle manual, basta desativar essa geração e manter um arquivo `AssemblyInfo` tradicional no código.



Observação

Quando você programa em C#, escreve arquivos `.cs` (código-fonte). Depois que o projeto é compilado, o resultado vira um arquivo pronto para uso, normalmente um `.dll` ou um `.exe`. Esse arquivo compilado, com o código já traduzido para a linguagem que o .NET entende, é o que se chama de assembly.

Ou seja, assembly é o produto final da compilação: é o pacote que contém o seu código compilado junto a várias informações sobre ele. Essas informações extras são os metadados do assembly: nome do produto, empresa, versão, descrição e assim por diante. É como se o assembly fosse um livro, e os metadados fossem a capa e a página de rosto, em que aparecem título, autor, editora, ano de publicação etc.

Em soluções maiores, é comum haver muitas configurações repetidas entre projetos diferentes. O ecossistema de MSBuild oferece uma forma de consolidar essas definições através de arquivos `Directory.Build.props` e `Directory.Build.targets`. Colocados em um diretório acima dos projetos, eles são importados implicitamente por todos os `.csproj` localizados em subdiretórios, permitindo que propriedades e targets comuns sejam definidos em um único lugar. Isso combina bem com o estilo SDK, que já tende a evitar redundância dentro de cada arquivo de projeto.

2 POO: PILARES E ENCAPSULAMENTO NA PRÁTICA

A programação orientada a objetos (POO) é um paradigma de desenvolvimento que modela o software como um conjunto de objetos, cada um combinando **estado** (atributos) e **comportamento** (métodos).

A literatura clássica de engenharia de software descreve esse paradigma como uma estratégia para lidar com a complexidade de sistemas de grande porte por meio de um modelo de objetos que explora sistematicamente conceitos como abstração, encapsulamento, modularidade e hierarquia.

A linguagem C# é definida como uma linguagem simples, moderna, orientada a objetos e segura quanto a tipos, inserida em um ambiente de execução (CLR) projetado para suportar diretamente essas construções. Assim, em projetos escritos em C# 14 e desenvolvidos em IDEs contemporâneas, como o Visual Studio, continua-se a trabalhar sobre o mesmo núcleo conceitual que a literatura descreve desde os anos 1990: objetos, classes e os mecanismos que estruturam suas relações.

Uma **classe** é uma descrição abstrata de um conjunto de coisas que têm características e comportamentos em comum. É como um modelo: nela o programador especifica quais **dados** existirão (os **atributos** ou **campos**) e quais **ações** poderão ser executadas (os **métodos**). A classe, por si só, funciona como um tipo de dado definido pelo programador, um molde lógico que organiza informações e operações relacionadas em um único bloco coerente.

Entre esses métodos, em linguagens orientadas a objetos como C#, existe uma categoria especial chamada **método construtor**. O construtor é executado automaticamente quando um novo objeto da classe é criado e é usado para inicializar os atributos e estabelecer o estado inicial necessário para que esse objeto seja utilizado de maneira consistente. Em C#, o construtor tem o mesmo nome da classe, não declara tipo de retorno e pode receber parâmetros, permitindo que o código que cria o objeto forneça, já na construção, os valores ou as dependências indispensáveis ao seu uso.

Um **objeto**, por sua vez, é uma realização concreta de uma classe durante a execução do programa. Quando o programa é executado e o código solicita a criação de algo com base em uma classe, o que surge na memória é um objeto. Esse objeto carrega valores específicos para os atributos definidos na classe e é capaz de executar os métodos que a classe descreve.

Se a classe diz que Carro tem cor, modelo e ano, e que pode acelerar ou frear, o objeto é um carro específico, com uma cor concreta, um modelo e um ano determinados, que pode efetivamente acelerar e frear quando o programa chama os métodos correspondentes.

```
class Carro
{
    public string Cor { get; set; }
    public string Modelo { get; set; }
    public int Ano { get; set; }

    // Método construtor
    public Carro(string cor, string modelo, int ano)
```



```
{  
    Cor = cor;  
    Modelo = modelo;  
    Ano = ano;  
}  
  
public void Acelerar() { /* ... */ }  
public void Frear() { /* ... */ }  
}
```

Nesse código, **Carro** é a classe: ela define quais dados cada carro terá (**Cor**, **Modelo**, **Ano**) e quais operações podem ser feitas (**Acelerar**, **Frear**). O trecho **public Carro(string cor, string modelo, int ano)** é o método construtor. Ele tem o mesmo nome da classe, não declara tipo de retorno e é executado automaticamente quando um novo objeto do tipo **Carro** é criado, inicializando os atributos com valores fornecidos pelo código cliente. Quando o programa contém uma linha como:

```
var carroDoAluno = new Carro("Azul", "Hatch", 2022);
```

o que surge na memória é um objeto concreto, **carroDoAluno**, isto é, uma instância da classe **Carro**. Esse objeto possui valores específicos para os atributos definidos na classe (cor **Azul**, modelo **Hatch**, ano **2022**) e passa a responder aos métodos declarados, de modo que chamadas como **carroDoAluno.Acelerar();** e **carroDoAluno.Frear();** utilizam o comportamento descrito na definição de **Carro**.

Esse símbolo de ponto final (por exemplo, no meio de **carroDoAluno.Acelerar()**) é conhecido como **dot operator** e é o operador de acesso a membro. A documentação da linguagem o descreve como o token usado para acessar um membro de um namespace ou de um tipo, isto é, para navegar de um nome maior para um elemento mais específico: um namespace interno, um tipo dentro de um namespace ou um membro (campo, propriedade, método) de um tipo.

Quando você escreve algo como **System.Console.WriteLine("olá")**, o ponto está sendo empregado em cadeia. Primeiro, **System.Console** é um nome qualificado formado com **.** para indicar que a classe **Console** está dentro do namespace **System**. Depois, em **Console.WriteLine**, o ponto é usado de novo, agora para acessar o método **WriteLine** definido nessa classe. A própria documentação de operadores de acesso explica explicitamente esse uso: utilizar **.** para formar nomes qualificados de tipos em um namespace e para acessar membros de tipos, tanto estáticos quanto de instância.

A mesma regra vale dentro do código que você escreve. Se uma classe possui uma propriedade **Nome** e um método **Salvar**, expressões como **cliente.Nome** e **cliente.Salvar()** usam o operador **.** para acessar membros de instância do objeto referenciado pela variável **cliente**.

Já em **Math.PI** ou **Console.WriteLine**, o ponto acessa membros estáticos (**PI** e **WriteLine**) das classes **Math** e **Console**. Essa distinção entre membros estáticos e não estáticos aparece nos exemplos oficiais de **List<double> constants = [Math.PI, Math.E];**

`Console.WriteLine(constants.Count);`, em que `Math.PI` e `Math.E` são acessos estáticos e `constants.Count` é acesso a propriedade de instância.

O mesmo operador também é utilizado para alcançar namespaces aninhados. Em `using System.Collections.Generic;`, o ponto encadeia `System`, `Collections` e `Generic`, todos namespaces aninhados na BCL. Isso está de acordo com a descrição da Microsoft: usar `.` para acessar um namespace aninhado e para formar nomes qualificados de tipos dentro de um namespace.

Do ponto de vista da especificação, expressões do tipo `e.M` são chamadas de expressões de acesso a membro e fazem parte do conjunto de operações que o compilador precisa vincular a um método, propriedade ou campo concreto durante a resolução de sobrecargas.

A seção de expressões da linguagem lista explicitamente `e.M` como o formato de acesso a membro, ao lado de chamadas como `e.M(e1, ..., ev)` para invocação de método. Isso vale tanto para tipos estáticos quanto para instâncias; o operador em si é o mesmo, o que muda é o tipo à esquerda do ponto.

De forma sintética, o dot operator em C# é o operador que conecta nomes e valores em cadeias de acesso: ele entra em namespaces, alcança tipos e, a partir deles ou de instâncias, alcança campos, propriedades e métodos, inclusive métodos de extensão. A partir do momento em que a expressão à esquerda não é nula e o membro à direita existe e é acessível, `.` é o caminho padrão que o compilador usa para construir a expressão de acesso a membro descrita na especificação e na documentação oficial da linguagem.

Em POO, o desenvolvimento de software gira em torno da definição de boas classes e da criação e da manipulação dos objetos derivados delas, pois são esses objetos que interagem entre si para realizar as tarefas desejadas pelo programa.

Uma convenção didática amplamente adotada em livros técnicos é descrever a POO a partir de quatro mecanismos centrais – encapsulamento, abstração, herança e polimorfismo –, frequentemente chamados de "os quatro pilares" da orientação a objetos. Manuais de POO com C# explicitam que a modelagem orientada a objetos se baseia nessas noções, usando-as como fio condutor tanto para a análise quanto para o desenho de sistemas. Esses quatro mecanismos não são independentes: em conjunto, definem como uma aplicação em C# organiza seus tipos, limita o acesso a detalhes internos, reutiliza comportamentos entre classes e permite que diferentes objetos respondam de maneiras distintas a uma mesma operação.

O primeiro pilar, o **encapsulamento**, diz respeito ao empacotamento de dados e operações que atuam sobre esses dados em uma unidade lógica, normalmente uma classe, isolando a representação interna e expondo apenas uma interface bem definida.

Textos clássicos sobre POO com C++ e C# descrevem encapsulamento como a reunião de atributos e funções em uma mesma unidade, associada ao princípio de data hiding: o código externo não acessa diretamente o estado interno, mas interage por meio de métodos ou propriedades, o que permite controlar invariantes e restringir modificações indevidas.

Em C#, isso se concretiza em decisões como declarar campos privados, expor propriedades com `get / set` controlados, usar modificadores de acesso (`public`, `protected`, `internal`, `private`) e definir métodos que representam operações conceitualmente relevantes sobre o objeto. Em um projeto moderno em C#, seja qual for a versão da linguagem, o encapsulamento continua sendo o mecanismo básico para garantir que mudanças na implementação interna de uma classe não obriguem a reescrita do código cliente, preservando a estabilidade das interfaces públicas – exatamente o tipo de benefício atribuído à orientação a objetos nos manuais acadêmicos.

A **abstração**, segundo a mesma literatura, é o processo de selecionar os aspectos essenciais de uma entidade e ignorar detalhes contingentes, de modo que um tipo represente um conceito coerente do domínio de problema. Tanto Booch *et al.* (2007) quanto autores que tratam da essência da POO com linguagens modernas ressaltam que abstração consiste em definir modelos que capturam propriedades conceitualmente relevantes, sem expor os mecanismos internos necessários para realizá-las.

Em C#, classes e interfaces são os principais instrumentos de abstração: uma interface pode capturar um conjunto de operações que caracterizam, por exemplo, um repositório ou um serviço de notificação, sem impor como essas operações serão implementadas; classes concretas então realizam essa abstração em termos de campos, algoritmos específicos e recursos da plataforma.

Do ponto de vista metodológico, a abstração orienta as etapas iniciais de análise e projeto: livros de POO com C# mostram que o desenvolvedor começa identificando as abstrações relevantes do domínio, mapeando-as para tipos, antes de tratar de detalhes de banco de dados, interface gráfica ou desempenho.

A **herança**, terceiro pilar, é descrita nas referências como o mecanismo pelo qual uma classe (subclasse ou classe derivada) reaproveita e especializa a estrutura e o comportamento de outra (superclasse ou classe base). No material específico de C#, enfatiza-se que todas as classes derivam, em última instância, de `System.Object`, e que a linguagem oferece herança simples de classes combinada com implementação múltipla de interfaces, permitindo construir hierarquias em que tipos mais gerais capturam propriedades comuns, enquanto tipos mais específicos acrescentam comportamentos ou restringem usos.

Em termos de projeto, a herança é usada em C# para modelar relações é-um (por exemplo, uma classe `ContaPoupanca` derivada de `ContaBancaria`) e para definir classes base abstratas que especificam operações obrigatórias para subclasses, favorecendo a reutilização de código e a definição de famílias de tipos com contratos compartilhados.

A literatura também alerta que a herança deve ser aplicada com critério, pois hierarquias mal projetadas podem introduzir acoplamento excessivo; por isso, livros recentes combinam o estudo de herança em C# com princípios de projeto como **SOLID**, que influenciam a estrutura de bibliotecas em versões atuais da linguagem.

O quarto pilar, o **polimorfismo**, é a capacidade de diferentes objetos responderem à mesma mensagem de formas específicas, preservando uma interface comum. Em textos didáticos sobre C# e POO, o polimorfismo é descrito como a situação em que duas ou mais classes oferecem o mesmo conjunto

de operações, mas com implementações distintas; o código cliente invoca a operação por meio de uma referência do tipo mais geral, e o sistema de despacho dinâmico escolhe, em tempo de execução, a implementação apropriada ao objeto concreto. Em C#, isso aparece de forma típica na combinação de herança com métodos `virtual` e `override`, ou na programação voltada a interfaces, em que componentes manipulam instâncias por meio de interfaces em vez de classes específicas.

Autores que abordam POO com C# insistem que esse mecanismo é fundamental para escrever código extensível: novas subclasses ou novas implementações de interfaces podem ser introduzidas sem alterar o código que depende apenas dos contratos abstratos.

Tomados em conjunto, encapsulamento, abstração, herança e polimorfismo fornecem o arcabouço conceitual com que a literatura especializada descreve a POO em C#, e esse arcabouço continua válido nas versões mais recentes da linguagem e nas ferramentas modernas de desenvolvimento.

O encapsulamento e a abstração determinam como as classes e as interfaces em C# 14 expressam modelos de domínio claros; a herança e o polimorfismo estruturam hierarquias de tipos e viabilizam arquiteturas baseadas em contratos que podem ser estendidos sem ruptura.

Em um projeto desenvolvido em C# com suporte de IDEs atuais, aplicar esses quatro mecanismos de acordo com as recomendações continua sendo o caminho principal para obter sistemas mais compreensíveis, reutilizáveis e fáceis de evoluir, exatamente o objetivo que motivou o surgimento da orientação a objetos na engenharia de software.

2.1 Encapsulamento, abstração, herança e polimorfismo: visão geral

2.1.1 Encapsulamento

O encapsulamento é um dos pilares fundamentais da POO e desempenha um papel crucial na construção de software modular e seguro. Em essência, **encapsular** significa agrupar dados e métodos relacionados em uma única unidade (a classe), ocultando os detalhes internos e expondo apenas o necessário para o uso externo. Dessa forma, uma classe funciona como uma cápsula que agrega dados (atributos) e comportamentos (métodos) em um objeto, estabelecendo uma nítida separação entre a interface pública e a implementação privada dessa classe. Em outras palavras, a classe define quais operações estão disponíveis para o mundo exterior (sua interface pública) enquanto protege seus dados internos e como essas operações são realizadas (sua implementação).

Segundo Booch *et al.* (2007, p. 52, tradução nossa), o encapsulamento pode ser definido formalmente como "o processo de compartimentalizar os elementos de uma abstração que constituem sua estrutura e comportamento; esse processo serve para separar a interface contratual de uma abstração de sua implementação". Essa definição ressalta que cada objeto apresenta aos demais apenas uma visão externa (a interface contratual), mantendo escondidos os detalhes de sua estrutura interna e dos procedimentos que executa. Tudo o que não contribui para as características essenciais do objeto pode – e deve – ser ocultado de outros objetos.

Assim, somente as partes relevantes para o uso da classe são expostas, enquanto detalhes internos ficam inacessíveis, evitando dependências indevidas. De fato, boas práticas de engenharia de software pregam que nenhuma parte de um sistema complexo deve depender dos detalhes internos de qualquer outra parte.

O encapsulamento está intimamente relacionado ao princípio do ocultamento de informações (information hiding). Ao encapsular, escondemos os dados e a lógica de implementação por trás de barreiras bem definidas (geralmente limites de classe ou módulo), expondo apenas o que for necessário por meio de métodos ou propriedades públicas. Isso significa que atributos de um objeto geralmente ficam marcados como privados, inacessíveis diretamente de fora, e somente podem ser consultados ou modificados por meio de operações definidas na interface pública desse objeto.

Ao fazer isso, restrições e validações podem ser aplicadas internamente, garantindo integridade dos dados. Por exemplo, podemos ter certeza de que uma classe `ContaBancaria` nunca permitirá um saldo negativo se o campo interno que guarda o saldo for privado e as operações de depósito/saque implementarem verificações adequadas, em vez de expor diretamente esse campo para livre modificação externa.

A vantagem imediata dessa abordagem é clara: ao proteger o estado interno de um objeto, prevenimos usos incorretos e erros difíceis de rastrear. Variáveis e métodos não destinados ao uso externo podem (e devem) ser escondidos para limitar o potencial de erros de programação.

Por exemplo, suponha uma classe `Pessoa` com um atributo público `idade`. Qualquer código externo poderia livremente atribuir um valor inválido a `idade` (como um número negativo ou muito acima do plausível), deixando o objeto em um estado inconsistente. Já com o encapsulamento, podemos tornar `idade` um campo privado e fornecer um método ou uma propriedade pública para alterá-lo, o qual rejeitaria ou ajustaria valores inválidos.

Assim, o objeto controla seus invariantes internos, assegurando que qualquer modificação em seu estado respeite regras predefinidas. Esse controle centralizado dificulta que partes externas coloquem o objeto em um estado inválido – em vez disso, devem interagir através da interface segura que a classe expõe.

Outro benefício importante é que encapsular reduz o **acoplamento** entre diferentes partes do programa. Objetos interagem apenas com as interfaces de outros objetos, sem precisar conhecer (nem depender de) detalhes internos uns dos outros. Isso leva a uma arquitetura mais modular, em que mudanças internas em uma classe não afetam outras classes desde que a interface pública permaneça a mesma.

Como observado na literatura, enquanto a abstração concentra-se em ajudar as pessoas a pensar sobre o que estão fazendo, o encapsulamento permite que mudanças no programa sejam realizadas de forma confiável com esforço limitado. Em termos práticos, isso significa que podemos alterar a implementação interna de uma classe (por exemplo, trocar a forma como calculamos algo ou a estrutura de dados usada para armazenar informações) sem quebrar o código externo que depende dessa classe – desde que respeitemos o contrato definido por sua interface.

Essa capacidade de evolução e manutenção do software é vital em projetos de longo prazo, pois reduz drasticamente o risco de efeitos colaterais indesejados ao melhorar ou corrigir partes internas do sistema. Para entender melhor, vejamos um exemplo prático em C#. Suponha uma classe simplificada que representa uma conta bancária:

```
public class ContaBancaria
{
    public decimal Saldo;
}
```

Nesse código, `Saldo` é um campo público. Isso fere o encapsulamento, pois qualquer código externo pode alterar livremente esse valor. Por exemplo, em algum outro ponto do programa, poderíamos ter:

```
ContaBancaria conta = new ContaBancaria();
conta.Saldo = 100m;           // define saldo inicial
conta.Saldo -= 150m;          // retira 150, resultando em saldo negativo!
Console.WriteLine(conta.Saldo); // exibe "-50", um estado inconsistente
```

Com `Saldo` exposto publicamente, não há nada que impeça esse código de colocar a conta em um estado inválido (saldo negativo). Esse design é propenso a erros e torna difícil garantir invariantes lógicos. Agora, vejamos uma versão encapsulada da mesma classe:

```
public class ContaBancaria
{
    private decimal saldo; // campo privado

    public decimal Saldo // propriedade pública somente leitura
    {
        get { return saldo; }
        private set { saldo = value; }
    }

    public ContaBancaria(decimal saldoInicial = 0m)
    {
        saldo = saldoInicial;
    }

    public bool Depositar(decimal quantia)
    {
        if (quantia <= 0)
            return false;
        saldo += quantia;
        return true;
    }

    public bool Sacar(decimal quantia)
    {
        if (quantia <= 0 || quantia > saldo)
            return false;
        saldo -= quantia;
        return true;
    }
}
```

Nessa versão, o campo interno `saldo` é privado, e a classe fornece uma propriedade pública somente leitura `Saldo` (com `get` público e `set` privado) para permitir consulta externa do saldo, mas não modificação arbitrária. As operações de depósito e saque são expostas via métodos públicos que contêm a lógica de validação: não se pode depositar quantias não positivas, e não se pode sacar mais do que o saldo disponível.

Note como agora é impossível para um código externo violar as regras de negócio ou corromper o estado da conta – toda interação com o saldo passa pelos métodos `Depositar` e `Sacar`, que garantem a integridade. A tentativa de compilar um código como `conta.Saldo -= 150m;` resultaria em erro, pois `Saldo` não possui um setter acessível externamente.

Desse modo, o encapsulamento assegura que um objeto controle rigorosamente como seus dados podem ser usados ou alterados. Essa característica previne erros e facilita mudanças futuras. Se decidirmos, por exemplo, alterar a implementação interna de `ContaBancaria` para incluir um registro de transações ou aplicar taxas, podemos fazê-lo dentro desses métodos sem impactar nenhum código cliente – desde que a interface (assinaturas de `Depositar` e `Sacar` e propriedade `Saldo`) permaneça igual.

Os outros objetos do sistema não sabem e não precisam saber como `ContaBancaria` gerencia seu saldo internamente; eles apenas confiam que depositar uma quantia positiva aumenta o saldo e sacar uma quantia válida o reduz, conforme a interface promete.

É importante destacar que o encapsulamento não implica necessariamente ocultar todos os detalhes, mas sim aqueles que não são pertinentes para o uso externo. Em C#, a visibilidade de membros é controlada por modificadores de acesso como `private` (privado), `public` (público), `protected` (protegido), `internal` (interno) etc. O encapsulamento se concretiza, em grande parte, através do uso adequado desses modificadores.

Campos privados são invisíveis fora da classe; métodos e propriedades públicos formam a interface acessível aos demais; membros `protected` ficam visíveis apenas a subclasses (permitindo algum compartilhamento controlado de implementação dentro de hierarquias de herança); e membros `internal` restringem o acesso ao mesmo assembly (módulo) – outra forma de encapsular componentes em nível de projeto. Esse mecanismo linguístico garante barreiras explícitas entre diferentes partes do código. Assim, desenvolvedores podem (e devem) delimitar claramente quais partes de uma classe são detalhes internos sujeitos a mudança livre e quais partes constituem o contrato estável com o mundo exterior.

Cabe observar que encapsulamento e abstração são conceitos complementares que andam de mãos dadas. A abstração lida com o que um objeto representa e o que ele oferece – foca o comportamento observável do objeto, omitindo complexidades desnecessárias. Já o encapsulamento lida com como esse comportamento é implementado – oculta a complexidade interna que produz o comportamento.

Uma boa abstração define uma interface clara e simples, enquanto o encapsulamento garante que a implementação por trás dessa interface permaneça escondida. Por exemplo, ao usar um objeto de

uma classe `Mapa`, interessa-nos que ele forneça operações como `BuscarCaminho(cidadeA, cidadeB)` (abstração da funcionalidade); não precisamos saber qual algoritmo exato de roteamento ele usa internamente, nem em quais estruturas de dados armazena as cidades e as rotas – esses detalhes ficam encapsulados dentro de `Mapa`. Como bem sintetizado por uma observação clássica, para que a abstração funcione, as implementações devem estar encapsuladas.

Separando interface de implementação, cada classe pode ser entendida e usada através de sua interface pública (abstração), sem exigir conhecimento de seus segredos internos. Isso reduz a complexidade cognitiva ao desenvolver sistemas grandes, pois os programadores podem raciocinar sobre cada parte em alto nível, confiando que os detalhes estão controlados localmente dentro de cada classe.

Além de proteger dados e facilitar manutenção, o encapsulamento também contribui para outras propriedades desejáveis do design de software, como a **modularidade** e a **manutenibilidade**. Módulos bem encapsulados têm limites claros, interagindo com outros módulos apenas por interfaces bem definidas, o que permite dividir um sistema complexo em partes menores e mais gerenciáveis.

Cada módulo (ou classe) torna-se responsável por um conjunto específico de funcionalidades, e detalhes de implementação de um módulo não afetam os outros. Isso também apoia o trabalho em equipe no desenvolvimento: diferentes desenvolvedores podem trabalhar em módulos distintos sem o risco de interferir nos detalhes internos uns dos outros, desde que concordem nos contratos das interfaces.

Vale mencionar que, apesar de suas vantagens, o encapsulamento não é absoluto – as linguagens geralmente fornecem meios controlados de quebrar o encapsulamento em casos especiais. Por exemplo, em C# existem classes aninhadas (nested classes) que podem acessar membros privados da classe englobante, e modificadores como `protected` que deliberadamente expõem alguns detalhes para subclasses.

Esses recursos existem para permitir implementações eficientes ou flexíveis quando necessário, mas devem ser usados com parcimônia. A recomendação geral é manter os dados realmente privados, a menos que haja um motivo claro para expor algum nível de acesso adicional.

Até mesmo Bjarne Stroustrup, criador da linguagem C++, destacou que os mecanismos de ocultação de informação das linguagens servem para a prevenção de acidentes, não para impedir fraudes. Isto é, o encapsulamento ajuda a evitar usos equivocados não intencionais, mas não é uma barreira de segurança intransponível – um programador malicioso ou descuidado pode contornar proteções (via reflexão, por exemplo). Ainda assim, em cenários normais, respeitar o encapsulamento torna a colaboração entre componentes mais segura e confiável, uma vez que limita as maneiras pelas quais um objeto pode interferir em outro.

Em suma, o encapsulamento pode ser visualizado como um escudo protetor em torno do objeto, garantindo que interações externas se deem apenas através de portas bem definidas (métodos/propriedades públicos). Essa abordagem promove um desenvolvimento mais seguro, pois erros permanecem localizados (um objeto não corrompe acidentalmente o estado interno de outro), e

oferece um projeto mais flexível, já que cada módulo do sistema pode evoluir internamente sem quebrar contratos com outros módulos.

Compreender o encapsulamento é dar um passo importante rumo a escrever código de melhor qualidade: classes encapsuladas tendem a ser mais fáceis de usar, pois expõem interfaces claras, e a ser mais fáceis de depurar e manter, tendo em vista que isolam complexidades e possibilitam alterações internas com impacto mínimo no restante do código.

2.1.2 Abstração

Agora vamos explorar em detalhes o conceito de abstração, um dos pilares centrais da POO.

De maneira geral, a abstração é o processo de destacar as características essenciais de um objeto, omitindo detalhes supérfluos ou irrelevantes para determinado contexto. Esse princípio permite lidar com a complexidade de sistemas de software de modo mais gerenciável, concentrando-se apenas no que é importante em cada nível de design. Booch *et al.* (2007) afirmam que a abstração é uma das formas fundamentais de **enfrentarmos a complexidade**, pois, ao abstrairmos um problema, conseguimos focar os aspectos cruciais e ignorar temporariamente os detalhes menos importantes. Em termos práticos, ao modelar um sistema orientado a objetos, utilizamos a abstração para criar representações simplificadas de entidades do mundo real ou de conceitos do domínio do problema, realçando apenas as características que importam para nossos objetivos.

Para compreender melhor, imagine a ideia de veículo. Quando pensamos em um veículo de forma abstrata, reconhecemos características básicas que todos os veículos compartilham (como o fato de transportar passageiros, possuir um meio de propulsão etc.) sem nos preocuparmos com detalhes específicos de um carro, uma motocicleta ou um caminhão. Essa visão simplificada é a abstração em ação: definimos o conceito genérico de veículo enfatizando atributos e comportamentos essenciais, deixando de fora particularidades de cada tipo específico.

Do mesmo modo, em POO podemos criar uma **classe abstrata** `Veiculo` que captura essa essência genérica, enquanto **classes concretas** como `Carro` ou `Motocicleta` estendem `Veiculo` e implementam detalhes particulares (como o número de rodas e o tipo de combustível, entre outros). Nesse exemplo, `Veiculo` é uma abstração que reúne as características comuns, servindo como modelo para as especializações.

Do ponto de vista formal, a abstração em POO tem dois significados complementares. O primeiro é conceitual: trata-se de modelar entidades do mundo real ou conceitos de forma simplificada, incluindo apenas atributos e operações pertinentes ao contexto da aplicação. O segundo significado é técnico: refere-se aos mecanismos da linguagem que nos permitem esconder detalhes de implementação e enfatizar comportamentos gerais.

Em C# e em outras linguagens orientadas a objetos modernas, a abstração se manifesta principalmente através de classes abstratas e interfaces. Esses recursos da linguagem possibilitam definir **contratos** ou

modelos genéricos de funcionamento, que depois são concretizados por **classes derivadas** ou classes que **implementam essas interfaces**.

De acordo com Booch *et al.* (2007, p. 44, tradução nossa), a "abstração é definida como as características essenciais de um objeto que o distinguem de todos os outros tipos de objeto, provendo limites conceituais bem definidos do ponto de vista do observador". Em outras palavras, a abstração é o ato de se concentrar nos aspectos fundamentais de uma entidade e ignorar quaisquer detalhes que não sejam relevantes naquele contexto.



Lembrete

Esse conceito anda de mãos dadas com o encapsulamento e a ocultação de informações que vimos anteriormente, pois, ao definir uma interface abstrata para um objeto (seja via classe abstrata ou interface da linguagem), estamos separando **o que** o objeto faz do **como** ele faz.

Assim, os usuários dessa abstração (isto é, outras partes do programa que interagem com o objeto) precisam se preocupar apenas com a interface pública exposta – os métodos e as propriedades essenciais –, sem precisar conhecer os pormenores da implementação interna. Essa dinâmica torna o software mais modular e flexível: mudanças internas em uma classe concreta não afetam as partes do código que dependem apenas da abstração exposta, desde que a interface permaneça a mesma.

Em C#, definimos uma classe abstrata usando a palavra-chave `abstract`. Uma classe abstrata é aquela que não pode ser instanciada diretamente; ela existe principalmente para ser herdada por outras classes. Normalmente, uma classe abstrata representa um conceito genérico ou uma generalização, e suas subclasses representam implementações específicas desse conceito.

A classe abstrata pode incluir membros abstratos – métodos ou propriedades declarados sem implementação – e também membros concretos (implementados normalmente). **Membros abstratos** funcionam como lacunas a serem preenchidas pelas subclasses: ao herdar de uma classe abstrata, a classe derivada é obrigada a fornecer implementação para todos os métodos e propriedades abstratos definidos na base. Dessa forma, garantimos que cada subclasse concreta tenha seu próprio comportamento específico para aquelas operações, ao mesmo tempo que todas elas compartilham a mesma interface e, possivelmente, algum comportamento padrão fornecido pela classe abstrata.

Vamos considerar um exemplo prático para ilustrar. Suponha que estamos desenvolvendo um simulador simples de animais e desejamos modelar o comportamento de emitir som de forma genérica. Poderíamos ter uma classe abstrata `Animal` com um método abstrato `EmitirSom()`. Cada tipo específico de animal vai implementar esse método de maneira diferente. O código a seguir demonstra essa ideia em C#:

```
// Classe abstrata representando um Animal genérico
public abstract class Animal
{
    public string Nome { get; set; }

    // Construtor da classe abstrata
    public Animal(string nome)
    {
        Nome = nome;
    }

    // Método abstrato: cada subclasse deve implementar o seu próprio som
    public abstract void EmitirSom();
}

// Subclasse concreta que estende Animal
public class Cachorro : Animal
{
    public Cachorro(string nome) : base(nome) { }
    // Implementação específica do som de um cachorro
    public override void EmitirSom()
    {
        Console.WriteLine($"{Nome}: Au Au!");
    }
}

// Outra subclasse concreta de Animal
public class Gato : Animal
{
    public Gato(string nome) : base(nome) { }

    // Implementação específica do som de um gato
    public override void EmitirSom()
    {
        Console.WriteLine($"{Nome}: Miau!");
    }
}

// Programa principal para testar as classes anteriores
public class Program
{
    public static void Main()
    {
        Animal meuAnimal1 = new Cachorro("Rex");
        Animal meuAnimal2 = new Gato("Felix");

        // Mesmo sem saber exatamente o tipo, podemos chamar EmitirSom graças à
        abstração
        meuAnimal1.EmitirSom(); // Saída: "Rex: Au Au!"
        meuAnimal2.EmitirSom(); // Saída: "Felix: Miau!"
    }
}
```

Nesse exemplo, definimos uma classe abstrata `Animal` que contém um membro abstrato `EmitirSom()`. Essa operação representa a ação genérica de emitir um som, mas não há implementação na classe `Animal`, pois faria pouco sentido definir como um animal genérico emite som – já que cada espécie tem seu próprio som característico.

Em vez disso, deixamos a implementação em aberto (abstrata) e marcamos o método com `abstract`. As classes `Cachorro` e `Gato` herdam de `Animal` e utilizam `override` para fornecer implementações específicas de `EmitirSom()`, adequadas a cada animal (latido para cachorro, miado para gato).

Note que, por serem subclasses concretas, elas podem ser instanciadas, ao contrário de `Animal`. No método `Main`, criamos instâncias de `Cachorro` e `Gato`, mas as referenciamos pelo tipo abstrato `Animal`. Graças a isso, podemos tratar ambos os objetos de forma uniforme – chamando `meuAnimal.EmitirSom()` sem nos preocupar com qual é o tipo específico do animal em tempo de execução. Esse é um poderoso efeito da abstração aliada ao polimorfismo (conceito que explicaremos em detalhes mais adiante): o código que usa o objeto invoca a operação abstrata definida na superclasse (`Animal`), e quem resolve qual implementação executar (latir ou miar) é o próprio objeto concreto em tempo de execução. Assim, a abstração nos permitiu escrever código genérico (que lida com `Animal`) enquanto os detalhes específicos ficaram encapsulados nas subclasses.

É importante destacar que uma classe abstrata em C# pode também fornecer implementações comuns para várias subclasses. Por exemplo, poderíamos adicionar um método concreto em `Animal` – digamos, `Dormir()` – que seria herdado por todos os animais sem precisar ser reescrito em cada subclasse, já que o ato de dormir poderia ser genérico para todos os animais.

A decisão do que vai ser abstrato (sem implementação) e o que vai ser concreto (implementado na própria classe base) faz parte do processo de abstração e modelagem: os comportamentos que variam de acordo com o tipo específico de objeto geralmente são declarados como abstratos, enquanto aqueles que são compartilhados igualmente por todas as variantes podem ser implementados diretamente na classe base abstrata para reutilização.

Além de classes abstratas, outra ferramenta poderosa para aplicar abstração em POO são as **interfaces**. Uma interface define um conjunto de membros (métodos, propriedades, indexadores, eventos) que representam uma capacidade ou um comportamento que pode ser implementado por qualquer classe (ou `struct`) que deseje aderir àquele contrato.

Considere que queremos abstrair o conceito de objetos geométricos que possuem área e perímetro. Poderíamos criar uma interface `IForma` (segundo a convenção de nomenclatura do .NET de prefixar com `I`) definindo dois métodos: `CalcularArea()` e `CalcularPerimetro()`. Qualquer classe que implementar `IForma` será obrigada a fornecer sua própria lógica para esses cálculos. Vejamos um exemplo:

```
// Interface representando uma forma geométrica
public interface IForma
{
    double CalcularArea();
    double CalcularPerimetro();
}

// Classe concreta Retangulo implementa a interface IForma
public class Retangulo : IForma
{
    public double Largura { get; set; }
    public double Altura { get; set; }

    public Retangulo(double largura, double altura)
    {
        Largura = largura;
        Altura = altura;
    }

    public double CalcularArea()
    {
        return Largura * Altura;
    }

    public double CalcularPerimetro()
    {
        return 2 * (Largura + Altura);
    }
}

// Classe concreta Circulo implementa a interface IForma
public class Circulo : IForma
{
    public double Raio { get; set; }

    public Circulo(double raio)
    {
        Raio = raio;
    }

    public double CalcularArea()
    {
        return Math.PI * Raio * Raio;
    }

    public double CalcularPerimetro()
    {
        return 2 * Math.PI * Raio;
    }
}

// Exemplo de uso das classes que implementam a interface
public class GeometriaApp
{
    public static void Main()
    {
    }
}
```

```
List<IForma> formas = new List<IForma>();
formas.Add(new Retangulo(3.0, 4.0));
formas.Add(new Circulo(5.0));

foreach (IForma forma in formas)
{
    Console.WriteLine($"Área: {forma.CalcularArea()}");
    Console.WriteLine($"Perímetro: {forma.CalcularPerimetro()}");
    Console.WriteLine("-----");
}
}
```

Nesse código, definimos a interface **IForma** com dois métodos abstratos: **CalcularArea()** e **CalcularPerimetro()**. Em seguida, implementamos essa interface em duas classes concretas: **Retangulo** e **Circulo**.

Cada uma dessas classes fornece a sua própria versão dos métodos, de acordo com as fórmulas geométricas pertinentes. Observe que **Retangulo** e **Circulo** não têm relação de herança entre si (uma não é subclasse da outra); contudo, ambas compartilham um mesmo contrato comum, expresso por **IForma**. Isso nos permite tratá-las de forma polimórfica através da interface.

No método **Main** de **GeometriaApp**, criamos uma lista do tipo `List<IForma>` e adicionamos instâncias de **Retangulo** e **Circulo** a essa lista. Graças à abstração fornecida pela interface, podemos iterar por essa coleção de **IForma** e chamar **CalcularArea()** e **CalcularPerimetro()** em cada objeto sem saber, naquele ponto do código, qual é a classe concreta de cada elemento – basta confiar que todos eles cumprem o contrato da interface e possuem essas operações implementadas.

Novamente, vemos aqui o princípio de programar voltado à interface, não à implementação: o código que consome os objetos **IForma** lida apenas com a interface abstrata, aproveitando a generalidade do conceito de forma geométrica, enquanto cada classe concreta encapsula seu comportamento específico.

As interfaces são extremamente úteis para modelar comportamentos comuns a classes que não compartilham uma relação direta de herança. Elas permitem a implementação do que chamamos de abstração de comportamento: diferentes objetos, possivelmente de hierarquias distintas, podem todos cumprir um determinado contrato.

Por exemplo, em uma aplicação real poderíamos ter interfaces, como **IArmazenavel** (definindo métodos como **Salvar()** e **Carregar()**), que diferentes classes – desde configurações do sistema até níveis de um jogo – implementariam para garantir que podem ser persistidas em disco, ou **IComparavel** (similar à interface genérica **IComparable** do .NET), para indicar que objetos podem ser comparados e ordenados.

Em todos esses casos, a interface estabelece um conjunto de operações abstratas que as classes devem oferecer, mas cada classe é livre para realizar essas operações da maneira mais adequada a si, ocultando seus detalhes internos.

Por fim, ao aplicar a abstração na POO, conseguimos construir sistemas mais modulares, extensíveis e de fácil manutenção. Ao nos concentrarmos nas características essenciais e em contratos gerais, podemos alterar a implementação interna das classes concretas sem que isso afete o resto do sistema – desde que a interface pública (abstrata) permaneça consistente.

Por exemplo, se decidíssemos otimizar o cálculo da área de um círculo usando uma aproximação mais eficiente de π , poderíamos mudar o código dentro de `Circulo.CalcularArea()` livremente, pois, do ponto de vista dos clientes da classe (que interagem via `IForma` ou como `Circulo` mesmo), nada muda em termos de interface: o método ainda se chama `CalcularArea()` e retorna um `double` representando a área. Essa separação clara entre interface e implementação é exatamente o benefício da abstração aliada ao encapsulamento.

Entretanto, é preciso usar o mecanismo de abstração com critério. Uma regra prática em design de software é não abstrair prematuramente. Em outras palavras, não faz sentido criar classes ou métodos abstratos para comportamentos que não se sabe ao certo se variarão ou se precisarão mesmo de especializações futuras.

Abstrações mal planejadas podem tornar o código mais complexo do que o necessário. Idealmente, identificam-se quais conceitos do domínio do problema têm variantes ou detalhes específicos e, a partir deles, extraem-se generalizações adequadas.

As abstrações devem fazer sentido no contexto do domínio e agregar valor em termos de reutilização ou organização do código. Quando bem empregadas, elas permitem evitar duplicação de código (métodos comuns podem residir na superclasse abstrata, evitando reimplementações repetidas em cada classe concreta) e facilitam a extensão do sistema.

Por exemplo, para adicionar um novo tipo de animal ou uma nova forma geométrica, basta criar uma subclasse concreta nova (ou uma nova classe implementando a interface apropriada), sem precisar modificar o código existente que trabalha com as abstrações já definidas.

Resumindo, o pilar da abstração em POO nos incentiva a pensar nas entidades de um programa em termos do que elas representam e do que podem fazer, antes de mergulhar em como elas fazem. Abstraindo corretamente, delineamos fronteiras conceituais claras para nossas classes e interfaces, o que conduz a um design mais limpo e compreensível.

Ao combinar a abstração com os outros pilares (encapsulamento para esconder dados internos, herança para organizar hierarquias de generalização/especialização, e polimorfismo para interagir com objetos de forma genérica), obtemos todo o poder da POO na construção de um software robusto, flexível e aderente ao mundo real.

2.1.3 Herança

A herança é o terceiro pilar fundamental da POO. Em termos gerais, é o mecanismo que permite definir uma nova classe a partir de uma classe existente, reaproveitando atributos e comportamentos já implementados na classe base.

A classe existente cuja definição é reutilizada é chamada de **classe base** (ou superclasse), enquanto a nova classe definida que herda essas características é chamada de **classe derivada** (ou subclasse). Dizemos que a subclasse estende a superclasse, incorporando seus membros (dados e métodos) e podendo adicionar novos membros ou modificar comportamentos, conforme necessário.

Esse conceito promove uma forte **reutilização de código**: em vez de reimplementar funcionalidades comuns em várias classes, é possível defini-las na classe base e reutilizá-las em todas as subclasses, reduzindo a duplicação de código e o risco de inconsistências.

Deitel e Deitel (2016) definem herança como uma forma de reutilização de software na qual novas classes adquirem os membros de classes existentes e as aprimoram com novas capacidades. Em outras palavras, a subclasse torna-se uma extensão especializada da superclasse, reutilizando tudo que for aplicável da classe base e acrescentando suas próprias particularidades quando preciso.

Por exemplo, considere uma hierarquia simples em que a classe `Veiculo` define atributos e métodos genéricos (como `cor`, `ano`, `acelerar()`, `frear()`), e uma classe derivada `Carro` herda de `Veiculo`. O `Carro` automaticamente possui todas as propriedades e comportamentos definidos em `Veiculo`, como `cor` e `acelerar()`, sem precisar reescrevê-los. Além disso, `Carro` pode introduzir características específicas (como um atributo `tipoCombustivel` ou um método `abrirPorta()`). Assim, `Carro` é um `Veiculo`, estabelecendo uma relação do tipo é-um (is-a) própria da herança.

A reutilização proporcionada pela herança traz diversos benefícios em termos de qualidade e manutenção do software. Ao evitar a repetição de código em várias classes, reduz-se a probabilidade de erros e inconsistências.

Quando um comportamento comum a várias classes precisa ser ajustado ou aprimorado, basta modificá-lo na superclasse – as alterações entrarão em vigor automaticamente em todas as subclasses que herdaram aquele comportamento. Isso simplifica a manutenção e a evolução do programa, pois correções de bugs ou adição de melhorias centralizadas na classe base beneficiam toda a hierarquia de classes derivadas.

Além disso, a herança favorece a organização conceitual do sistema em uma **hierarquia de generalização-especialização**. Classes genéricas e de alto nível de abstração ficam no topo da hierarquia, capturando características amplas de um conjunto de objetos, enquanto classes mais específicas são definidas como subclasses, especializando o comportamento dessas generalizações.

Similarmente ao exemplo que vimos há pouco, pode-se ter uma classe geral `FormaGeométrica` com atributos e métodos aplicáveis a qualquer forma (por exemplo, uma operação abstrata `calcularArea()`, ou propriedades comuns como `cor` e `posicao`) e várias subclasses concretas como `Circulo`, `Retangulo` e `Triangulo` implementando detalhes específicos. Todas elas são formas geométricas (ou seja, cada uma é um tipo de `FormaGeométrica`), mas cada subclasse representa uma variação particular, aproveitando a estrutura geral definida na superclasse. Essa arquitetura hierárquica torna o design do software mais intuitivo e extensível – novas formas geométricas podem ser adicionadas facilmente herdando de `FormaGeométrica`, reutilizando código existente e mantendo a consistência estrutural.

Em C#, a herança de classes é implementada especificando-se a superclasse após **dois-pontos** na declaração da subclasse. Cada classe em C# pode ter **no máximo uma classe base direta** – ou seja, o C# adota **herança simples** (single inheritance).

Vale lembrar que toda classe em C# herda, ainda que indiretamente, da classe `System.Object`, a raiz de toda a hierarquia de tipos do .NET (se nenhuma base for especificada em uma declaração de classe, `Object` será a superclasse implícita). Como vimos anteriormente, podemos definir uma classe base `Animal` e criar uma classe derivada `Cachorro` que estende `Animal`. No código a seguir, a classe `Animal` define algumas propriedades e comportamentos genéricos, e a classe `Cachorro` herda esses membros e acrescenta funcionalidades específicas:

```
class Animal {
    public string Nome { get; set; }
    public void Comer() {
        Console.WriteLine($"{Nome} está comendo.");
    }
    public virtual void FazerSom() {
        Console.WriteLine("Som de animal genérico");
    }
}
class Cachorro : Animal {
    public void AbanarRabo() {
        Console.WriteLine($"{Nome} está abanando o rabo.");
    }
    public override void FazerSom() {
        Console.WriteLine("Au Au!");
    }
}
```

Nesse exemplo, a classe `Animal` define uma propriedade pública `Nome`, um método público `Comer()` e um método virtual `FazerSom()`. A classe derivada `Cachorro` herda automaticamente a propriedade `Nome` e o método `Comer()` – por isso, dentro de `Cachorro` podemos usar `Nome` (como no método `AbanarRabo()`) e podemos chamar `Comer()` sem precisar reimplementá-los. Além disso, `Cachorro` introduz um novo método específico `AbanarRabo()`, que não existe em `Animal`, e fornece sua própria implementação do método `FazerSom()` usando a palavra-chave **override**. Note que `FazerSom()` foi definido como **virtual** em `Animal`; isso sinaliza que as subclasses podem sobrescrevê-lo. Em `Cachorro`, usamos **override** para redefinir o comportamento

desse método. Dessa forma, quando um **Cachorro** executar **FazerSom()**, a mensagem “**Au Au!**” será exibida, em vez do som genérico definido na classe base.

É importante ressaltar que, se a classe **Animal** não declarasse o método **FazerSom()** como **virtual**, não seria possível sobrescrevê-lo corretamente em **Cachorro** – qualquer tentativa de declarar um método **FazerSom()** em **Cachorro** sem a correspondência **virtual** resultaria em um novo método independente (ocultando o da superclasse). O C# até permite essa ocultação deliberada usando o modificador **new**, mas tal prática não fornece polimorfismo real e geralmente não é recomendada, pois pode confundir a compreensão do código.

Para aproveitar o polimorfismo proporcionado pela herança, os métodos que devem ter comportamento especializado nas subclasses precisam ser marcados como **virtual** na superclasse (ou **abstract**, caso a classe base queira apenas definir a assinatura e exigir implementação pelas derivadas) e então sobrescritos com **override** nas subclasses.

Vale notar também que membros públicos da superclasse permanecem acessíveis como públicos na subclasse; membros declarados como **protected** na superclasse também são herdados e podem ser utilizados dentro da subclasse (como se fizessem parte dela), mas não ficam acessíveis para outras classes fora da hierarquia. Já os membros **private** da superclasse não são visíveis na subclasse – eles existem apenas no escopo interno da própria classe base.

Podemos agora instanciar objetos dessas classes e observar o comportamento em tempo de execução, evidenciando o polimorfismo proporcionado pela herança. No trecho a seguir, criamos um objeto **Animal** genérico e um objeto do tipo **Cachorro**, porém referenciado através de uma variável do tipo **Animal** (isto é, utilizando a referência da superclasse). Chamamos métodos nesses objetos para verificar quais implementações são executadas.

```
Animal animal1 = new Animal();
animal1.Nome = "Genérico";
animal1.FazerSom();    // Saída: Som de animal genérico

Animal animal2 = new Cachorro();
animal2.Nome = "Rex";
animal2.FazerSom();    // Saída: Au Au! (implementação de Cachorro)

// animal2.AbanarRabo(); // ERRO: o método não existe no tipo Animal
((Cachorro)animal2).AbanarRabo(); // Saída: Rex está abanando o rabo.

animal2.Comer();    // Saída: Rex está comendo.
```

Nesse código, **animal1.FazerSom()** produz a mensagem genérica definida em **Animal** (“**Som de animal genérico**”), como era esperado para um objeto do tipo **Animal** comum. Em seguida, instanciamos **animal2** como um novo **Cachorro**, mas o armazenamos em uma variável do tipo **Animal**. Mesmo assim, ao chamar **animal2.FazerSom()**, o resultado impresso é “**Au Au!**” – ou seja, foi executada a versão sobrescrita em **Cachorro**. Isso ilustra o polimorfismo: um mesmo método (**FazerSom**) chama implementações diferentes dependendo do tipo concreto do

objeto (`animal2` é na verdade um `Cachorro`, então usa a implementação de `Cachorro`, apesar de a variável ser do tipo `Animal`).

Por outro lado, se tentarmos chamar `animal2.AbanarRabo()`, o compilador gera um erro, pois a variável `animal2` é do tipo `Animal` e esse método não existe na classe `Animal`. Para acessar comportamentos específicos de `Cachorro`, é preciso tratar o objeto como `Cachorro` – no exemplo, foi feito um casting (`(Cachorro) animal2`) para então invocar `AbanarRabo()`. Após o cast, o método passa a ser acessível e imprime **“Rex está abanando o rabo.”**, conforme definido na subclasse.

Note que chamamos `animal2.Comer()` diretamente, mesmo com `animal2` sendo referenciado como `Animal`. Esse método não foi redefinido em `Cachorro` (não é virtual), então a chamada simplesmente usa a implementação herdada da superclasse, exibindo **“Rex está comendo.”**. Tanto o método `Comer()` quanto a propriedade `Nome` foram herdados de `Animal` e podem ser usados em `Cachorro` normalmente.

Esse exemplo evidencia que, por meio da herança, objetos de subclasses podem ser manipulados através de referências da superclasse e, graças ao polimorfismo, os métodos virtuais invocados agem conforme o tipo real do objeto em tempo de execução.

Outro aspecto fundamental da herança é o fluxo de inicialização de objetos ao construir instâncias de subclasses. Quando criamos um objeto de uma classe derivada, o construtor da classe base é executado antes do construtor da subclasse. Ou seja, a etapa de construção sobe a hierarquia: primeiro se executam os inicializadores e o construtor da superclasse, depois os da classe derivada.

Em C#, se a subclasse não especificar explicitamente qual construtor da superclasse utilizar, o compilador tentará invocar o construtor padrão (sem parâmetros) da classe base. Caso a classe base não possua um construtor sem parâmetros, a subclasse é obrigada a chamar um dos construtores base usando a sintaxe `: base(...)` na declaração de seu construtor. Essa chamada deve ser a primeira instrução do construtor da subclasse.

Por exemplo, poderíamos acrescentar um construtor à classe `Animal` que exige um nome, e na classe `Cachorro` chamar `base(nome)` em seu construtor para garantir que a parte `Animal` do objeto seja inicializada adequadamente. Se `Animal` tivesse um construtor `Animal(string nome)` e `Cachorro` tivesse um construtor `Cachorro(string nome, int idade) : base(nome)`, ao instanciar `new Cachorro("Rex", 5)`, primeiro o construtor de `Animal` receberia "Rex" e executaria sua lógica (por exemplo, definindo o `Nome`), e em seguida o construtor de `Cachorro` executaria a inicialização específica de `Cachorro` (por exemplo, armazenar a idade).

Esse encadeamento garante que toda subestrutura de um objeto derivado seja corretamente inicializada. Se houver vários níveis de herança (por exemplo, uma classe `Gato` que também deriva de `Animal`), o mesmo princípio se aplica recursivamente: os construtores são chamados do nível mais alto na hierarquia ao mais baixo, na ordem.

A herança em C# também está relacionada a conceitos de classes abstratas e interfaces. Uma classe pode ser declarada como `abstract`, indicando que ela não pode ser instanciada diretamente – serve apenas como modelo para subclasses. Classes abstratas podem definir tanto comportamentos padrão (métodos com implementação) quanto comportamentos abstratos que as subclasses devem obrigatoriamente implementar (métodos declarados como `abstract`, sem corpo na classe base).

Essa abordagem é útil quando a superclasse é uma generalização conceitual que não faz sentido instanciar por si só, mas provê uma estrutura comum. No exemplo anterior da hierarquia de formas geométricas, poderíamos declarar `FormaGeométrica` como abstrata e definir `calcularArea()` como um método abstrato, forçando cada subclasse (`Circulo`, `Retangulo` etc.) a fornecer sua própria implementação desse cálculo.

Por contraste, C# também permite especificar que uma classe não pode ser herdada, o que é feito usando o modificador `sealed`. Se uma classe é declarada `sealed`, nenhuma subclassificação dela é permitida. De modo semelhante, um método `virtual` pode ser marcado como `sealed` no momento em que é sobrescrito por uma subclasse, para impedir que subclasses posteriores continuem sobrescrevendo-o.

Esses recursos são úteis quando se deseja fechar uma hierarquia para extensão além de um certo ponto – por exemplo, selar uma classe que já provê todas as funcionalidades requeridas, garantindo que seu comportamento não seja alterado inadvertidamente em subclasses futuras.

Outra forma de polimorfismo em C# são as interfaces, que especificam um conjunto de métodos (e possivelmente propriedades) que uma classe deve implementar, sem fornecer implementação concreta. Diferentemente da herança de classes (também chamada herança de implementação), a implementação de interfaces é às vezes chamada de herança de interface ou subtipagem. Uma classe em C# pode implementar múltiplas interfaces (contornando em parte a limitação de herança simples de classes).

Por exemplo, poderíamos ter uma interface `IAnimado` com um método `Mover()`, e tanto a classe `Cachorro` quanto, digamos, uma classe `Robô` poderiam implementar `IAnimado` para indicar que ambas têm o comportamento de se mover, sem que haja uma relação de herança de classes direta entre elas.

As interfaces permitem compartilhar contratos de comportamento entre classes que não estão na mesma hierarquia de herança de classes. A herança de classes estabelece uma relação forte (uma subclasse é uma especialização da superclasse), enquanto as interfaces estabelecem um contrato que a classe se compromete a cumprir.

Por fim, é importante utilizar a herança de forma criteriosa. Ela deve representar de fato uma relação conceitual é-um entre classes – isto é, a subclasse deve poder ser vista como um tipo especializado da superclasse. Se essa relação não for clara, talvez a herança não seja a ferramenta ideal.

Em muitos casos, pode ser mais adequado usar **composição** (relação **tem-um**), na qual um objeto contém outro como parte de sua estrutura, em vez de herdar de uma classe apenas para reutilizar código.

Por exemplo, um `Carro` tem um `Motor`, portanto faz sentido que `Motor` seja um componente (atributo) dentro da classe `Carro`, em vez de `Carro` herdar de `Motor` – afinal, um carro não é um motor. Inclusive, na literatura de engenharia de software existe o princípio “prefira composição em vez de herança” (Gamma *et al.*, 2005), indicando que se deve avaliar bem quando estender classes.

Herança mal-empregada pode levar a acoplamento excessivo e fragilidade: alterações na superclasse podem impactar indiretamente todas as subclasses. Os próprios autores Gamma *et al.* (2005) observam que a herança é considerada uma forma de reutilização caixa-branca (white-box), pois expõe detalhes internos da superclasse às subclasses, potencialmente violando o encapsulamento se a hierarquia não for bem projetada.

Quando a herança é aplicada corretamente – ou seja, para modelar relações genuínas de especialização –, ela traz enormes benefícios: promove alto grau de reutilização de código, organiza conceitos em hierarquias claras e viabiliza o polimorfismo, permitindo escrever programas mais flexíveis e extensíveis. O pilar da herança, portanto, permanece central na POO, devendo ser compreendido e empregado com boas práticas de projeto em mente. A seguir, detalharemos o quarto pilar, que é o polimorfismo.

2.1.4 Polimorfismo

Em essência, o polimorfismo descreve a capacidade de um mesmo componente ou referência de assumir diferentes formas de comportamento. Ou seja, uma mesma interface ou chamado de método pode resultar em ações diferentes, dependendo do objeto concreto com o qual está trabalhando. Costuma-se resumir essa ideia na frase “uma interface, múltiplas implementações” – indicando que um único conjunto de operações (interface) pode desencadear métodos distintos conforme a classe específica do objeto em questão.

Em termos práticos, isso significa que objetos de classes distintas, mas pertencentes a uma mesma hierarquia ou que implementam o mesmo contrato, podem ser manipulados de forma uniforme, cada um respondendo à mesma mensagem ou chamada de método à sua própria maneira.

Por exemplo, imagine um sistema com várias formas geométricas: círculos, quadrados e estrelas. Todas essas formas poderiam fornecer um método `Desenhar()`; graças ao polimorfismo. Ao enviar a mensagem “desenhar” para qualquer forma, cada uma executa seu próprio desenho específico, apesar de o comando ser conceitualmente o mesmo para todas as formas. Essa versatilidade de comportamentos a partir de uma mesma origem de chamada é exatamente o que define o polimorfismo em POO.

Do ponto de vista conceitual, o polimorfismo pode ocorrer de duas maneiras principais: em tempo de compilação e em tempo de execução. No **polimorfismo em tempo de compilação**, também conhecido como polimorfismo estático ou early binding, a decisão de qual função ou método executar é tomada pelo compilador, antes da execução do programa. Isso é geralmente alcançado por meio de sobrecarga de métodos (method overloading) ou sobrecarga de operadores.

Em C#, por exemplo, uma classe pode ter múltiplos métodos com o mesmo nome, desde que possuam assinaturas diferentes (isto é, diferentes tipos ou quantidade de parâmetros). O compilador distingue qual versão do método chamar analisando os argumentos fornecidos em cada chamada. Assim, métodos sobrecarregados apresentam o mesmo nome, porém comportamentos diferentes de acordo com o contexto de chamada, realizando o princípio de "uma interface, múltiplas implementações" em nível de compilação. Esse é um tipo de polimorfismo *ad hoc*, pois trata casos distintos sob um nome comum, mas não envolve necessariamente hierarquia de classes ou herança.

Para ilustrar o polimorfismo em tempo de compilação, considere a classe `Calculadora` a seguir. Ela define dois métodos chamados `Somar`, um que opera com números inteiros e outro com cadeias de caracteres. Embora compartilhem o mesmo nome, suas funcionalidades diferem de acordo com os parâmetros.

```
using System;

class Calculadora {
    // Método Somar para somar dois números inteiros
    public int Somar(int a, int b) {
        return a + b;
    }

    // Método Somar para "somar" (concatenar) duas strings
    public string Somar(string a, string b) {
        return a + b;
    }
}

class Programa {
    static void Main() {
        Calculadora calc = new Calculadora();
        // Chamando Somar com inteiros - invoca a primeira versão do método
        int resultadoNumerico = calc.Somar(5, 3);
        Console.WriteLine("Soma de inteiros: " + resultadoNumerico); // Saída:
        "Soma de inteiros: 8"

        // Chamando Somar com strings - invoca a segunda versão do método
        string resultadoTexto = calc.Somar("Olá, ", "Mundo!");
        Console.WriteLine("Soma de strings: " + resultadoTexto); // Saída:
        "Soma de strings: Olá, Mundo!"
    }
}
```

Nesse exemplo, a classe `Calculadora` implementa dois métodos `Somar` com assinaturas distintas. O `Somar(int a, int b)` espera dois números inteiros e retorna sua soma aritmética, enquanto o `Somar(string a, string b)` espera duas strings e retorna sua concatenação. O compilador C# é capaz de decidir, em tempo de compilação, qual versão do método chamar com base no tipo dos argumentos fornecidos em cada chamada.

Assim, `calc.Somar(5, 3)` chama o método que lida com inteiros, ao passo que `calc.Somar("Olá, ", "Mundo!")` invoca o método que lida com strings. Embora ambos os métodos

tenham o mesmo nome, o comportamento é adaptado ao tipo de dado – caracterizando o polimorfismo estático (por sobrecarga).

Note que essa decisão é tomada antes da execução do programa: se passamos dois inteiros, já está definido que será usada a versão que soma inteiros; se passamos duas strings, será usada a versão de concatenação. Esse mecanismo torna a interface de uso uniforme (**Somar** pode ser chamado de formas semelhantes), mas com múltiplas implementações internamente adequadas a cada situação. É importante ressaltar que, na sobrecarga, não há escolha em tempo de execução – não há surpresa durante a execução sobre qual método será executado; tudo fica resolvido pelo compilador conforme a assinatura.

O outro tipo de polimorfismo, e geralmente o mais destacado nas discussões, é o **polimorfismo em tempo de execução**, também conhecido como polimorfismo dinâmico ou late binding. Esse é o polimorfismo no sentido mais clássico em orientação a objetos: a habilidade de um objeto de uma classe derivada de ser tratado como um objeto de sua classe base, mas executar automaticamente a versão sobrescrita de um método que for específica de sua classe real.

Em outras palavras, o mesmo método invocado através de uma referência a um tipo base pode ter múltiplas implementações diferentes, escolhidas de acordo com o tipo concreto do objeto em tempo de execução. Aqui, o vínculo entre a chamada do método e o código executado é resolvido tardiamente, durante a execução do programa, quando o tipo real do objeto é conhecido.

Para que o polimorfismo dinâmico funcione, geralmente entram em cena dois conceitos intimamente ligados: herança (ou implementação de interfaces) e sobrescrita de métodos (method overriding). A herança permite definir uma hierarquia de classes em que classes derivadas estendem (herdam) características de uma classe base. Já a sobrescrita permite que uma classe derivada forneça uma implementação específica para um método que foi declarado na classe base.

Em C#, por padrão, os métodos não são polimórficos a menos que sejam marcados explicitamente – isto é, a classe base deve declarar o método com a palavra-chave **virtual** (ou ser um método **abstract**) para indicá-lo como passível de sobrescrita, e a classe derivada usa a palavra-chave **override** para sobrescrever a implementação. Somente então o runtime do C# irá assegurar que, ao invocar esse método numa referência de tipo base, seja executada a versão **override** da classe derivada específica do objeto.

Vamos reformular um exemplo que vimos há pouco neste tópico para ficar mais claro. Imagine uma hierarquia de classes relacionada a animais, na qual a classe base genérica **Animal** define um método **FazerSom()**. Diferentes subclasses de **Animal** – como **Cachorro** e **Gato** – podem sobrescrever esse método para produzir sons distintos (latido, miado etc.). Graças ao polimorfismo, poderemos tratar tanto cães quanto gatos simplesmente como **Animal** na maior parte do programa e, mesmo assim, obter o som específico de cada um quando chamarmos **FazerSom()**. A seguir está a implementação em C# dessa ideia:


```
using System;

// Classe base Animal com um método virtual
class Animal {
    public virtual void FazerSom() {
        Console.WriteLine("Som de animal indefinido.");
    }
}

// Subclasse Cachorro, derivada de Animal, sobrescrevendo o método FazerSom
class Cachorro : Animal {
    public override void FazerSom() {
        Console.WriteLine("O cachorro faz: Au au!");
    }
}

// Subclasse Gato, derivada de Animal, sobrescrevendo o método FazerSom
class Gato : Animal {
    public override void FazerSom() {
        Console.WriteLine("O gato faz: Miau!");
    }
}

class Programa {
    static void Main() {
        // Podemos criar objetos das subclasses:
        Cachorro dog = new Cachorro();
        Gato cat = new Gato();

        // Podemos tratá-los como seu tipo específico:
        dog.FazerSom(); // Saída: "O cachorro faz: Au au!"
        cat.FazerSom(); // Saída: "O gato faz: Miau!"

        // Porém, mais importante, podemos referenciar objetos de subclasses
        usando uma variável do tipo da classe base:
        Animal animal1 = new Cachorro();
        Animal animal2 = new Gato();

        // Chamando o método FazerSom() nas variáveis do tipo base:
        animal1.FazerSom(); // Saída: "O cachorro faz: Au au!"
        animal2.FazerSom(); // Saída: "O gato faz: Miau!"

        // Os objetos ainda são Cachorro e Gato por baixo dos panos, mas o
        programa pode tratá-los genericamente como Animal.
        // Graças ao polimorfismo em tempo de execução, cada chamada invoca a
        implementação correspondente à classe real do objeto.

        // Também é possível armazenar diversos objetos derivados em uma
        estrutura de dados do tipo da classe base:
        Animal[] animais = { new Cachorro(), new Gato() };
        foreach (Animal animal in animais) {
            animal.FazerSom();
            // Para cada elemento, a implementação específica de FazerSom() é
            chamada conforme o tipo real (Cachorro ou Gato).
        }
    }
}
```


Nesse código, definimos uma classe base `Animal` que possui um método `virtual FazerSom()`. Esse método padrão simplesmente imprime “Som de animal indefinido.”, representando um som genérico caso o animal específico não tenha uma implementação própria.

Em seguida, definimos duas subclasses, `Cachorro` e `Gato`, cada uma usando `override` para fornecer sua própria versão de `FazerSom()`. A classe `Cachorro` imprime “O cachorro faz: Au au!”, enquanto `Gato` imprime “O gato faz: Miau!”.

No método `Main`, inicialmente instanciamos `Cachorro` e `Gato` e chamamos `FazerSom()` diretamente nesses objetos, resultando no comportamento esperado para cada espécie.

O aspecto crucial é demonstrado logo após: atribuímos um `Cachorro` a uma variável do tipo `Animal` (`Animal animal1 = new Cachorro()`) e um `Gato` a outra variável do tipo `Animal` (`Animal animal2 = new Gato()`). Aqui, upcasting implícito ocorre, ou seja, estamos tratando objetos de subclasses como se fossem do tipo da superclasse. Isso é permitido porque um `Cachorro` é um `Animal` (relação é-um da herança) e um `Gato` também é um `Animal`.

Em seguida, ao chamar `animal1.FazerSom()`, o programa não executa a versão definida em `Animal`, mas sim a versão sobrescrita dentro de `Cachorro`. De forma análoga, `animal2.FazerSom()` invoca a versão de `Gato`. Em tempo de execução, o sistema verifica o tipo real do objeto referenciado (`animal1` referencia um `Cachorro`) e liga a chamada do método à implementação apropriada dessa classe derivada. Esse mecanismo é o polimorfismo em tempo de execução em ação: a chamada `FazerSom()` na referência do tipo base se traduz dinamicamente no método específico da classe concreta do objeto.

Também exemplificamos o uso de uma coleção (`Animal[]`) para armazenar objetos de diferentes subclasses (`Cachorro` e `Gato`). Ao iterar sobre o array e chamar `animal.FazerSom()` para cada elemento, cada objeto responde de acordo com seu tipo específico.

Assim, sem modificações adicionais no código que realiza as chamadas, podemos adicionar novos tipos de `Animal` – por exemplo, poderíamos criar uma classe `Vaca: Animal` que sobrescrevesse `FazerSom()` para mugir, e o laço `foreach` passaria a invocar automaticamente o mugido para instâncias de `Vaca`, além de continuar invocando latidos para `Cachorro` e miados para `Gato`. O código que usa a classe base não precisa ser alterado para dar suporte a novos tipos de animal, pois ele permanece genérico e, por meio do polimorfismo, consegue integrar novas formas de comportamento. Essa característica torna sistemas orientados a objetos mais extensíveis e mantíveis, já que novas classes podem ser incorporadas com pouco ou nenhum impacto sobre o código existente.

É importante destacar alguns detalhes sobre a implementação de polimorfismo em C#. Primeiro, a palavra-chave `virtual` na classe base é o que permite que o método `FazerSom()` seja sobrescrito. Sem declarar o método como `virtual` (ou sem ser um método abstrato), a classe derivada não pode usar `override`, já que qualquer tentativa de definir um método de mesmo nome na subclasse resultaria em mera ocultação (hiding), e não em sobrescrita polimórfica.

Quando um método não é **virtual**, a ligação da chamada é sempre estática: o método chamado corresponde estritamente ao tipo da referência em tempo de compilação, e não ao tipo do objeto em tempo de execução.

No nosso exemplo, se **FazerSom()** não fosse **virtual** em **Animal**, então **animal1.FazerSom()** chamaria sempre a versão definida em **Animal**, mesmo que **animal1** referencie um **Cachorro**. Isso obviamente quebraria o comportamento polimórfico desejado.

Em C#, é possível forçar essa ocultação explícita usando a palavra-chave **new** na declaração do método na subclasse, mas esse é um caso especial no qual deliberadamente se opta por não participar do mecanismo de polimorfismo dinâmico, algo raramente recomendado para designs orientados a objetos claros.

Por outro lado, quando um método é marcado como **virtual** e corretamente sobrescrito com **override**, o C# garante a chamada dinâmica correta. Todo objeto em C# carrega consigo informação sobre seu tipo real e uma tabela de métodos virtuais (um conceito conhecido como **v-table** em linguagens de baixo nível) que mapeia chamadas de métodos virtuais para as implementações apropriadas de acordo com a classe.

Assim, a invocação de **animal1.FazerSom()** consulta essa tabela e descobre que a instância atual, sendo do tipo **Cachorro**, deve usar a versão de **Cachorro**. Esse processo ocorre de forma transparente para o programador.

Outro conceito relacionado é o de métodos abstratos e classes abstratas. Em nosso exemplo, a classe **Animal** forneceu uma implementação padrão (ainda que genérica) de **FazerSom()**. Contudo, em muitos cenários, faz sentido que a classe base não tenha uma implementação concreta, forçando todas as subclasses a definirem a sua.

Podemos tornar **Animal** uma classe abstrata e **FazerSom()** um método abstrato, removendo seu corpo e marcando-o com **abstract**. Dessa maneira, nenhuma instância de **Animal** pode ser criada e garantimos que cada tipo de animal concreto implemente seu som característico.

O polimorfismo permanece funcionando do mesmo modo: se **Animal** fosse uma classe abstrata, **Animal animal = new Cachorro(); animal.FazerSom();** ainda chamaria a versão de **Cachorro**. A diferença é que agora não existe um comportamento padrão na superclasse; cada subclasse deve prover o seu. Dessa forma, a decisão de usar ou não métodos abstratos depende do contexto: se faz sentido um comportamento genérico na classe base, utilize método virtual com implementação default; se não, opte por abstrato para forçar a especialização total.

Além da herança de classes, interfaces também proporcionam polimorfismo em C#. Como vimos anteriormente, as interfaces definem um contrato (um conjunto de métodos/propriedades) que diversas classes podem implementar, mesmo que não estejam relacionadas por herança.

Por exemplo, poderíamos ter uma interface `IApresentacao` com um método `Apresentar()` e fazer com que classes distintas (como `Aluno`, `Professor`, `Empresa` etc.) implementassem essa interface. Assim, poderíamos tratar instâncias de `Aluno`, `Professor` ou `Empresa` de forma polimórfica através de uma variável do tipo `IApresentacao`: ao chamar `obj.Apresentar()`, cada objeto executaria sua própria lógica (um aluno poderia dizer seu nome e curso, um professor poderia dizer seu nome e disciplina etc.), sem que o código que faz a chamada precise saber qual é o tipo concreto de cada objeto.

Esse é o lema "uma interface, múltiplas implementações" em ação. Vale notar que aqui usamos interface tanto no sentido do conceito abstrato quanto no sentido da construção da linguagem C# chamada interface.

O mecanismo subjacente é similar: trata-se de invocar por meio de um tipo por referência genérico (a interface ou a classe base abstrata) e ter a execução determinada pelo tipo concreto.

As interfaces são particularmente úteis para permitir polimorfismo entre classes que não têm ancestral comum direto, já que uma classe em C# só pode herdar de uma classe base, mas pode implementar múltiplas interfaces. Assim, interfaces ampliam as possibilidades de polimorfismo, fomentando código fortemente baseado em contratos em vez de em tipos específicos.

Um dos principais benefícios do polimorfismo é a extensibilidade já mencionada: podemos adicionar novas subclasses ou novas classes implementando uma interface sem precisar modificar o código existente que interage com o tipo base ou a interface.

O código cliente pode confiar no contrato (métodos expostos pela classe base ou pela interface) e não se preocupar com as particularidades de cada implementação. Esse fato se relaciona com o princípio do aberto/fechado (open/closed principle) dos princípios SOLID, no qual classes devem estar abertas para extensão, mas fechadas para modificação.

O princípio do aberto/fechado (open/closed principle) aparece aqui como a justificativa conceitual para o ganho de extensibilidade obtido ao programar contra uma abstração (uma interface ou uma classe base abstrata) em vez de programar contra tipos concretos.

A ideia, formulada originalmente por Bertrand Meyer no contexto do projeto orientado a objetos, é que entidades de software como módulos, classes e funções devem ser abertas para extensão no sentido de poderem receber novos comportamentos por meio da adição de novas variantes, e fechadas para modificação no sentido de que o código já estabilizado – sobretudo o código cliente e as abstrações publicadas como contratos – não deveria precisar ser reescrito a cada novo requisito.

No cenário descrito, esse fechamento para modificação é obtido quando o código cliente depende somente do contrato expresso pela interface: ele conhece quais operações existem e quais obrigações semânticas essas operações carregam, porém não depende de como cada classe concreta realiza o trabalho.

Assim, quando surge uma nova necessidade, a mudança principal tende a ocorrer por extensão com a criação de uma nova classe que implementa a interface (ou herda da base abstrata) e passa a ser injetada, configurada ou selecionada em tempo de execução, sem que o cliente que chama o contrato precise ser alterado.

O polimorfismo permite exatamente isso: estender o comportamento via novas classes, sem alterar o código que já foi escrito para usar o sistema. Ele também está ligado ao princípio da substituição de Liskov (LSP, do inglês Liskov substitution principle), que informalmente diz que objetos de classes derivadas devem poder substituir objetos da classe base sem quebrar a corretude do programa. Em outras palavras, onde quer que se espere um **Animal**, um **Cachorro** ou um **Gato** devem funcionar perfeitamente, respeitando as expectativas da interface da classe base. Se as subclasses seguirem esse contrato, o código polimórfico funcionará de forma segura e coerente.

Outro benefício é a redução de código duplicado e de condicionais extensos. Sem polimorfismo, frequentemente veríamos códigos assim:

```
if (obj is Cachorro)
    Latir(obj);
else if (obj is Gato)
    Miar(obj);
else if (obj is Vaca)
    Mugir(obj);
```

Essa abordagem de usar condicionais baseadas em tipo pode ficar rapidamente fora de controle conforme novos tipos são adicionados, tornando o código difícil de manter. Com polimorfismo, esse mesmo cenário se simplifica para:

```
obj.FazerSom();
```

Nessa ocorrência, cada objeto sabe fazer o som apropriado a si mesmo e novos tipos podem ser adicionados sem a necessidade de mexer nesse trecho de código. Assim, o código fica mais limpo, coeso e alinhado com os princípios da OO.

Convém notar que o polimorfismo em tempo de execução tem um pequeno custo de desempenho em comparação com chamadas diretas (ligação estática), pois o sistema faz uma resolução dinâmica da chamada. Contudo, esse custo é normalmente insignificante diante dos benefícios de design, e os compiladores modernos e runtimes como o .NET otimizam bastante essas chamadas virtuais.

Em aplicações de negócio e sistemas em geral, **privilegiam-se a clareza e a extensibilidade fornecidas pelo polimorfismo, usando-o abundantemente**. Somente em cenários de desempenho crítico (como dentro de loops extremamente intensivos) pode-se considerar evitar chamadas virtuais – e, ainda assim, existem técnicas de otimização, e o próprio JIT do .NET pode colocar métodos virtuais em linha em alguns casos, quando fica evidente o tipo concreto.

Sintetizando, o polimorfismo permite que um código geral manipule objetos de formas específicas de maneira transparente. É graças a ele que bibliotecas e frameworks orientados a objetos conseguem oferecer APIs flexíveis. Por exemplo, a plataforma .NET possui inúmeras classes que herdam de uma mesma classe base ou implementam interfaces comuns, possibilitando que estruturas de dados, algoritmos e serviços operem sobre elas indiferenciadamente. Sem polimorfismo, perderíamos grande parte da elegância e do reúso de código característicos da POO.

Em conclusão, o polimorfismo é a capacidade de um objeto de se apresentar de várias formas através de um ponto comum de interação. Seja via sobrecarga (em tempo de compilação) ou via sobrescrita (em tempo de execução), ele proporciona meios de tratar entidades distintas de maneira uniforme, delegando às próprias entidades o comportamento específico.

Tais características são essenciais para construir sistemas OO robustos, modulares e fáceis de expandir. Conforme vimos, em C# a implementação concreta do polimorfismo dinâmico envolve declarações de métodos virtuais na classe base e overrides nas subclasses, ou a utilização de interfaces, garantindo que o método apropriado seja invocado conforme o tipo real do objeto.

O polimorfismo forma, juntamente com a herança, a abstração e o encapsulamento, a espinha dorsal do paradigma orientado a objetos, possibilitando modelar o mundo real de modo intuitivo no código e promover soluções de software flexíveis e evolutivas.



Saiba mais

O livro a seguir é uma ótima referência focada diretamente em POO com C#, abordando associações, herança, polimorfismo, acesso a banco de dados, uso de Visual Studio e uma aplicação em camadas como exemplo de arquitetura.

ARAÚJO, E. C. *Orientação a objetos em C#: conceitos e implementações em .NET*. São Paulo: Casa do Código, 2017.

Já a obra seguinte discorre sobre a versão mais recente da linguagem e do runtime, cobrindo desde os fundamentos até o desenvolvimento web. Há capítulos específicos sobre POO, o que ajuda a consolidar conceitos como classes, herança, interfaces e boas práticas em C#.

PRICE, M. J. *C# 14 and .NET 10: modern cross-platform development fundamentals – build modern websites and services with ASP.NET Core, Blazor, and EF Core using Visual Studio 2026*. 10. ed. Birmingham: Packt Publishing, 2025.

2.2 Encapsulamento na prática: propriedades, invariantes, imutabilidade com record

Conforme apresentamos, o encapsulamento é um dos pilares da POO, caracterizado pela capacidade de esconder os detalhes internos de implementação de um objeto e expor apenas uma interface segura para uso externo. Em essência, o encapsulamento visa proteger os dados de uma classe contra acessos indevidos ou inconsistentes, permitindo que apenas métodos ou propriedades controlados manipulem seu estado.

Um erro comum de iniciante é presumir que tornar um atributo público seria mais simples; contudo, expor variáveis de instância diretamente viola o princípio de ocultação de informações e pode levar a estados inválidos no objeto.

Por exemplo, uma variável pública pode ser lida ou escrita livremente por qualquer parte do programa, sem controle. Já uma variável privada só pode ser acessada indiretamente, através de métodos ou propriedades fornecidos pela classe. Dessa forma, a classe controla rigorosamente de que maneira seus dados são obtidos ou alterados.

Deitel e Deitel (2016) enfatizam que, mesmo permitindo a manipulação de dados privados, as propriedades cuidadosamente controlam essas manipulações e mantêm os dados encapsulados com segurança dentro do objeto – algo impossível de garantir com variáveis de instância públicas, que poderiam receber valores inválidos livremente.

Em resumo, o encapsulamento fornece uma barreira de proteção em torno do estado do objeto, evitando acessos não autorizados e possibilitando impor regras para alterações.

No contexto do C#, a implementação prática do encapsulamento costuma ser feita com propriedades (properties). **Propriedades são membros da classe** que funcionam como interfaces de acesso a campos privados: elas expõem metodologias de leitura (`get`) e escrita (`set`) controladas, muitas vezes substituindo os antigos métodos getters e setters manuais das linguagens orientadas a objetos clássicas. Apesar de, à primeira vista, criar uma propriedade pública com `get/set` parecer equivalente a ter um campo público, há uma diferença fundamental: a lógica interna do `get/set` permite validar, converter ou restringir o acesso aos dados antes de efetivar qualquer mudança.

Por exemplo, imagine uma classe que armazena internamente o horário em segundos (um `int` privado). Podemos expor esse dado por meio de uma propriedade pública `Time` do tipo `string`. O acessor `get` da propriedade poderia converter os segundos armazenados em um formato de hora legível "HH:MM:SS", enquanto o acessor `set` aceitaria uma `string` nesse formato, converteria tal `string` para segundos e a armazenaria no campo privado.

Durante esse processo, é possível incluir validações: ao receber uma `string`, o `set` pode checar se ela representa um horário válido – por exemplo, "12:30:45" é válido, mas "42:85:70" não – antes de atualizar o campo. Caso a validação falhe, a propriedade poderia lançar uma exceção ou ignorar a atribuição, impedindo que o objeto entre em um estado incorreto.

Assim, mesmo para o código cliente que vê um atributo `Time` aparentemente público, internamente, os acessores estão garantindo que qualquer interação ocorra de forma controlada e segura. Os dados privados permanecem protegidos (encapsulados) e consistentes dentro do objeto.

Um exemplo simples em C# ajuda a ilustrar o uso de propriedades para encapsulamento e manutenção de regras de negócio (invariantes). Considere uma classe `Pessoa` que deve armazenar a idade de um indivíduo. Uma restrição lógica natural (**invariante**) é que a idade não pode ser negativa. Para impor isso, declara-se o campo de dados como privado e expõe-se uma propriedade pública com lógica de validação no `set`:

```
public class Pessoa
{
    private int idade; // campo encapsulado

    public int Idade
    {
        get { return idade; }
        set
        {
            if (value < 0)
                throw new ArgumentOutOfRangeException("Idade não pode ser negativa.");
            idade = value;
        }
    }
}
```

Nesse código, qualquer tentativa de atribuir um valor negativo a `Idade` resultará em uma exceção, impedindo a violação do invariante de que `idade ≥ 0`. O uso da propriedade faz com que códigos externos manipulem a idade apenas através do nosso filtro (o acessor `set`), mantendo o objeto `Pessoa` sempre em um estado válido. Caso `idade` fosse um campo público acessado diretamente, nada impediria atribuições inválidas como `objPessoa.idade = -5`, possivelmente propagando erros difíceis de rastrear.

Com a propriedade, o encapsulamento garante que toda mudança de estado passe por uma verificação central, facilitando a manutenção e a evolução. Por exemplo, se futuramente decidirmos representar a idade em meses em vez de anos, podemos alterar a implementação interna mantendo a propriedade `Idade` – o código cliente continuaria usando `Idade` da mesma forma, sem ser afetado pela mudança interna.

Essa flexibilidade evidencia outro benefício do encapsulamento: baixo acoplamento entre a classe e quem a utiliza, pois detalhes internos podem mudar sem quebrar o contrato externo (desde que a interface – no caso, a propriedade – permaneça). De fato, recomenda-se que até mesmo métodos internos de uma classe acessem seus próprios campos privados via propriedades, em vez de diretamente, se tais propriedades implementarem lógicas importantes.

Essa prática torna a classe mais robusta e fácil de manter, pois qualquer lógica especial (como validações ou formatações) fica concentrada nos acessores, reduzindo chances de inconsistências espalhadas pelo código.

A necessidade de garantir que um objeto nunca viole certas condições lógicas traz o conceito formal de invariante de classe. Em termos simples, um invariante de classe é uma condição que deve ser verdadeira para todo objeto válido daquela classe em qualquer momento observável externamente.

Segundo Horstmann (2006), o invariante deve valer antes e depois de cada chamada a métodos públicos do objeto, podendo ser quebrado temporariamente apenas durante a execução interna de um método (enquanto o objeto ajusta seu estado), desde que restaurado ao final da operação.

No exemplo da classe **Pessoa**, a condição `idade ≥ 0` é um invariante: sempre que observamos um objeto **Pessoa** do mundo externo, esperamos que essa condição esteja mantida. Caso algum método interno precise alterar a idade (por exemplo, reduzir a idade em um ano – o que não faria sentido aqui, mas suponhamos um ajuste), ele poderia, durante parte de sua execução, trabalhar com um valor intermediário inválido, contanto que, ao término, a idade do objeto volte a satisfazer `idade ≥ 0`.

A chave para estabelecer invariantes confiáveis é justamente o encapsulamento: ao restringir o acesso direto aos campos e centralizar as modificações em pontos controlados (propriedades ou métodos), a classe pode assegurar que todas as transições de estado respeitem as regras definidas.

Como observa Horstmann (2006), mantendo os campos de instância privados, temos controle total sobre todas as operações que os modificam – possibilitando garantir, por exemplo, que valores permaneçam sempre dentro de faixas válidas ou que determinadas referências nunca sejam nulas.

Os invariantes se tornam a ferramenta apropriada para documentar e verificar essas garantias lógicas do programa. Em outras palavras, se o encapsulamento é o mecanismo que impede interferências externas indevidas, os invariantes representam as regras internas que devem sempre ser obedecidas para a consistência do objeto.

Em linguagens de programação com suporte a design by contract (como Eiffel), invariantes de classe podem até ser especificados explicitamente e verificados automaticamente. Em C#, não há um mecanismo nativo de invariantes embutido na linguagem, mas é possível emular essa ideia utilizando, por exemplo, as classes de code contracts (contratos de código) ou simplesmente assegurando manualmente, via código, que cada método público preserve os invariantes.

A propriedade `Idade` é justamente uma implementação manual de garantia de invariante. Podemos dizer que, por meio do encapsulamento e dessas verificações, o objeto defende suas próprias condições de validade. Vale ressaltar que invariantes devem ser estabelecidos já no término do construtor (ou seja, logo após a criação do objeto, ele já deve estar consistente) e mantidos por todos os métodos públicos subsequentes.

Isso significa que o construtor de `Pessoa` deveria rejeitar (ou ajustar) qualquer valor inválido inicial de idade, garantindo que nenhum objeto seja "nascido" em estado incorreto. Após o objeto ser construído corretamente, a propriedade `Idade` cuidará de evitar violações futuras desse estado.

Até este ponto discutimos encapsulamento controlando modificações mutáveis no estado do objeto. Entretanto, uma estratégia ainda mais rigorosa para garantir invariantes e simplificar o raciocínio sobre o programa é projetar objetos imutáveis.

Uma classe é considerada imutável quando suas instâncias, depois de criadas, não permitem alteração de estado, logo, todas as informações contidas em cada objeto ficam fixas durante toda a sua vida. Em vez de modificar o objeto, qualquer mudança conceitual é obtida criando-se um novo objeto com o novo valor desejado.

Essa ideia, que tem raízes profundas na programação funcional, traz diversos benefícios: objetos imutáveis são intrinsecamente thread-safe (podem ser compartilhados entre threads sem risco de condições de corrida, já que nada muda), não precisam de cópias defensivas e jamais apresentam inconsistência temporária – não há um estado intermediário inválido, pois o objeto simplesmente não muda após construído. Em outras palavras, se um objeto inicia sua vida obedecendo a todos os invariantes, ele permanecerá obedecendo, pois não há operações que o levem a um estado diferente.

Como destaca Bloch (2018), as classes deveriam ser imutáveis a menos que haja uma razão muito boa para serem mutáveis. Ele sugere um conjunto de regras simples para criar uma classe imutável: não fornecer métodos que modifiquem o estado, declarar todos os campos como `private` e `final` (em C#, equivalentes seriam campos `private readonly`) e garantir acesso exclusivo a quaisquer componentes mutáveis internos (por exemplo, fazendo cópias defensivas). Além disso, todo objeto imutável deve ser completamente inicializado no fim do construtor, com todos os seus invariantes estabelecidos de saída.

A linguagem C# incorporou muitas dessas boas práticas. Antes mesmo de suportar nativamente tipos imutáveis personalizados, já oferecia mecanismos para facilitar a criação de objetos cujo estado não muda após a construção. Por exemplo, podemos declarar propriedades como somente leitura omitindo o acessor `set` ou marcando-o como `private`. Também existe o modificador `readonly` para campos, que assegura que o campo só recebe valor uma vez (no construtor ou na declaração).

Reconhecendo esse padrão comum, o C# 9 introduziu uma funcionalidade específica para facilitar a criação de objetos imutáveis: o `record`. Records são um tipo de referência especial em C# projetado para trabalhar bem com dados imutáveis (de apenas leitura).

Em termos simples, ao declarar um `record`, o compilador gera automaticamente grande parte do código cerimonial necessário para uma classe imutável, incluindo: propriedades `init-only` (que podem ser atribuídas apenas durante a criação do objeto), um método `Equals` e `GetHashCode` baseado no conteúdo (implementando igualdade estrutural em vez de referência), e até um método `Deconstruct` para fácil desempacotamento dos dados.

Por exemplo, podemos definir um record assim:

```
public record Pessoa(string Nome, int Idade);
```

Em uma única linha, criamos uma classe imutável **Pessoa** com duas propriedades (**Nome** e **Idade**). O compilador gerará uma propriedade **public string Nome { get; init; }** e uma propriedade **public int Idade { get; init; }**, além de um construtor que recebe **Nome** e **Idade** como parâmetros. As propriedades marcadas com **init** só podem ser atribuídas durante a construção do objeto (seja no construtor ou em um inicializador de objeto logo depois), mas tornam-se somente leitura após o objeto estar pronto.

Assim, **Pessoa p = new Pessoa("Ana", 30);** cria um objeto cuja propriedade **Idade** vale 30, e nenhuma parte do código poderá alterá-la depois – não existe método **set** público. Qualquer tentativa de **p.Idade = 31** resultaria em erro de compilação, garantindo imutabilidade.

Se quisermos modificar a idade, a abordagem será criar um novo objeto derivado do existente, processo que o C# torna simples com a expressão **with**. Por exemplo, **Pessoa p2 = p with { Idade = 31 };** cria um novo objeto **Pessoa** com o mesmo **Nome** de **p**, mas a **Idade** alterada para 31, mantendo **p** original intocado. Esse é o chamado *nondestructive mutation* (mutação não destrutiva), considerado o recurso mais útil dos records, pois automatiza o processo de copiar os dados imutáveis existentes para um novo objeto alterando apenas o necessário.

Internamente, o compilador gera para isso um construtor de cópia protegido e implementa o operador **with** para que realize a cópia dos campos.

Outra vantagem fornecida pelos records é que, por serem destinados a valores imutáveis, eles adotam por padrão a igualdade estrutural. Isso significa que dois objetos **record** do mesmo tipo são considerados iguais (**Equals** retorna **true**) se todos os seus campos públicos tiverem valores iguais.

Essa semântica de igualdade é normalmente desejável para objetos imutáveis que representam, por exemplo, entidades de valor (como datas, coordenadas etc.), pois queremos compará-los pelo conteúdo e não pela posição na memória.

Em classes regulares, o programador precisaria sobrescrever manualmente **Equals** e **GetHashCode** para obter esse comportamento; nos records, ele vem pronto "de fábrica". De fato, um record em C# é conceitualmente semelhante a uma tupla nomeada (ideal para simplesmente agrupar dados), mas com a segurança de tipo e documentação própria de uma classe, além de permitir métodos e outros recursos típicos da POO.

É importante frisar que, embora os records tornem a criação de objetos imutáveis muito mais concisa, nada impede que sejam usados de forma mutável, por exemplo, você pode deliberadamente incluir um campo ou uma propriedade com **set** tradicional dentro de um record, se necessário. Entretanto, segundo a cultura de uso do recurso e a intenção original de design do C# 9+, os records devem ser utilizados majoritariamente para modelar dados imutáveis.

As propriedades `init-only` introduzidas concomitantemente reforçam esse propósito, pois até classes normais podem usá-las para conseguir um efeito de `setup-only`, que garante que o valor seja definido durante a inicialização (no construtor ou inicializador) e não podendo ser alterado posteriormente.

Em suma, C# provê ao desenvolvedor tanto a abordagem clássica de encapsulamento com propriedades (para objetos mutáveis que controlam cuidadosamente seu estado) quanto uma abordagem de criar objetos intrinsecamente imutáveis que evitam mudanças de estado por completo.

Ambas servem ao objetivo maior de manter invariantes e garantir a correção do programa, mas cada qual é aplicada em cenários diferentes: objetos que representam entidades do mundo real que mudam ao longo do tempo (por exemplo, um objeto `ContaBancaria` cujo saldo varia) naturalmente utilizam propriedades com lógica de encapsulamento; já objetos que representam valores ou snapshots (por exemplo, uma `Data` ou os dados de uma transferência bancária realizada) podem ser mais bem modelados como imutáveis, facilitando o raciocínio e evitando erros.

Do ponto de vista didático, é útil entender encapsulamento, invariantes e imutabilidade como partes de um mesmo esforço: o de controlar e preservar o estado válido dos objetos. Encapsular com propriedades nos permite interceptar e validar alterações de estado, enquanto definir objetos imutáveis elimina a possibilidade de alteração após a criação. Em ambos os casos, estamos evitando que o objeto assuma configurações inválidas.

Ao aplicar essas técnicas em conjunto – por exemplo, encapsulando bem os campos mutáveis e preferindo imutabilidade quando apropriado –, obtemos sistemas mais robustos e confiáveis. Conforme afirmam Deitel e Deitel (2016), as propriedades bem projetadas mantêm os dados encapsulados com segurança dentro do objeto, e Horstmann (2006) complementa que invariantes documentam garantias essenciais de correção do programa, possíveis graças ao encapsulamento dos dados. Já Bloch (2018) ressalta que objetos imutáveis simplificam enormemente o desenvolvimento, evitando uma série de bugs e tornando o comportamento mais previsível.

De fato, criar tipos imutáveis é uma estratégia crescente para simplificar software e reduzir erros, além de ser um princípio adotado diretamente na plataforma .NET através dos records e das diversas estruturas imutáveis da biblioteca padrão (como `System.String`, que sempre foi imutável).

Assim, entender e aplicar o encapsulamento na prática – seja via propriedades monitorando invariantes, seja projetando objetos imutáveis – é fundamental para construir programas orientados a objetos corretos, claros e de fácil manutenção, nos quais cada classe protege sua integridade e expõe apenas o que for consistente e seguro. Encapsular é proteger, e essa proteção é a base sobre a qual se constroem programas confiáveis.



Resumo

Esta unidade apresentou o C# como a linguagem central do ecossistema .NET 10 e descreveu por que ela é uma escolha estratégica para quem está começando a programar. Detalhamos os fundamentos da sintaxe de C#, o uso de ponto e vírgula para encerrar instruções e de chaves para delimitar blocos, estruturas de controle como `if`, `switch`, `for`, `while` e `foreach`, e a sensibilidade a maiúsculas e minúsculas. A partir daí, o primeiro tópico aprofundou o sistema de tipos – os tipos primitivos (`int`, `double`, `decimal`, `char`, `bool`, `string`, `object`) – distinguiu tipos por valor de tipos por referência e relacionou essa diferença com o modo como os dados são armazenados em `stack` e `heap` e geridos pelo GC.

Dedicamos parte da unidade à nulabilidade. Mostramos primeiro os tipos por valor anuláveis (como `int?`) e depois os tipos por referência anuláveis introduzidos nas versões modernas da linguagem, explicando como o compilador passa a diferenciar, por exemplo, `string` e `string?`. Em paralelo, discutimos a declaração de variáveis e o uso de `var` para inferência de tipo, nos levando a esclarecer que `var` não torna o código dinâmico nem sem tipo, apenas permite que o compilador deduza o tipo a partir da expressão à direita.

Explicamos, também, a estrutura de soluções (`.sln`) e projetos (`.csproj`), a separação em pastas como `src`, `tests`, `samples`, `build` e `docs` e o uso de arquivos SDK-style `.csproj`. Em seguida, apresentamos o Visual Studio 2026, suas edições (Community, Professional, Enterprise) e seus pacotes auxiliares, como `Insiders`, `Team Explorer`, `Remote Tools` e `Build Tools`, com foco em como eles se encaixam em cenários de estudo, equipes pequenas e ambientes corporativos. Há um passo a passo de criação de um aplicativo de console na IDE, colagem do código de exemplo e execução, integrando a teoria à prática.

Complementarmente, falamos sobre o `dotnet CLI` como interface de linha de comando multiplataforma do .NET, explicando os comandos `dotnet new`, `build`, `run`, `test`, `publish` e `restore`, o papel do `MSBuild` nos bastidores, a restauração de pacotes `NuGet` e o uso desses comandos em pipelines de integração e entrega contínuas em servidores de build.

Por fim, conectamos tudo isso à POO em C#: definimos classes, objetos e construtores e apresentamos os quatro pilares (encapsulamento, abstração, herança e polimorfismo) como base conceitual robusta, além de diversos exemplos. O encapsulamento e a abstração são discutidos em mais detalhe,

relacionando modificadores de acesso, propriedades e interfaces ao controle de complexidade e à estabilidade das interfaces públicas, preparando o terreno para aprofundar a herança e o polimorfismo nos tópicos seguintes.



Exercícios

Questão 1. Um desenvolvedor está depurando um programa C#. Ele declara `int a = 10; int b = a;` e depois altera `b`. Em outra parte, ele faz `Pessoa p1 = new Pessoa(); Pessoa p2 = p1;` e altera um campo por `p2`. Além disso, em certo momento, ele tenta acessar `mensagem.Length` quando a mensagem vale `null` e recebe uma exceção. Considerando tipos por valor, tipos por referência e nulabilidade em C#, qual alternativa é a correta?

- A) Em atribuições, C# sempre copia o conteúdo completo. Por isso, `p2 = p1` cria automaticamente um novo objeto independente no heap.
- B) Em tipos por valor, a atribuição copia o valor. Em tipos por referência, a atribuição copia a referência, e duas variáveis podem apontar para o mesmo objeto. Ao acessar um membro de uma referência nula, pode ocorrer `System.NullReferenceException`.
- C) Um `int` sempre aceita `null` por padrão, sem necessidade de sintaxe especial.
- D) Ao ativar nullable reference types, o runtime passa a impedir exceções de referência nula automaticamente, sem depender do código.
- E) A palavra-chave `var` cria uma variável de tipo dinâmico, que pode mudar de tipo durante a execução.

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: em `p2 = p1`, não há cópia automática do objeto. Há cópia da referência, de modo que ambas as variáveis podem referenciar a mesma instância.

B) Alternativa correta.

Justificativa: tipos por valor (como `int`) tendem a ser copiados em atribuições. Tipos por referência (como instâncias de classes) copiam a referência, permitindo o compartilhamento do mesmo objeto. Se uma variável de referência estiver `null` e o código tentar acessar um membro, poderá ocorrer `System.NullReferenceException`.

C) Alternativa incorreta.

Justificativa: tipos por valor não admitem `null` por padrão. Para isso, existe a forma anulável, como `int?`, que corresponde a `Nullable<int>`.

D) Alternativa incorreta.

Justificativa: nullable reference types introduzem principalmente avisos/análises em tempo de compilação para reduzir o risco. Eles não mudam o tipo em runtime nem bloqueiam exceções por si só.

E) Alternativa incorreta.

Justificativa: a indicação `var` faz inferência de tipo em tempo de compilação. O tipo fica fixo e não vira dinâmico nem muda durante a execução.

Questão 2. Em um sistema de contas, uma equipe modelou a classe `ContaBancaria` com um campo `public decimal Saldo;` para simplificar o código. Em testes, surgiram estados inválidos (por exemplo, saldo negativo após uma subtração direta). Considerando o encapsulamento, o ocultamento de informações e a ideia de invariantes de classe, qual alternativa descreve a correção de design mais consistente?

- A) Manter o campo de saldo público e pedir que todo código cliente faça validações antes de alterar o valor, pois isso mantém a classe menor e delega as regras de negócio para fora.
- B) Tornar o campo de saldo privado e expor uma propriedade pública com leitura, restringindo a escrita, além de oferecer métodos públicos (por exemplo, `Depositar` e `Sacar`) com validações que impeçam quantias inválidas e saques acima do saldo.
- C) Tornar o campo de saldo `protected` para que quaisquer subclasses e partes do sistema possam modificá-lo livremente, evitando a necessidade de métodos e propriedades adicionais.
- D) Manter o saldo público, mas criar um método `Validar()` que pode ser chamado eventualmente, pois, se o saldo ficar incorreto, o método corrige o estado depois.
- E) Trocar a classe por um record com propriedades mutáveis (`set` público), pois records resolvem automaticamente a integridade do estado e impedem estados inválidos sem validações.

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: ao exigir que regras fiquem espalhadas em vários pontos do código cliente, a classe deixa de controlar seu próprio estado e seus invariantes, aumentando o acoplamento e tornando mais provável que algum ponto esqueça a validação.

